

LAPORAN AKHIR PRAKTIKUM PEMROGRAMAN MOBILE



Oleh:

Dina Izzati Elfadheya NIM 2310817120001

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
2025**

LEMBAR PENGESAHAN

LAPORAN AKHIR PRAKTIKUM PEMROGRAMAN MOBILE

Laporan Praktikum Pemrograman Mobile

Modul 1 : *Android Basic With Kotlin*

Modul 2 : *Android Layout*

Modul 3 : *Build A Scrollable List*

Modul 4 : Viewmodel Dan Debugging

Modul 5 : *Connect To Internet*

ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile.

Laporan Akhir Praktikum ini dikerjakan oleh:

Nama Praktikan : Dina Izzati Elfadheya

NIM : 2010817320006

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Zulfa Auliya Akbar
NIM. 2210817210026

Muti`a Maulida S.Kom M.T.I
NIP. 19881027 201903 20 13

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI	3
DAFTAR GAMBAR	5
DAFTAR TABEL	7
MODUL 1: ANDROID BASIC WITH KOTLIN	8
SOAL 1	10
A. Source Code	12
B. Output Program	15
C. Pembahasan	17
MODUL 2: ANDROID LAYOUT	20
SOAL 1	20
A. Source Code	22
B. Output Program	29
C. Pembahasan	33
MODUL 3: BUILD A SCROLLABLE LIST	39
SOAL 1	39
A. Source Code	40
B. Output Program	56
C. Pembahasan	59
SOAL 2	67
A. Jawaban	67
MODUL 4: VIEWMODEL DAN DEBUGGING	66
SOAL 1	68
A. Source Code	69
B. Output Program	88
C. Pembahasan	93
SOAL 2	107
A. Jawaban	107
MODUL 5: CONNECT TO INTERNET	109
SOAL 1	109
A. Source Code	109

B. Output Program	150
C. Pembahasan	156
TAUTAN GIT	190

DAFTAR GAMBAR

MODUL 1: ANDROID BASIC WITH KOTLIN

Gambar 1 Soal Tampilan Awal Aplikasi Modul 1.....	10
Gambar 2 Tampilan Dadu Setelah Di Roll Modul 1.....	11
Gambar 3 Soal Contoh Tampilan Roll Dadu Double Modul.....	11
Gambar 4 Screenshot Hasil Jawaban Modul 1	15
Gambar 5 Screenshot Hasil Jawaban Modul 1	16
Gambar 6 Screenshot Hasil Jawaban Modul 1	16

MODUL 2: ANDROID LAYOUT

Gambar 7 Contoh Tampilan Awal Aplikasi Modul 2.....	20
Gambar 8 Contoh Tampilan Aplikasi Setelah Dijalankan Modul 2	21
Gambar 9 Screenshot Hasil Tampilan SplashScreen Modul 2	29
Gambar 10 Screenshot Hasil Tampilan Awal Modul 2	30
Gambar 11 Screenshot Hasil Tampilan Aplikasi Setelah Dijalankan Modul 2	31
Gambar 12 Screenshot Hasil Tampilan Aplikasi Jika Input Salah Modul 2.....	32

MODUL 3: BUILD A SCROLLABLE LIST

Gambar 13 Screenshot Hasil Tampilan Halaman Home Modul 3.....	56
Gambar 14 Screenshot Hasil Tampilan Halaman Detail Modul 3.....	57
Gambar 15 Screenshot Hasil Tampilan Link Netflix Modul 3	58

MODUL 4: VIEWMODEL DAN DEBUGGING

Gambar 16 Screenshot Hasil Tampilan Halaman Home Modul 4.....	88
Gambar 17 Screenshot Hasil Tampilan Halaman Detail Modul 4.....	89
Gambar 18 Screenshot Hasil Tampilan Link Netflix Modul 4	90
Gambar 19 Penggunaan Debugger Modul 4.....	91
Gambar 20 Penggunaan Fitur f7 (Step Into) pada Debug Modul 4	91
Gambar 21 Penggunaan Fitur f8 (Step Over) pada Debug Modul 4.....	92
Gambar 22 Penggunaan Fitur shift+ f8 (Step Out) pada Debug Modul 4	92
Gambar 23 Penggunaan Fitur f9 (Resume Program) pada Debug Modul 4.....	93

MODUL 5: CONNECT TO INTERNET

Gambar 24 Screenshot Hasil Tampilan Halaman Utama Modul 5.....	150
Gambar 25 Screenshot Hasil Tampilan Halaman Detail Modul 5.....	151
Gambar 26 Screenshot Hasil Tampilan Halaman Utama DarkMode Modul 5 ..	152
Gambar 27 Screenshot Hasil Tampilan Halaman Detail DarkMode Modul 5 ...	153
Gambar 28 Screenshot Hasil Tampilan Halaman Utama Jika Offline Modul 5.	154
Gambar 29 Screenshot Hasil Tampilan Halaman Detail Jika Offline Modul 5..	155

DAFTAR TABEL

MODUL 1: ANDROID BASIC WITH KOTLIN

Tabel 1 Source Code Jawaban MainActivity.kt Modul 1	12
Tabel 2 Source Code Jawaban Activity_main.kt Modul 1	13

MODUL 2: ANDROID LAYOUT

Tabel 3 Source Code Jawaban MainActivity.kt Modul 2	22
Tabel 4 Source Code Jawaban Activity_main.xml Modul 2	24
Tabel 5 Source Code Jawaban button_color.xml Modul 2	26
Tabel 6 Source Code Jawaban ViewModel.kt Modul 2	26
Tabel 7 Source Code Jawaban themes.xml Modul 2	27
Tabel 8 Source Code Jawaban AndroidManifest.kt Modul 2	28
Tabel 9 Source Code Jawaban AndroidManifest.xml Modul 2	28

MODUL 3: BUILD A SCROLLABLE LIST

Tabel 10 Source Code Jawaban MainActivity.kt Modul 3	40
Tabel 11 Source Code Jawaban Drama.kt Modul 3	40
Tabel 12 Source Code Jawaban HomeFragment.kt Modul 3	41
Tabel 13 Source Code Jawaban DetailFragment.kt Modul 3	43
Tabel 14 Source Code Jawaban DramaAdapter.kt Modul 3	44
Tabel 15 Source Code Jawaban Activity_main.xml Modul 3	46
Tabel 16 Source Code Jawaban fragment_home.xml Modul 3	46
Tabel 17 Source Code Jawaban fragment_detail.xml Modul 3	47
Tabel 18 Source Code Jawaban item_list.xml Modul 3	48
Tabel 19 Source Code Jawaban string.xml Modul 3	50
Tabel 20 Source Code Jawaban build.gradle.kts Modul 3	53
Tabel 21 Source Code Jawaban AndroidManifest.xml Modul 3	54

MODUL 5: VIEWMODEL DAN DEBUGGING

Tabel 22 Source Code Jawaban MainActivity.kt Modul 4	69
--	----

Tabel 23 Source Code Jawaban Drama.kt Modul 4.....	69
Tabel 24 Source Code Jawaban HomeFragment.kt Modul 4	70
Tabel 25 Source Code Jawaban DetailFragment.kt Modul 4	72
Tabel 26 Source Code Jawaban DramaAdapter.kt Modul 4.....	73
Tabel 27 Source Code Jawaban DramaViewModel.kt Modul 4.....	75
Tabel 28 Source Code Jawaban DramaViewModelFactory.kt Modul 4	77
Tabel 29 Source Code Jawaban Activity_main.xml Modul 4	77
Tabel 30 Source Code Jawaban fragment_home Modul 4	78
Tabel 31 Source Code Jawaban fragment_detail.xml Modul 4	79
Tabel 32 Source Code Jawaban item_list.xml Modul 4	81
Tabel 33 Source Code Jawaban string.xml Modul 4	83
Tabel 34 Source Code Jawaban build.gradle.kts Modul 4.....	85
Tabel 35 Source Code Jawaban AndroidManifest.xml Modul 4.....	87

MODUL 5: CONNECT TO INTERNET

Tabel 36 Source Code Jawaban DramaDao.kt Modul 5	109
Tabel 37 Source Code Jawaban DramaDatabase.kt Modul 5	110
Tabel 38 Source Code Jawaban DramaEntity.kt Modul 5	111
Tabel 39 Source Code Jawaban DramaApiService.kt Modul 5.....	111
Tabel 40 Source Code Jawaban DramaDto.kt Modul 5.....	111
Tabel 41 Source Code Jawaban PopularDramasResponse.kt Modul 5	112
Tabel 42 Source Code Jawaban ApiClient.kt Modul 5.....	112
Tabel 43 Source Code Jawaban DramaRepositoryImpl.kt Modul 5.....	114
Tabel 44 Source Code Jawaban DramaMappers.kt Modul 5.....	117
Tabel 45 Source Code Jawaban Drama.kt Modul 5.....	118
Tabel 46 Source Code Jawaban DramaRepository.kt Modul 5	118
Tabel 47 Source Code Jawaban GetDramaUseCase.kt Modul 5	119
Tabel 48 Source Code Jawaban DetailFragment.kt Modul 5	119
Tabel 49 Source Code Jawaban DramaAdapter.kt Modul 5.....	120
Tabel 50 Source Code Jawaban DramaListFragment.kt Modul 5	122
Tabel 51 Source Code Jawaban DramaViewModel.kt Modul 5.....	126
Tabel 52 Source Code Jawaban DramaViewModelFactory.kt Modul 5	128

Tabel 53 Source Code Jawaban MainActivity.kt Modul 5	129
Tabel 54Source Code Jawaban AppPreferences.kt Modul 5	130
Tabel 55 Source Code Jawaban Resource.kt Modul 5.....	131
Tabel 56 Source Code Jawaban Activiy_main.xml Modul 5	131
Tabel 57 Source Code Jawaban fragment_detail.xml Modul 5	132
Tabel 58Source Code Jawaban fragment_drama_list.xml Modul 5	134
Tabel 59 Source Code Jawaban item_drama.xml Modul 5	136
Tabel 60 Source Code Jawaban main_menu.xml Modul 5.....	138
Tabel 61Source Code Jawaban nav_graph.xml Modul 5.....	138
Tabel 62 Source Code Jawaban colors.xml Modul 5.....	139
Tabel 63Source Code Jawaban themes.xml Modul 5	142
Tabel 64 Source Code Jawaban themes.xml Modul 5	144
Tabel 65 Source Code Jawaban build.gradle.kts Modul 5.....	146
Tabel 66 Source Code Jawaban AndroidManifest.xml Modul 5.....	148

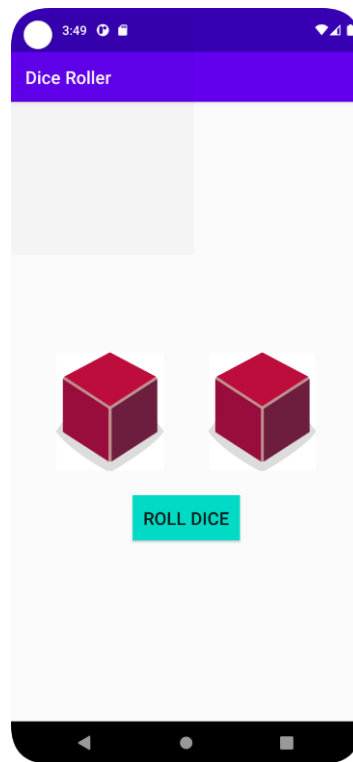
MODUL 1:

ANDROID BASIC WITH KOTLIN

SOAL 1

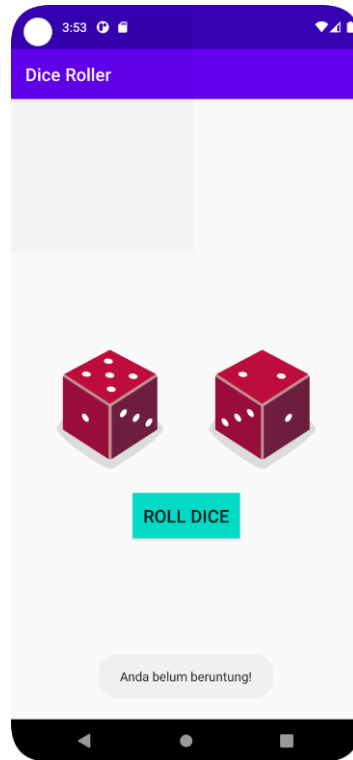
Buatlah sebuah aplikasi yang dapat menampilkan 2 (dua) buah dadu yang dapat berubah-ubah tampilannya pada saat user menekan tombol “Roll Dice”. Aturan aplikasi yang akan dibangun adalah sebagaimana berikut:

1. Tampilan awal aplikasi setelah dijalankan akan menampilkan 2 buah dadu kosong seperti dapat dilihat pada Gambar 1.



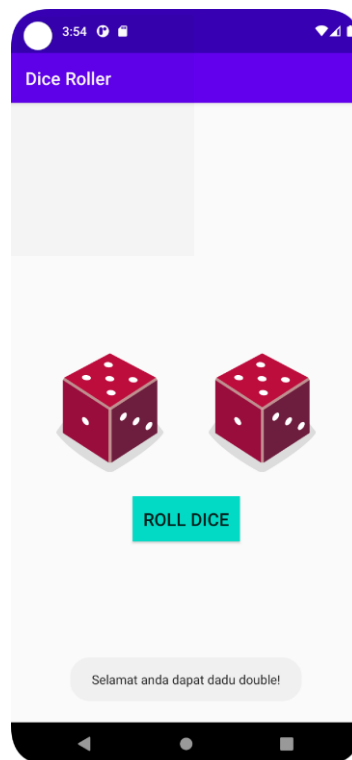
Gambar 1 Soal Tampilan Awal Aplikasi

2. Setelah user menekan tombol “Roll Dice” maka masing-masing dadu akan memunculkan sisi dadu masing-masing dengan angka antara 1 s/d 6. Apabila user mendapatkan nilai dadu yang berbeda antara Dadu 1 dengan Dadu 2 maka akan menampilkan pesan “Anda belum beruntung!” seperti dapat dilihat pada Gambar 2.



Gambar 2 Tampilan Dadu Setelah Di Roll

3. Apabila user mendapatkan nilai dadu yang sama antara Dadu 1 dan Dadu 2 atau nilai double, maka aplikasi akan menampilkan pesan “Selamat anda dapat dadu double!” seperti dapat dilihat pada Gambar 3.



Gambar 3 Soal Contoh Tampilan Roll Dadu Double

4. Upload aplikasi yang telah anda buat kedalam repository github ke dalam **folder Module 2 dalam bentuk project**. Jangan lupa untuk melakukan **Clean Project** sebelum mengupload pekerjaan anda pada repo.
5. Untuk gambar dadu dapat didownload pada link berikut:
<https://drive.google.com/file/d/14V3qXGdFmoYN4AGd95gFh8kw8X9vSm/view>

A. Source Code

1. MainActivity.kt

Tabel 1 Source Code Jawaban MainActivity.kt Modul 1

1	package com.example.diceroller
2	
3	import android.os.Bundle
4	import android.widget.Button
5	import android.widget.ImageView
6	import android.widget.Toast
7	import androidx.activity.enableEdgeToEdge
8	import androidx.appcompat.app.AppCompatActivity
9	import androidx.core.view.ViewCompat
10	import androidx.core.view.WindowInsetsCompat
11	import com.example.diceroller.databinding.ActivityMainBinding
12	
13	class MainActivity : AppCompatActivity() {
14	private lateinit var binding: ActivityMainBinding
15	override fun onCreate(savedInstanceState: Bundle?) {
16	super.onCreate(savedInstanceState)
17	enableEdgeToEdge()
18	binding = ActivityMainBinding.inflate(layoutInflater)
19	setContentView(binding.root)
20	
21	binding.rollButton.setOnClickListener {
22	rollDice()
23	}
24	}
25	fun rollDice() {
26	
27	val randomInt1 = (1..6).random()
28	val drawableResource1 = when (randomInt1) {

29	1 -> R.drawable.dice_1
30	2 -> R.drawable.dice_2
31	3 -> R.drawable.dice_3
32	4 -> R.drawable.dice_4
33	5 -> R.drawable.dice_5
34	6 -> R.drawable.dice_6
35	else -> R.drawable.dice_0
36	}
37	val randomInt2 = (1..6).random()
38	val drawableResource2 = when
	(randomInt2) {
39	1 -> R.drawable.dice_1
40	2 -> R.drawable.dice_2
41	3 -> R.drawable.dice_3
42	4 -> R.drawable.dice_4
43	5 -> R.drawable.dice_5
44	6 -> R.drawable.dice_6
45	else -> R.drawable.dice_0
46	}
47	
48	binding.diceImage1.setImageResource(drawableResource1)
49	
50	binding.diceImage2.setImageResource(drawableResource2)
51	
52	if (randomInt1 == randomInt2) {
53	Toast.makeText(this, "Selamat
	Kamu Dapat Dadu Double!",
	Toast.LENGTH_SHORT).show()
54	} else {
55	Toast.makeText(this, "Anda Belum
	Beruntung", Toast.LENGTH_SHORT).show()
56	}
57	}
	}

2. Activity_main.xml

Tabel 2 Source Code Jawaban Activity main.kt Modul 1

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	
6	xmlns:tools="http://schemas.android.com/tools"

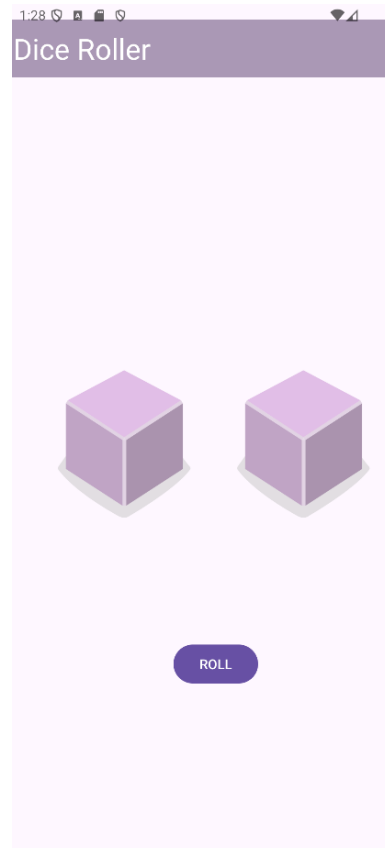
```

7      android:id="@+id/main"
8      android:layout_width="match_parent"
9      android:layout_height="match_parent"
10     tools:context=".MainActivity">
11     <TextView
12         android:layout_width="match_parent"
13         android:layout_height="wrap_content"
14         android:layout_marginTop="16dp"
15         android:background="#AA98B5"
16         android:padding="10dp"
17         android:textColor="@color/white"
18         android:text="Dice Roller"
19         android:textSize="30sp"
20     app:layout_constraintStart_toStartOf="parent"
21         app:layout_constraintTop_toTopOf="parent"
22     />
23     <ImageView
24         android:id="@+id/dice_image1"
25         android:layout_width="200dp"
26         android:layout_height="200dp"
27         android:layout_marginStart="25dp"
28         android:src="@drawable/dice_0"
29     app:layout_constraintBottom_toBottomOf="parent"
30     app:layout_constraintStart_toStartOf="parent"
31     app:layout_constraintTop_toTopOf="parent"
32     />
33     <ImageView
34         android:id="@+id/dice_image2"
35         android:layout_width="200dp"
36         android:layout_height="200dp"
37         android:layout_marginStart="25dp"
38         android:src="@drawable/dice_0"
39
40     app:layout_constraintBottom_toBottomOf="parent"
41     app:layout_constraintEnd_toEndOf="parent"
42     app:layout_constraintTop_toTopOf="parent"
43     />
44     <Button
45         android:id="@+id/roll_button"
46         android:text="ROLL"
47         android:layout_width="wrap_content"
48         android:layout_height="wrap_content"
49         android:layout_marginEnd="144dp"
50         android:layout_marginBottom="204dp"
51
52     app:layout_constraintBottom_toBottomOf="parent"
53     app:layout_constraintEnd_toEndOf="parent"
54     app:layout_constraintHorizontal_bias="0.983"

```

55	app:layout_constraintStart_toStartOf="parent"
56	app:layout_constraintTop_toTopOf="parent"
57	app:layout_constraintVertical_bias="1.0" />
58	</androidx.constraintlayout.widget.ConstraintLayout>

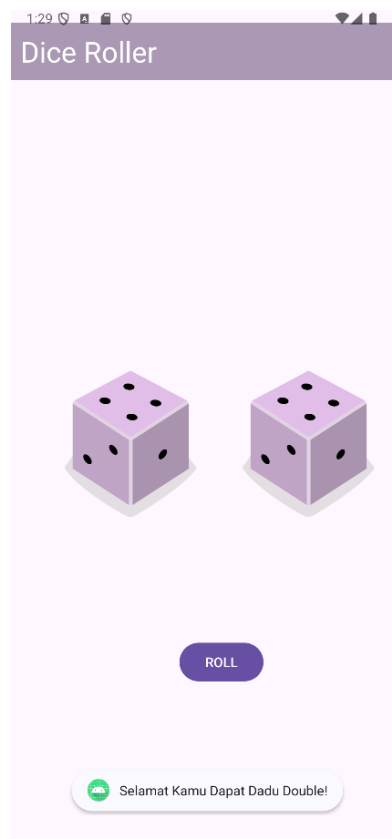
B. Output Program



Gambar 4 Screenshot Hasil Jawaban Modul 1



Gambar 5 Screenshot Hasil Jawaban Modul 1



Gambar 6 Screenshot Hasil Jawaban Modul 1

C. Pembahasan

1. MainActivity.kt

Pada baris 1, deklarasi nama package tempat file Kotlin ini berada. Baris 3, diimpor `Bundle` digunakan untuk menyimpan dan meneruskan data antar aktivitas. Baris 4 `Button` diimpor agar tombol dalam tampilan bisa dikenali dan diatur lewat kode. Baris 5 `ImageView` dipakai untuk menampilkan gambar dadu. Baris 6 `Toast` digunakan untuk menampilkan pesan singkat di layar sebagai notifikasi kepada pengguna. Baris 7 `enableEdgeToEdge` digunakan untuk memperluas konten aplikasi sampai ke tepi layar, memberi tampilan yang lebih modern. Baris 8 `AppCompatActivity` merupakan kelas dasar untuk aktivitas. Kemudian Baris 9–10 `ViewCompat` dan `WindowInsetsCompat` digunakan untuk mengatur tampilan agar bisa menyesuaikan dengan area layar tertentu seperti notch atau status bar. Impor `ActivityMainBinding` digunakan pada program bisa diakses komponen UI langsung tanpa perlu mencari satu satu. Pada baris 13, dideklarasikan kelas `MainActivity` yang merupakan turunan dari `AppCompatActivity`, yaitu kelas dasar untuk aktivitas Android. Baris 14 variabel `binding` digunakan untuk menghubungkan layout XML dengan kode Kotlin menggunakan View Binding. View Binding ini memudahkan akses langsung ke komponen-komponen UI yang ada di layout tanpa perlu mencari satu satu. Selanjutnya, baris 15 fungsi `onCreate()` kode ini akan dijalankan saat aktivitas pertama kali dibuat. Pertama, `super.onCreate(savedInstanceState)` dipanggil agar fungsi bawaan dari `AppCompatActivity` tetap bekerja sebagaimana mestinya. Kemudian `enableEdgeToEdge()` dipanggil untuk membuat aplikasi bisa tampil penuh hingga ke batas layar. Setelah itu, `binding` diinisialisasi dengan `ActivityMainBinding.inflate(layoutInflater)` untuk menghubungkan tampilan dengan kode, lalu `setContentView(binding.root)` dipanggil untuk menampilkan layout tersebut di layar. Pada bagian akhir fungsi `onCreate`, dibuat event

listener pada tombol `rollButton` menggunakan `setOnClickListener`. Artinya, ketika tombol ditekan, maka aplikasi akan menjalankan fungsi `rollDice()` untuk melempar dadu. Fungsi `rollDice()` sendiri dimulai pada baris 23. Di dalamnya, aplikasi akan secara acak memilih angka dari 1 sampai 6 untuk dua dadu, masing-masing disimpan dalam `randomInt1` dan `randomInt2`. Kemudian, angka-angka tersebut dicocokkan dengan gambar dadu yang sesuai menggunakan struktur `when`. Jika angka yang didapat adalah 1, maka gambar yang digunakan adalah `R.drawable.dice_1`, begitu juga untuk angka-angka lainnya hingga 6. Jika entah bagaimana angkanya di luar rentang itu, maka gambar `dice_0` akan digunakan sebagai gambar default. Setelah menentukan gambar yang tepat, gambar tersebut ditampilkan ke dalam dua `ImageView`, yaitu `diceImage1` dan `diceImage2` menggunakan `setImageResource`. Terakhir, fungsi ini mengecek apakah angka dari kedua dadu sama. Jika ya, maka ditampilkan notifikasi singkat menggunakan `Toast` yang mengatakan “Selamat Kamu Dapat Dadu Double!”. Namun jika tidak sama, maka muncul pesan “Anda Belum Beruntung.”

2. Activity_Main.xml

Pada baris 1 deklarasi XML dan encoding. Baris selanjutnya definisi `ConstraintLayout` sebagai layout utama dari tampilan aplikasi. Di pada kode `ConstraintLayout`, ada tiga namespace, yaitu `xmlns:android`, `xmlns:app`, dan `xmlns:tools` digunakan untuk atribut-atribut standar Android, fitur tambahan, dan alat bantu tampilan. Pada bagian `TextView`, elemen ini digunakan untuk menampilkan judul aplikasi yaitu “Dice Roller” di bagian atas layar. Lebar diatur memenuhi seluruh layar, tinggi menyesuaikan isi teks, dan memiliki margin atas agar tidak terlalu menempel ke tepi layar. Latar belakang diberi warna ungu muda, teksnya berwarna putih, dan ukurannya cukup besar yaitu 30sp, Selanjutnya pada bagian `ImageView`, terdapat dua buah gambar yang masing-masing mewakili dadu pertama dan kedua. Kedua gambar ini

menggunakan gambar awal `dice_0`, yaitu gambar dadu kosong sebagai tampilan awal sebelum tombol roll ditekan. Ukuran gambar dadu dibuat cukup besar yaitu 200dp agar mudah dilihat, dan diletakkan sejajar secara horizontal di tengah layar, masing-masing berada di sisi kiri dan kanan. Letaknya juga ditentukan agar tetap proporsional mengikuti ukuran layar, baik secara vertikal maupun horizontal. Kemudian di bagian `Button`, terdapat tombol dengan tulisan “ROLL” yang berfungsi untuk melempar dadu ketika ditekan oleh pengguna. Tombol ini memiliki lebar yang menyesuaikan isi tulisan, dan diletakkan di bagian bawah layar dengan sedikit jarak dari tepi bawah. Posisinya disejajarkan secara horizontal agar tampil rapi di tengah layout. Tombol ini terhubung ke kode Kotlin melalui ID `roll_button`, sehingga saat ditekan, aplikasi akan menjalankan fungsi untuk menghasilkan angka acak dan mengganti gambar dadu. Tampilan dan posisi tombol ini disesuaikan agar nyaman digunakan dan mudah dijangkau oleh pengguna.

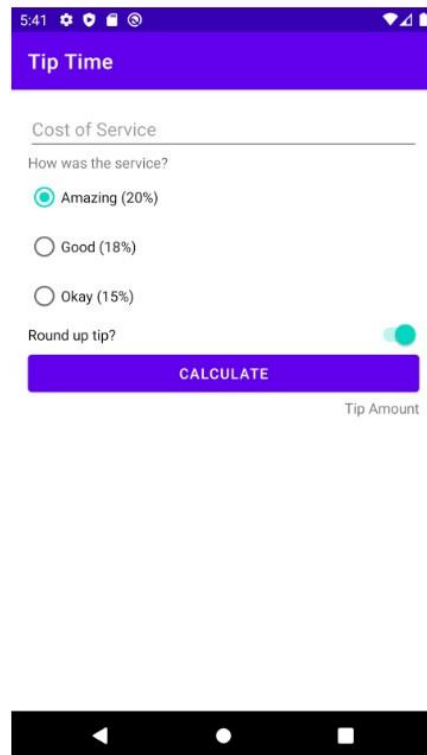
MODUL 2: ANDROID LAYOUT

SOAL 1

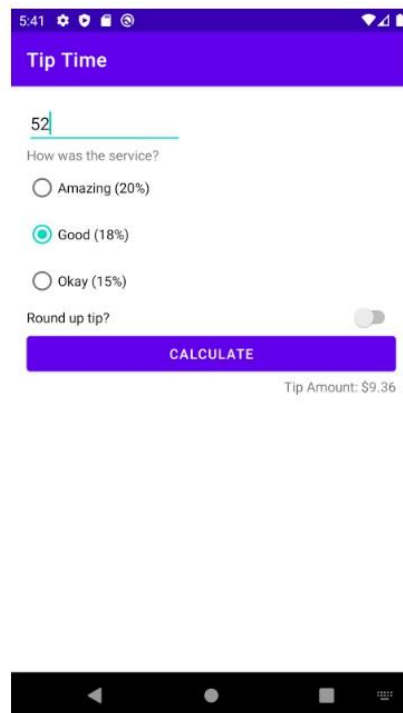
Buatlah sebuah aplikasi kalkulator tip yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:

1. Input Biaya Layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.
2. Pilihan Persentase Tip: Pengguna dapat memilih persentase tip yang diinginkan dari opsi yang disediakan, yaitu 15%, 18%, dan 20%.
3. Pengaturan Pembulatan Tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.
4. Tampilan Hasil: Aplikasi akan menampilkan jumlah tip yang harus dibayar secara langsung setelah pengguna memberikan input.

a



Gambar 7 Contoh Tampilan Awal Aplikasi



Gambar 8 Contoh Tampilan Aplikasi Setelah Dijalankan

A. Source Code

1. MainActivity.kt

Tabel 3 Source Code Jawaban MainActivity.kt Modul 2

```
1 package com.example.tiptime
2
3 import android.os.Bundle
4 import android.util.Log
5 import android.widget.*
6 import androidx.appcompat.app.AppCompatActivity
7 import
8     androidx.core.splashscreen.SplashScreen.Companion.installSplashScreen
9 import java.text.NumberFormat
10 import kotlin.math.ceil
11 import androidx.activity.viewModels
12 import com.example.tiptime.TipViewModel
13
14 class MainActivity : AppCompatActivity() {
15     private val viewModel: TipViewModel by
16     viewModels()
17
18     override fun onCreate(savedInstanceState:
19     Bundle?) {
20         super.onCreate(savedInstanceState)
21         Thread.sleep(3000)
22         installSplashScreen()
23         setContentView(R.layout.activity_main)
24
25         val costOfServiceEditText =
26         findViewById<EditText>(R.id.cost_of_service_edit
27         _text)
28         val tipOptions =
29         findViewById<RadioGroup>(R.id.tip_options)
30         val roundUpSwitch =
31         findViewById<Switch>(R.id.round_up_switch)
32         val calculateButton =
33         findViewById<Button>(R.id.calculate_button)
34         val tipResult =
35         findViewById<TextView>(R.id.tip_result)
36
37         if (viewModel.tipResult.isNotEmpty()) {
38             tipResult.text = "Tip Amount:
39             ${viewModel.tipResult}"
40         }
41
42         calculateButton.setOnClickListener {
43             val costInput =
44             costOfServiceEditText.text.toString()
```

```

35         val cost =
costInput.toDoubleOrNull()
36
37         if (costInput.isEmpty()) {
38             tipResult.text = ""
39             Toast.makeText(this, "Masukkan
biaya layanan!", Toast.LENGTH_SHORT).show()
40             return@setOnClickListener
41         }
42
43         if (cost == null || cost <= 0.0) {
44             tipResult.text = ""
45             Toast.makeText(
46                 this,
47                 "Input tidak valid. Masukkan
angka yang benar!",
48                 Toast.LENGTH_SHORT
49             ).show()
50             return@setOnClickListener
51         }
52
53         val tipPercentage = when
(tipOptions.checkedRadioButtonId) {
54             R.id.amazing_option -> 0.20
55             R.id.good_option -> 0.18
56             R.id.okay_option -> 0.15
57             else -> {
58                 Toast.makeText(
59                     this,
60                     "Pilih kualitas layanan
terlebih dahulu!",
61                     Toast.LENGTH_SHORT
62                 ).show()
63                 return@setOnClickListener
64             }
65         }
66         val result =
viewModel.calculateTip(cost, tipPercentage,
roundUpSwitch.isChecked)
67         tipResult.text = "Tip Amount:
$result"
68
69         Log.d("TipTime", "Tip dihitung:
$result")
70     }
71 }
72 }

```

2. Activity_main.xml

Tabel 4 Source Code Jawaban Activity main.xml Modul 2

1	<?xml version="1.0" encoding="utf-8"?>
2	<LinearLayout
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:tools="http://schemas.android.com/tools"
4	android:id="@+id/mainLayout"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent"
7	android:orientation="vertical"
8	android:padding="16dp"
9	tools:context=".MainActivity">
10	
11	<TextView
12	android:id="@+id/toolBarText"
13	android:layout_width="match_parent"
14	android:layout_height="wrap_content"
15	android:paddingBottom="8dp"
16	android:text="Tip Time"
17	android:textSize="18sp"
18	android:background="@android:color/white"
19	android:backgroundTint="#ebccff" />
20	
21	<EditText
22	android:id="@+id/cost_of_service_edit_text"
23	android:layout_width="match_parent"
24	android:layout_height="wrap_content"
25	android:inputType="numberDecimal"
26	android:hint="Cost of Service"
27	android:padding="8dp"
28	android:background="@android:drawable/edit_text"
	/>
29	
30	<TextView
31	android:id="@+id/question"
32	android:layout_width="wrap_content"
33	android:layout_height="wrap_content"
34	android:text="How was the service?"
35	android:layout_marginTop="16dp" />
36	
37	<RadioGroup
38	android:id="@+id/tip_options"
39	android:layout_width="wrap_content"
40	android:layout_height="wrap_content">
41	


```

42         <RadioButton
43             android:id="@+id/amazing_option"
44             android:layout_width="wrap_content"
45             android:layout_height="wrap_content"
46             android:text="Amazing (20%" />
47
48         <RadioButton
49             android:id="@+id/good_option"
50             android:layout_width="wrap_content"
51             android:layout_height="wrap_content"
52             android:text="Good (18%" />
53
54         <RadioButton
55             android:id="@+id/okay_option"
56             android:layout_width="wrap_content"
57             android:layout_height="wrap_content"
58             android:text="Okay (15%" />
59     </RadioGroup>
60
61     <LinearLayout
62         android:layout_width="wrap_content"
63         android:layout_height="wrap_content"
64         android:orientation="horizontal"
65         android:layout_marginTop="16dp"
66         android:gravity="center_vertical">
67
68         <TextView
69             android:layout_width="wrap_content"
70             android:layout_height="wrap_content"
71             android:text="Round up tip?" />
72
73         <Switch
74             android:id="@+id/round_up_switch"
75             android:layout_width="wrap_content"
76             android:layout_height="wrap_content"
77             android:layout_marginStart="240dp"
78     />
79 </LinearLayout>
80
81     <android.widget.Button
82         android:id="@+id/calculate_button"
83         android:layout_width="match_parent"
84         android:layout_height="wrap_content"
85         android:text="CALCULATE"
86         android:layout_marginTop="24dp"
87         android:background="@drawable/button_color"
88         android:textColor="@android:color/white"
89     />

```

88	
89	<TextView
90	android:id="@+id/tip_result"
91	android:layout_width="wrap_content"
92	android:layout_height="wrap_content"
93	android:layout_marginTop="16dp"
94	android:text="Tip Amount:"
95	android:textSize="16sp"
96	android:layout_marginStart="240dp" />
97	</LinearLayout>

3. Button_Color.xml

Tabel 5 Source Code Jawaban button_color.xml Modul 2

1	<?xml version="1.0" encoding="utf-8"?>
2	<selector
	xmlns:android="http://schemas.android.com/apk/res/android">
3	<item android:state_pressed="true">
4	<shape android:shape="rectangle">
5	<solid android:color="#7d499d" />
6	<corners android:radius="12dp" />
7	</shape>
8	</item>
9	
10	<item>
11	<shape android:shape="rectangle">
12	<solid android:color="#d395f9" />
13	<corners android:radius="12dp" />
14	</shape>
15	</item>
16	</selector>

4. ViewModel.kt

Tabel 6 Source Code Jawaban ViewModel.kt Modul 2

1	package com.example.tiptime
2	
3	import androidx.lifecycle.ViewModel
4	import java.text.NumberFormat
5	import kotlin.math.ceil
6	
7	class TipViewModel : ViewModel() {
8	var tipResult: String = ""
9	
10	fun calculateTip(cost: Double, tipPercent:
	Double, roundUp: Boolean): String {
11	var tip = cost * tipPercent

12	if (roundUp) tip = ceil(tip)
13	
14	tipResult =
15	NumberFormat.getCurrencyInstance().format(tip)
16	return tipResult
17	}

5. Values/themes.xml

Tabel 7 Source Code Jawaban themes.xml Modul 2

1	<resources
2	xmlns:tools="http://schemas.android.com/tools">
3	<!-- Base application theme. -->
4	<style name="Base.Theme.TipTime"
5	parent="Theme.Material3.DayNight.NoActionBar">
6	<!-- Customize your light theme here. --
7	>
8	<item name=
9	"colorPrimary">@color/white</item>
10	<item name=
11	"colorPrimaryVariant">@color/white</item>
12	<item name=
13	"colorOnPrimary">@color/white</item>
14	<item name=
15	"android:statusBarColor">?attr/colorPrimaryVaria
16	nt</item>
17	<!-- <item
18	name="colorPrimary">@color/my_light_primary</ite
19	m> -->
20	</style>
21	<style name="Theme.TipTime"
22	parent="Base.Theme.TipTime" />
23	
24	<style name="Theme.App.SplashScreen"
25	parent="Theme.SplashScreen">
26	<item
27	name="windowSplashScreenBackground">@color/white
28	</item>
29	<item
30	name="windowSplashScreenAnimatedIcon">@drawable/
31	android</item>
32	<item
33	name="postSplashScreenTheme">@style/Base.Theme.T
34	ipTime</item>
35	</style>
36	</resources>

6. AndroidManifest.xml

Tabel 8 Source Code Jawaban AndroidManifest.kt Modul 2

1	<?xml version="1.0" encoding="utf-8"?>
2	<manifest
	xmlns:android="http://schemas.android.com/apk/re
	s/android"
3	xmlns:tools="http://schemas.android.com/tools">
4	
5	<application
6	android:allowBackup="true"
7	android:dataExtractionRules="@xml/data_extractio
	n_rules"
8	android:fullBackupContent="@xml/backup_rules"
9	android:icon="@mipmap/ic_launcher"
10	android:label="@string/app_name"
11	android:roundIcon="@mipmap/ic_launcher_round"
12	android:supportsRtl="true"
13	android:theme="@style/Theme.TipTime"
14	tools:targetApi="31">
15	<activity
16	android:name=".ViewModel"
17	android:exported="false" />
18	<activity
19	android:name=".MainActivity"
20	android:exported="true"
21	android:theme="@style/Theme.App.SplashScreen">
22	<intent-filter>
23	<action
	android:name="android.intent.action.MAIN" />
24	<category
	android:name="android.intent.category.LAUNCHER"
	/>
25	</intent-filter>
26	</activity>
27	</application>
28	
29	</manifest>

Tabel 9 Source Code Jawaban AndroidManifest.xml

B. Output Program

12:47  



*Gambar 9 Screenshot Hasil Tampilan
SplashScreen Modul 2*

Tip Time

Cost of Service

How was the service?

☒ Amazing (20%)

☐ Good (18%)

☐ Okay (15%)

Round up tip?

CALCULATE

Tip Amount:

Gambar 10 Screenshot Hasil Tampilan Awal Modul 2

Tip Time

52

How was the service?

☐ Amazing (20%)

☒ Good (18%)

☐ Okay (15%)

Round up tip?

CALCULATE

Tip Amount: \$9.36

Gambar 11 Screenshot Hasil Tampilan Aplikasi Setelah Dijalankan Modul 2

Tip Time

0

How was the service?

☐ Amazing (20%)

☐ Good (18%)

☐ Okay (15%)

Round up tip? ☐

CALCULATE

 Input tidak valid. Masukkan angka yang benar!

Gambar 12 Screenshot Hasil Tampilan Aplikasi Jika Input Salah Modul 2

C. Pembahasan

1. MainActivity.kt

Pada baris 1 Package berfungsi sebagai nama unik untuk mengelompokkan kelas-kelas dalam proyek Android. Baris 3–11 merupakan bagian impor dari berbagai kelas penting yang dibutuhkan supaya aplikasi bisa menjalankan fitur UI modern, menyimpan data lewat ViewModel, dan menangani input/output dengan baik di Android. Baris 14 kelas MainActivity adalah tempat utama aplikasi dijalankan saat pengguna membukanya. Baris 15 membuat properti viewModel dengan bantuan byviewModels() yang otomatis membuat objek TipViewModel. Ini berguna supaya data tetap tersimpan meskipun layar diputar atau konfigurasi berubah. Baris 17 onCreate() adalah fungsi yang dijalankan saat pertama kali aktivitas dibuat. Baris 18 Thread.sleep(3000) digunakan untuk memberi jeda selama 3 detik agar splash screen terlihat lebih lama. Baris 19 memanggil installSplashScreen() untuk menampilkan splash screen saat aplikasi dibuka. Baris 20 setContentView(R.layout.activity_main) berfungsi untuk menampilkan tampilan layar dari file XML bernama activity_main.xml. Baris 22–27 menghubungkan komponen UI di XML ke variabel Kotlin dengan findViewById, seperti EditText, RadioGroup, Switch, dan Button. Baris 29–31 mengecek apakah sebelumnya sudah ada hasil tipResult yang disimpan di ViewModel. Kalau ada, hasilnya langsung ditampilkan di TextView. Baris 33 menambahkan aksi ketika tombol calculateButton ditekan dengan setOnClickListener. Baris 34 mengambil input dari EditText dan menyimpannya sebagai teks. Baris 35 mencoba mengubah teks tadi menjadi angka desimal dengan toDoubleOrNull(), yang akan jadi null kalau input tidak valid. Baris 37–40 kalau input kosong, akan muncul pesan Toast untuk memberi tahu pengguna bahwa input belum diisi, lalu proses berhenti. Baris 42–47 mengecek apakah input valid dan lebih dari nol. Kalau tidak, maka akan muncul pesan error dan hasil tidak akan ditampilkan. Baris 49–

55 menentukan berapa persen tip berdasarkan pilihan yang dipilih di `RadioGroup`. Kalau belum memilih apa pun, akan muncul pesan supaya memilih kualitas layanan dulu. Baris 56 memanggil fungsi `calculateTip` dari `TipViewModel` dengan memasukkan nilai biaya, persen tip, dan apakah dibulatkan. Hasilnya lalu ditampilkan ke `tipResult`. Baris 58 mencatat log debug dengan `Log.d()` agar pengembang bisa melihat nilai tip yang dihitung saat tombol ditekan.

2. Activity_Main.xml

Baris 1–9 `LinearLayout` bagian ini adalah tempat utama untuk menyusun tampilan dengan arah dari atas ke bawah. `LinearLayout` ini diberi jarak ke dalam (`padding`) supaya isi layar tidak terlalu mepet ke pinggir, dan sudah terhubung dengan `MainActivity` sebagai bagian utamanya. Baris 11–19 `TextView` berfungsi sebagai judul aplikasi dengan tulisan “Tip Time”. Ukuran teks dibuat sedikit besar supaya terlihat seperti judul di bagian atas. Meski ada dua pengaturan latar belakang, hanya satu yang benar-benar bekerja untuk menampilkan warna. Baris 21–28 `EditText` digunakan agar pengguna bisa mengisi jumlah biaya layanan. Masukan hanya dibatasi untuk angka desimal dan sudah ada tulisan petunjuk (`hint`) supaya lebih jelas. Tampilan kotak input memakai latar belakang bawaan dari Android. Baris 30–35 `TextView` berfungsi sebagai label atau pertanyaan untuk menanyakan kualitas layanan, yang nantinya dijawab dengan memilih salah satu dari pilihan yang ada di radio button. Baris 37–59 `RadioGroup` bagian ini berisi tiga pilihan `RadioButton`. Setiap pilihan menunjukkan seberapa baik layanan yang diterima, seperti “Amazing”, “Good”, atau “Okay”, dan masing-masing punya persentase tip yang berbeda. Baris 61–78 `LinearLayout` horizontal ini menampilkan pertanyaan dan switch (saklar) untuk memilih apakah hasil tip mau dibulatkan ke atas. Tapi tata letaknya memakai jarak tetap (`margin`) yang bisa kurang cocok di ukuran layar yang berbeda-beda. Baris 80–87 `Button` digunakan untuk memulai proses menghitung tip. Desain tombol dibuat lebih mudah dilihat dan menarik perhatian pengguna. Baris 89–97

TextView dipakai untuk menampilkan hasil perhitungan tip setelah tombol ditekan. Teksnya dibuat agak ke kanan dengan margin, tapi cara ini kurang fleksibel kalau ditampilkan di berbagai ukuran layar.

3. Button_Color.xml

Baris 1 `<?xml version="1.0" encoding="utf-8"?>` menunjukkan bahwa file ini menggunakan format XML versi 1.0 dan karakter UTF-8. Baris 2 `<selector>` adalah elemen utama yang digunakan untuk menentukan tampilan berbeda berdasarkan keadaan (state) dari sebuah elemen UI, seperti tombol. Baris 3 `<item android:state_pressed="true">` berarti gaya ini akan diterapkan saat tombol ditekan. Baris 4 `<shape android:shape="rectangle">` membentuk tombol dengan sudut membulat. Baris 5 `<solid android:color="#7d499d" />` memberi warna ungu gelap saat tombol ditekan. Baris 6 `<corners android:radius="12dp" />` membulatkan sudut tombol sebesar 12dp. Baris 10 `<item>` tanpa atribut state berarti gaya ini digunakan saat tombol dalam keadaan normal (tidak ditekan). Baris 11 `<shape android:shape="rectangle">` membentuk tombol biasa berbentuk kotak. Baris 12 `<solid android:color="#d395f9" />` memberi warna ungu muda saat tombol tidak ditekan. Baris 13 `<corners android:radius="12dp" />` membulatkan sudut tombol seperti sebelumnya.

4. ViewModel.kt

Baris 1 `package com.example.tiptime` menunjukkan bahwa file ini berada di dalam paket bernama `com.example.tiptime`, yang berfungsi sebagai identitas tempat menyimpan kelas. Baris 3 `import androidx.lifecycle.ViewModel` digunakan untuk mengimpor `ViewModel`, yaitu komponen Android yang menyimpan dan mengelola data agar tetap ada saat orientasi layar berubah.

Baris 4 `import java.text.NumberFormat` digunakan untuk mengimpor kelas yang bisa memformat angka menjadi format uang. Baris 5 `import kotlin.math.ceil` digunakan untuk mengimpor fungsi `ceil`, yang membulatkan angka ke atas. Baris 7 `class TipViewModel: ViewModel()` mendeklarasikan kelas `TipViewModel` yang mewarisi `ViewModel`. Untuk Menyimpan data perhitungan tip agar tidak hilang saat terjadi rotasi layar. Baris 8 `var tipResult: String = ""` adalah variabel untuk menyimpan hasil perhitungan tip dalam bentuk teks. Nilainya kosong saat awal. Baris 10–14 adalah fungsi `calculateTip` yang bertugas menghitung jumlah tip, Baris 10 `fun calculateTip(...)` mendeklarasikan fungsi dengan tiga parameter: `cost` (biaya), `tipPercent` (persentase tip), dan `roundUp` (apakah dibulatkan atau tidak). Baris 11 `var tip = cost * tipPercent` menghitung jumlah tip dengan mengalikan biaya dan persentase tip. Baris 12 `if (roundUp) tip = ceil(tip)` akan membulatkan nilai tip ke atas jika opsi `roundUp` dipilih. Baris 13 `tipResult = NumberFormat.getCurrencyInstance().format(tip)` mengubah nilai tip menjadi format mata uang dan menyimpannya di `tipResult`. Baris 14 `return tipResult` mengembalikan hasil dalam bentuk teks yang sudah diformat.

5. Values/themes.xml

Baris 1 `<resources xmlns:tools="http://schemas.android.com/tools">` adalah pembuka file resource yang berisi kumpulan style (gaya tampilan) untuk aplikasi. Baris 3–10 adalah style bernama `Base.Theme.TipTime` yang mewarisi tampilan dari `Theme.Material3.DayNight.NoActionBar`, artinya aplikasi akan mendukung mode terang dan gelap, serta tidak menampilkan Action Bar di atas. Mengatur warna utama, dan status bar menjadi warna putih

Baris 11 adalah komentar (tidak dijalankan), menunjukkan alternatif warna yang bisa digunakan jika ingin. Baris 13 `<style name="Theme.TipTime" parent="Base.Theme.TipTime" />` membuat style lain bernama `Theme.TipTime` yang menggunakan gaya dasar dari `Base.Theme.TipTime`. Baris 15–19 adalah style khusus untuk splash screen `Theme.App.SplashScreen` mewarisi `Theme.SplashScreen`. `windowSplashScreenBackground` mengatur warna latar belakang splash screen jadi putih. `windowSplashScreenAnimatedIcon` menentukan gambar ikon yang muncul saat splash screen, diambil dari `@drawable/android`. `postSplashScreenTheme` menetapkan tema yang dipakai setelah splash screen selesai, yaitu `Base.Theme.TipTime`.

6. AndroidManifest.xml

Baris 1 `<manifest>` adalah bagian utama dari file ini yang berfungsi sebagai identitas dan pengatur izin serta komponen utama aplikasi Android. Baris 2–3 menambahkan namespace yang diperlukan untuk menggunakan atribut Android dan alat bantu dari Android Studio. Baris 5–27 `<application>` adalah bagian yang mendefinisikan pengaturan utama aplikasi, seperti ikon, nama, tema, dan pengaturan backup. `android:allowBackup="true"` artinya data aplikasi bisa dibackup oleh sistem. `android:dataExtractionRules` dan `android:fullBackupContent` merujuk ke file XML yang berisi aturan backup data. `android:icon` dan `android:roundIcon` mengatur ikon aplikasi. `android:label="@string/app_name"` menampilkan nama aplikasi dari file string. `android:supportsRtl="true"` mendukung tata letak dari kanan ke `android:theme="@style/Theme.TipTime"` menetapkan tema utama aplikasi. `tools:targetApi="31"` menunjukkan target versi API Android saat pengujian di Android Studio. Baris 15-17 `<activity android:name=".ViewModel">` mendeklarasikan aktivitas

ViewModel sebagai bagian dari aplikasi, tetapi tidak bisa diakses langsung dari luar aplikasi (`exported="false"`). Baris 18–25 mendeklarasikan aktivitas `MainActivity` yang merupakan halaman pertama yang terbuka saat aplikasi dijalankan (`intent-filter` dengan aksi `MAIN` dan kategori `LAUNCHER`). `android:exported="true"` artinya aktivitas ini bisa diakses dari luar (dibutuhkan untuk aktivitas yang jadi titik masuk). `android:theme="@style/Theme.App.SplashScreen"` menetapkan bahwa `MainActivity` akan memakai tema splash screen saat pertama kali dibuka.

MODUL 3: BUILD A SCROLLABLE LIST

SOAL 1

Buatlah sebuah aplikasi Android menggunakan XML atau Jetpack Compose yang dapat menampilkan list dengan ketentuan berikut:

1. List menggunakan fungsi RecyclerView (XML) atau LazyColumn (Compose)
2. List paling sedikit menampilkan 5 item. Tema item yang ingin ditampilkan bebas
3. Item pada list menampilkan teks dan gambar sesuai dengan contoh di bawah
4. Terdapat 2 button dalam list, dengan fungsi berikut:
 - a. Button pertama menggunakan intent eksplisit untuk membuka aplikasi atau browser lain
 - b. Button kedua menggunakan Navigation component/intent untuk membuka laman detail item
5. Sudut item pada list dan gambar di dalam list melengkung atau rounded corner menggunakan Radius
6. Saat orientasi perangkat berubah/dirotasi, baik ke portrait maupun landscape, aplikasi responsif dan dapat menunjukkan list dengan baik. Data di dalam list tidak boleh hilang
7. Aplikasi menggunakan arsitektur single activity (satu activity memiliki beberapa fragment)
8. Aplikasi berbasis XML harus menggunakan ViewBinding

A. Source Code

1. MainActivity.kt

Tabel 10 Source Code Jawaban MainActivity.kt Modul 3

```
1 package com.example.recyclerview
2
3 import android.os.Bundle
4 import androidx.appcompat.app.AppCompatActivity
5
6 class MainActivity : AppCompatActivity() {
7     override fun onCreate(savedInstanceState:
8 Bundle?) {
9         super.onCreate(savedInstanceState)
10        setContentView(R.layout.activity_main)
11
12        val fragmentManager =
13        supportFragmentManager
14        val HomeFragment = HomeFragment()
15        val fragment =
16        fragmentManager.findFragmentByTag (HomeFragment::
17        class.java.simpleName)
18        if (fragment != HomeFragment) {
19            fragmentManager
20            .beginTransaction()
21            .add(R.id.frame_container,
22            HomeFragment,
23            HomeFragment::class.java.simpleName)
24            .commit()
25        }
26    }
27 }
```

2. Drama.kt

Tabel 11 Source Code Jawaban Drama.kt Modul 3

```
1 package com.example.recyclerview
2
3 import android.os.Parcelable
4 import kotlinx.parcelize.Parcelize
5
6 @Parcelize
7 data class Drama(
8     val title: String,
9     val link: String,
10    val photo: String,
11    val plot: String,
12    val year: String,
13    val cast: String
14 ) : Parcelable
```


3. HomeFragment.kt

Tabel 12 Source Code Jawaban HomeFragment.kt Modul 3

```
1 package com.example.recyclerview
2
3 import android.content.Intent
4 import android.net.Uri
5 import android.os.Bundle
6 import android.view.LayoutInflater
7 import android.view.View
8 import android.view.ViewGroup
9 import androidx.fragment.app.Fragment
10 import
11 androidx.recyclerview.widget.LinearLayoutManager
12 import
13 com.example.recyclerview.databinding.FragmentHomeBinding
14
15 class HomeFragment : Fragment() {
16     private var _binding: FragmentHomeBinding? =
17     null
18     private val binding get() = _binding!!
19
20     override fun onCreateView(
21         inflater: LayoutInflater, container:
22         ViewGroup?,
23         savedInstanceState: Bundle?
24     ): View? {
25         _binding =
26         FragmentHomeBinding.inflate(inflater, container,
27         false)
28         return binding.root
29     }
30
31     override fun onViewCreated(view: View,
32         savedInstanceState: Bundle?) {
33         super.onViewCreated(view,
34         savedInstanceState)
35         binding.rvDrama.layoutManager =
36         LinearLayoutManager(requireContext())
37         binding.rvDrama.setHasFixedSize(true)
38         val dramaAdapter =
39         DramaAdapter(getDramaList(),
40             onWikiClick = { link ->
41                 val uri = Uri.parse(link)
42                 startActivity(Intent(Intent.ACTION_VIEW, uri))
43             },
44             onDetailClick = { title, photo,
45                 plot, year, cast ->
```

```

36         val detailFragment =
DetailFragment().apply {
37             arguments = Bundle().apply {
38                 putString("EXTRA_TITLE",
title)
39                 putInt("EXTRA_PHOTO",
photo)
40                 putString("EXTRA_PLOT",
plot)
41                 putString("EXTRA_YEAR",
year)
42                 putString("EXTRA_CAST",
cast)
43             }
44         }
45     parentFragmentManager.beginTransaction()
46     .replace(R.id.frame_container, detailFragment)
47     .addToBackStack(null)
48     .commit()
49     })
50     binding.rvDrama.adapter = dramaAdapter
51 }
52
53     private fun getDramaList(): ArrayList<Drama>
54 {
55         val dataTitle =
resources.getStringArray(R.array.data_name)
56         val dataLink =
resources.getStringArray(R.array.data_link)
57         val dataPhoto =
resources.obtainTypedArray(R.array.data_photo)
58         val dataPlot =
resources.getStringArray(R.array.data_plot)
59         val dataYear =
resources.getStringArray(R.array.data_year)
60         val dataCast =
resources.getStringArray(R.array.data_cast)
61         val listDrama = ArrayList<Drama>()
62         for (i in dataTitle.indices) {
63             val drama = Drama(
64                 dataTitle[i],
65                 dataLink[i],
66                 dataPhoto.getResourceId(i, -1),
67                 dataPlot[i],
68                 dataYear[i],
69                 dataCast[i]
70             )
71             listDrama.add(drama)
72         }

```

73	dataPhoto.recycle()
74	return listDrama
75	}
76	override fun onDestroyView() {
77	super.onDestroyView()
78	_binding = null
79	}
80	}
81	

4. DetailFragment

Tabel 13 Source Code Jawaban DetailFragment.kt Modul 3

1	package com.example.recyclerview
2	
3	import android.os.Bundle
4	import androidx.fragment.app.Fragment
5	import android.view.LayoutInflater
6	import android.view.View
7	import android.view.ViewGroup
8	import com.bumptech.glide.Glide
9	import
	com.example.recyclerview.databinding.FragmentDetailBinding
10	
11	class DetailFragment : Fragment() {
12	
13	private var _binding: FragmentDetailBinding?
	= null
14	private val binding get() = _binding!!
15	
16	override fun onCreateView(
17	inflater: LayoutInflater, container:
	ViewGroup?,
18	savedInstanceState: Bundle?
19	): View? {
20	_binding =
	FragmentDetailBinding.inflate(inflater,
	container, false)
21	
22	val title =
	arguments?.getString("EXTRA_TITLE")
23	val photo =
	arguments?.getString("EXTRA_PHOTO")
24	val plot =
	arguments?.getString("EXTRA_PLOT")
	val year =
25	arguments?.getString("EXTRA_YEAR")

```

26         val cast =
arguments?.getString("EXTRA_CAST")
27
28         binding.tvName.text = title
29         binding.tvPlot.text = plot
30         binding.tvYear.text = "Year: $year"
31         binding.tvCast.text = "Cast: $cast"
32         photo?.let {
33             Glide.with(requireContext())
34                 .load(it)
35                 .into(binding.imgItemPhoto)
36         }
37
38         binding.buttonKembali.setOnClickListener
39     {
40         parentFragmentManager.popBackStack()
41     }
42     return binding.root
43 }
44 override fun onDestroyView() {
45     super.onDestroyView()
46     _binding = null
47 }
48 }

```

5. DramaAdapter.kt

Tabel 14 Source Code Jawaban DramaAdapter.kt Modul 3

```

1 package com.example.recyclerview
2
3 import android.view.LayoutInflater
4 import android.view.View
5 import android.view.ViewGroup
6 import android.widget.Button
7 import android.widget.ImageView
8 import android.widget.TextView
9 import androidx.recyclerview.widget.RecyclerView
10 import com.bumptech.glide.Glide
11
12 class DramaAdapter(
13     private val listDrama: ArrayList<Drama>,
14     private val onWikiClick: (String) -> Unit,
15     private val onDetailClick: (String, String,
String, String, String) -> Unit
16 ) :
RecyclerView.Adapter<DramaAdapter.ViewHolder>()
17 {
18     class ViewHolder(itemView: View) :
RecyclerView.ViewHolder(itemView) {

```

```

18         val imgPhoto: ImageView =
itemView.findViewById(R.id.img_item_photo)
19         val tvTitle: TextView =
itemView.findViewById(R.id.tv_item_name)
20         val tvPlot: TextView =
itemView.findViewById(R.id.tv_item_plot)
21         val btnWiki: Button =
itemView.findViewById(R.id.btn_wiki)
22         val btnDetail: Button =
itemView.findViewById(R.id.button_detail)
23     }

24     override fun onCreateView(parent:
25 ViewGroup, viewType: Int): ViewHolder {
26         val view: View =
LayoutInflater.from(parent.context).inflate(R.la
27 yout.item_list, parent, false)
        return ViewHolder(view)
    }

28
29     override fun getItemCount(): Int =
30 listDrama.size
31
        override fun onBindViewHolder(holder:
32 ViewHolder, position: Int) {
33         val (title, link, photo, plot, year,
cast) = listDrama[position]
34         holder.tvTitle.text = title
        Glide.with(holder.itemView.context)
35             .load(photo)
36             .into(holder.imgPhoto)
37         holder.tvPlot.text = "Plot: $plot"
38         holder.btnWiki.setOnClickListener {
39 onWikiClick(link) }
40         holder.btnDetail.setOnClickListener {
onDetailClick(title, photo, plot, year, cast) }
42     }
43
44

```

6. Activity_main.xml

Tabel 15 Source Code Jawaban Activity main.xml Modul 3

1	<androidx.constraintlayout.widget.ConstraintLayo
2	ut
3	xmlns:android="http://schemas.android.com/apk/re
4	s/android"
5	xmlns:app="http://schemas.android.com/apk/res-
6	auto"
7	xmlns:tools="http://schemas.android.com/tools"
8	android:layout_width="match_parent"
9	android:layout_height="match_parent"
10	tools:context=".MainActivity">
11	
12	<FrameLayout
13	android:id="@+id/frame_container"
14	android:layout_width="match_parent"
15	android:layout_height="match_parent" />
16	
17	</androidx.constraintlayout.widget.ConstraintLayo
18	ut>

7. fragment_home.xml

Tabel 16 Source Code Jawaban fragment home.xml Modul 3

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayo
3	ut
4	xmlns:android="http://schemas.android.com/apk/re
5	s/android"
6	xmlns:app="http://schemas.android.com/apk/res-
7	auto"
8	xmlns:tools="http://schemas.android.com/tools"
9	android:layout_width="match_parent"
10	android:layout_height="match_parent"
11	tools:context=".HomeFragment">
12	
13	<androidx.recyclerview.widget.RecyclerView
14	android:id="@+id/rv_Drama"
15	android:layout_width="0dp"
16	android:layout_height="0dp"
17	android:layout_margin="15dp"
18	
19	app:layout_constraintTop_toTopOf="parent"
20	
21	app:layout_constraintBottom_toBottomOf="parent"
22	
23	app:layout_constraintStart_toStartOf="parent"
24	

19	
20	app:layout_constraintEnd_toEndOf="parent" /> </androidx.constraintlayout.widget.ConstraintLayout>

8. fragment_detail.xml

Tabel 17 Source Code Jawaban fragment_detail.xml Modul 3

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	xmlns:tools="http://schemas.android.com/tools"
6	android:layout_width="match_parent"
7	android:layout_height="match_parent"
8	android:background="@drawable/background_image"
9	tools:context=".DetailFragment">
10	
11	<ImageView
12	android:id="@+id/img_item_photo"
13	android:layout_width="341dp"
14	android:layout_height="458dp"
15	android:layout_marginTop="32dp"
16	android:scaleType="centerCrop"
17	app:layout_constraintEnd_toEndOf="parent"
18	app:layout_constraintStart_toStartOf="parent"
19	app:layout_constraintTop_toTopOf="parent"
20	tools:src="@drawable/truebeauty" />
21	
22	<TextView
23	android:id="@+id/tv_name"
24	android:layout_width="wrap_content"
25	android:layout_height="wrap_content"
26	android:textStyle="bold"
27	android:textSize="20sp"
28	android:layout_marginTop="16dp"
29	app:layout_constraintTop_toBottomOf="@id/img_item_photo"
30	app:layout_constraintStart_toStartOf="parent"
31	app:layout_constraintEnd_toEndOf="parent"
32	tools:text="Drama Title" />
33	
34	<TextView
35	android:id="@+id/tv_year"
36	android:layout_width="wrap_content"
37	android:layout_height="wrap_content"
38	android:text="Year: "

39	android:textSize="16sp"
40	android:layout_marginTop="8dp"
41	app:layout_constraintTop_toBottomOf="@id/tv_name"
42	app:layout_constraintStart_toStartOf="parent"
43	app:layout_constraintEnd_toEndOf="parent"
	/>
44	
45	<TextView
46	android:id="@+id/tv_cast"
47	android:layout_width="0dp"
48	android:layout_height="wrap_content"
49	android:text="Cast: "
50	android:textSize="16sp"
51	android:layout_marginTop="8dp"
52	android:layout_marginHorizontal="16dp"
53	android:textAlignment="center"
54	app:layout_constraintTop_toBottomOf="@id/tv_year"
55	app:layout_constraintStart_toStartOf="parent"
56	app:layout_constraintEnd_toEndOf="parent" />
57	
58	<TextView
59	android:id="@+id/tv_plot"
60	android:layout_width="0dp"
61	android:layout_height="wrap_content"
62	android:layout_margin="16dp"
63	android:textAlignment="center"
64	android:textSize="16sp"
65	android:text="Plot: "
66	app:layout_constraintTop_toBottomOf="@id/tv_cast"
67	app:layout_constraintStart_toStartOf="parent"
68	app:layout_constraintEnd_toEndOf="parent"
	/>
69	
70	</androidx.constraintlayout.widget.ConstraintLayout>

9. item_list.xml

Tabel 18 Source Code Jawaban item_list.xml Modul 3

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.cardview.widget.CardView
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	


```

5   xmlns:app="http://schemas.android.com/apk/res-
6   auto"
7       android:layout_width="match_parent"
8       android:layout_height="wrap_content"
9       xmlns:tools="http://schemas.android.com/tools"
10      android:layout_margin="8dp"
11      app:cardCornerRadius="8dp"
12      app:cardElevation="4dp">
13  <androidx.constraintlayout.widget.ConstraintLayo
14      ut
15          android:layout_width="match_parent"
16          android:layout_height="wrap_content"
17          android:padding="8dp">
18      <ImageView
19          android:id="@+id/img_item_photo"
20          android:layout_width="70dp"
21          android:layout_height="100dp"
22          android:scaleType="centerCrop"
23      app:layout_constraintStart_toStartOf="parent"
24      app:layout_constraintTop_toTopOf="parent"
25      app:layout_constraintBottom_toBottomOf="parent"
26          tools:src="@drawable/truebeauty" />
27
28      <TextView
29          android:id="@+id/tv_item_name"
30          android:layout_width="0dp"
31          android:layout_height="wrap_content"
32          android:textSize="16sp"
33          android:textStyle="bold"
34      app:layout_constraintStart_toEndOf="@id/img_item
35      _photo"
36      app:layout_constraintTop_toTopOf="parent"
37      app:layout_constraintEnd_toEndOf="parent"
38          android:layout_marginStart="12dp"
39          tools:text="True Beauty" />
40
41      <TextView
42          android:id="@+id/tv_item_plot"
43          android:layout_width="0dp"
44          android:layout_height="wrap_content"
45          android:layout_marginTop="4dp"
46          android:textSize="13sp"
47      app:layout_constraintTop_toBottomOf="@id/tv_item
48      _name"
49      app:layout_constraintStart_toStartOf="@id/tv_ite
50      m_name"
51      app:layout_constraintEnd_toEndOf="parent"

```

49	tools:text="Plot: " />
50	
51	<Button
52	android:id="@+id/btn_wiki"
53	android:layout_width="wrap_content"
54	android:layout_height="wrap_content"
55	android:layout_marginStart="24dp"
56	android:layout_marginTop="8dp"
57	android:backgroundTint="#ffc5d3"
	android:text="IMDB"
58	app:layout_constraintStart_toStartOf="@id/tv_item_plot"
59	app:layout_constraintTop_toBottomOf="@id/tv_item_plot" />
60	
61	<Button
62	android:id="@+id/button_detail"
63	android:layout_width="wrap_content"
64	android:layout_height="wrap_content"
65	android:layout_marginStart="8dp"
66	android:backgroundTint="#ffc5d3"
67	android:text="Detail"
68	app:layout_constraintStart_toEndOf="@id/btn_wiki"
69	app:layout_constraintTop_toTopOf="@id/btn_wiki" />
70	
71	
72	</androidx.constraintlayout.widget.ConstraintLayout>
73	</androidx.cardview.widget.CardView>

10. string.xml

Tabel 19 Source Code Jawaban string.xml Modul 3

1	<resources>
2	<string name="app_name">modul3</string>
3	<string name="btn_detail">Detail</string>
4	<string name="btn_wiki">Wiki</string>
5	
6	<string-array name="data_name">
7	<item>True Beauty</item>
8	<item>Business Proposal</item>
9	<item>King the Land</item>
10	<item>Welcome to Samdalri</item>
11	<item>Love Next Door</item>
12	<item>Family by Choice</item>
13	<item>Lovely Runner</item>
14	<item>Melo Movie</item>

```

15     </string-array>
16
17     <string-array name="data_photo">
18         <item>@drawable/truebeauty</item>
19         <item>@drawable/businessproposal</item>
20         <item>@drawable/kingtheland</item>
21         <item>@drawable/welcometosamdalri</item>
22         <item>@drawable/lovenextdoor</item>
23         <item>@drawable/familybychoice</item>
24         <item>@drawable/lovelyrunner</item>
25         <item>@drawable/melomovie</item>
26     </string-array>
27
28     <string-array name="data_link">
29     <item>https://www.netflix.com/id/title/81410834<
30 /item>
31     <item>https://www.netflix.com/id-
32 en/title/81509440</item>
33     <item>https://www.netflix.com/id/title/81671440<
34 /item>
35     <item>https://www.netflix.com/id/title/81697983<
36 /item>
37     <item>https://www.netflix.com/id-
38 en/title/81739044</item>
39
40     <item>https://www.viu.com/ott/id/id/vod/2435994/
41 Family-By-Choice</item>
42     <item>https://www.netflix.com/id-
43 en/title/81914565</item>
44     <item>https://www.netflix.com/id-
45 en/title/81720593</item>
46 </string-array>
47
48     <string-array name="data_plot">
49         <item>True Beauty is a high school drama
50 about a girl who becomes a beauty through makeup
51 and faces challenges when her secret is
52 revealed.</item>
53         <item>Business Proposal tells the story
54 of a woman who participates in a business
55 proposal pretending to be her friend, only to
56 find out the man is her company s CEO</item>
57         <item>King the Land is about a CEO who
58 tries to open a new hotel and a woman who works
59 there while struggling with her own love
60 story.</item>
61         <item>Welcome to Samdalri is about a
62 group of individuals who meet in a small village

```

44	and form bonds through humorous and touching moments.</item>
45	<item>Love Next Door is a romantic drama revolving around a couple living next door and their developing relationship.</item>
46	<item>Family by Choice is a heartwarming drama about a group of people who form a family despite not being related by blood.</item>
47	<item>Lovely Runner follows a runner who is trying to make it in the world of competitive sports while balancing her personal life.</item>
48	<item>Melo Movie is a romantic comedy about a woman trying to find love while struggling with her career in the film industry.</item>
49	</string-array>
50	
51	<string-array name="data_year">
52	<item>2020</item>
53	<item>2022</item>
54	<item>2023</item>
55	<item>2023</item>
56	<item>2024</item>
57	<item>2024</item>
58	<item>2024</item>
59	<item>2025</item>
60	</string-array>
61	
62	<string-array name="data_cast">
63	<item>Moon Ga-young, Cha Eun-woo, Hwang In-yeop</item>
64	<item>Kim Se-jeong, Ahn Hyo-seop, Kim Min Kyu, Seol In Ah</item>
65	<item>YoonA, Lee Jun-ho</item>
66	<item>Ji Chang-wook, Shin Hye-sun</item>
67	<item>Jung Hae In, Jung So Min, Kim Ji Eun, Yun Ji On</item>
68	<item>Hwang In Yeop, Jung Chae Yeon, Bae Hyun Sung</item>
69	<item>Byeon Woo-seok, Kim Hye-yoon</item>
70	<item>Choi Woo Shik, Park Bo Young, Lee Jun Young, Jeon So Nee</item>
71	</string-array>
72	
73	
74	
75	<string name="hello_blank_fragment">Hello blank fragment</string>

	</resources>
--	--------------

11. build.gradle.kts (module:app)

Tabel 20 Source Code Jawaban build.gradle.kts Modul 3

1	plugins {
2	alias(libs.plugins.android.application)
3	alias(libs.plugins.kotlin.android)
4	id("kotlin-parcelize")
5	id("org.jetbrains.kotlin.kapt")
6	}
7	
8	android {
9	namespace = "com.example.recyclerview"
10	compileSdk = 35
11	
12	defaultConfig {
13	applicationId =
14	"com.example.recyclerview"
15	minSdk = 30
16	targetSdk = 35
17	versionCode = 1
18	versionName = "1.0"
19	
20	testInstrumentationRunner =
21	"androidx.test.runner.AndroidJUnitRunner"
22	}
23	
24	buildTypes {
25	release {
26	isMinifyEnabled = false
27	proguardFiles(
28	getDefaultProguardFile("proguard-android-
29	optimize.txt"),
30	"proguard-rules.pro"
31	)
32	}
33	}
34	compileOptions {
35	sourceCompatibility =
36	JavaVersion.VERSION_11
37	targetCompatibility =
	JavaVersion.VERSION_11
	}
	kotlinOptions {
	jvmTarget = "11"
	}

38	buildFeatures {
39	viewBinding = true
40	}
41	}
42	
43	dependencies {
44	
45	implementation(libs.androidx.core.ktx)
46	implementation(libs.androidx.appcompat)
47	implementation(libs.material)
48	implementation(libs.androidx.activity)
49	implementation("androidx.recyclerview:recyclerview:1.3.1")
50	implementation
	("androidx.constraintlayout:constraintlayout:2.1.4")
51	testImplementation(libs.junit)
52	androidTestImplementation(libs.androidx.junit)
53	androidTestImplementation(libs.androidx.espresso
	.core)
54	implementation("com.github.bumptech.glide:glide:4.15.1")
55	kapt("com.github.bumptech.glide:compiler:4.15.1")
	}
56	}
57	
58	fun kapt(s: String) {
59	
60	}

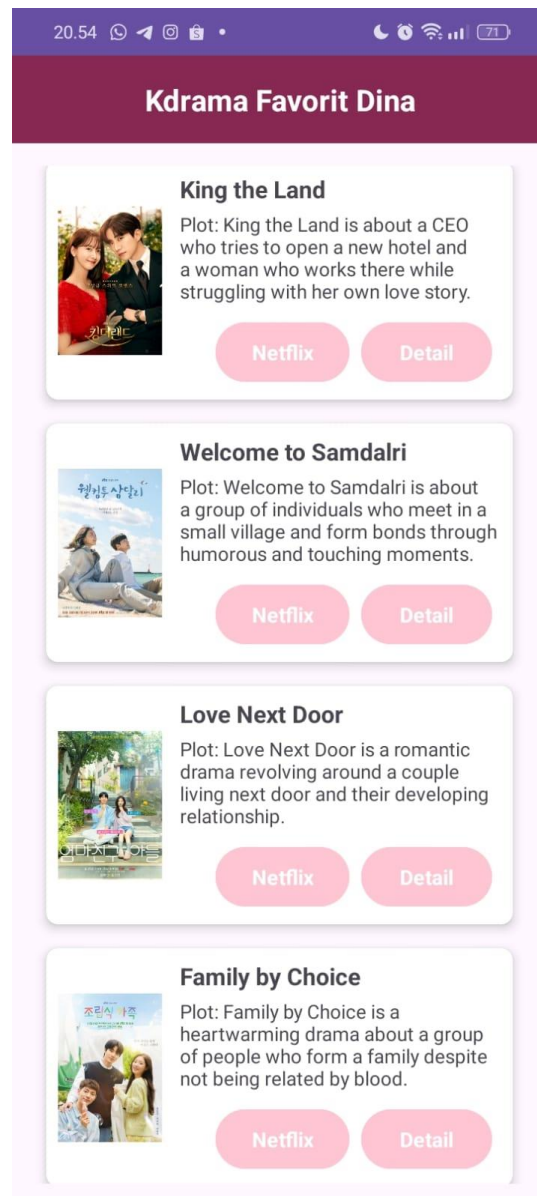
12. AndroidManifest.xml

Tabel 21 Source Code Jawaban AndroidManifest.xml Modul 3

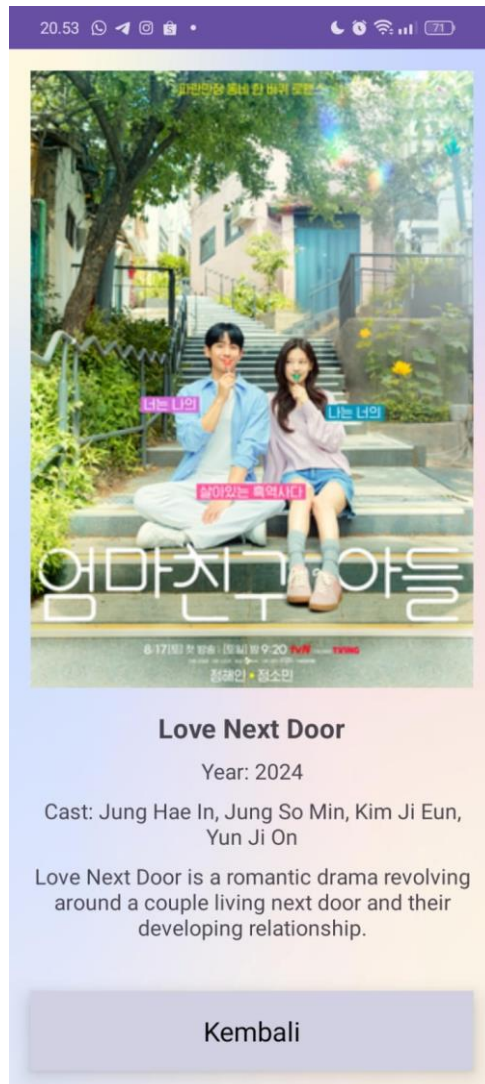
1	<?xml version="1.0" encoding="utf-8"?>
2	<manifest
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	
5	
6	xmlns:tools="http://schemas.android.com/tools">
7	
8	<application
9	android:allowBackup="true"
10	
11	android:dataExtractionRules="@xml/data_extraction_rules"
12	
13	android:fullBackupContent="@xml/backup_rules"
14	android:icon="@mipmap/ic_launcher"
15	android:label="@string/app_name"

```
16
17 android:roundIcon="@mipmap/ic_launcher_round"
18     android:supportsRtl="true"
19
19 android:theme="@style/Theme.RecyclerView"
20     tools:targetApi="31">
21     <activity
22         android:name=".MainActivity"
23         android:exported="true">
24         <intent-filter>
25             <action
26                 android:name="android.intent.action.MAIN" />
27             <category
28                 android:name="android.intent.category.LAUNCHER"
29             />
30         </intent-filter>
31     </activity>
32 </application>
</manifest>
```

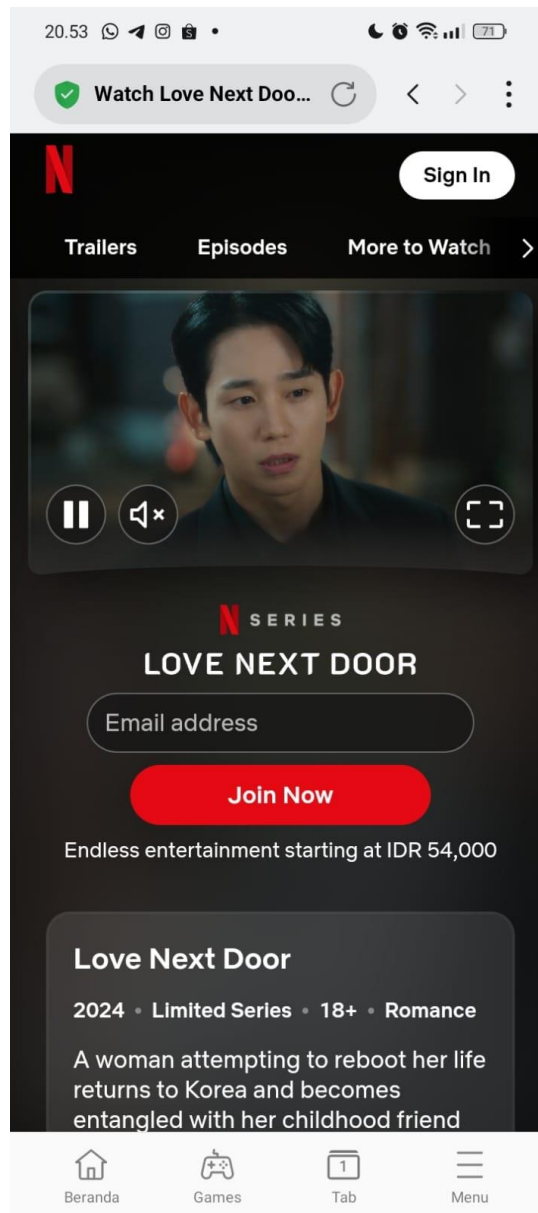
B. Output Program



Gambar 13 Screenshot Hasil Tampilan Halaman Home



Gambar 14 Screenshot Hasil Tampilan Halaman Detail



Gambar 15 Screenshot Hasil Tampilan Link Netflix

C. Pembahasan

1. MainActivity.kt

Pada baris 1 `package com.example.recyclerview` menunjukkan bahwa file Kotlin ini berada di dalam paket bernama `recyclerview` di bawah domain `com.example`. Baris 3–4 adalah bagian impor yang memungkinkan kelas ini menggunakan fitur seperti `Bundle` dan `AppCompatActivity` dari Android SDK. Baris 6 mendefinisikan kelas `MainActivity` yang merupakan titik masuk utama ketika aplikasi dijalankan. Baris 7 menjalankan fungsi `onCreate()` yang otomatis dipanggil saat activity dibuat. Baris 9 memanggil `setContentView(R.layout.activity_main)` untuk menampilkan layout `activity_main.xml` sebagai antarmuka pengguna. Baris 11 membuat objek `FragmentManager` untuk mengelola fragmen dalam activity ini. Baris 12 membuat instance dari kelas `HomeFragment`. Baris 13–14 mengecek apakah fragmen dengan tag yang sama sudah ditambahkan sebelumnya, dan jika belum, maka fragmen baru akan ditambahkan ke dalam `frame_container` dengan memanggil `beginTransaction()`, lalu `commit()` untuk menyimpan perubahan.

2. Drama.kt

Pada bagian awal, kode `package com.example.recyclerview` digunakan untuk mengelompokkan file ini ke dalam paket `recyclerview` agar lebih terorganisir dalam struktur proyek. Baris impor `import android.os.Parcelable` dan `import kotlinx.parcelize.Parcelize` memungkinkan kelas `Drama` dapat dikirim antar komponen Android dengan cara efisien menggunakan parcel. Anotasi `@Parcelize` digunakan supaya proses implementasi `Parcelable` tidak perlu dilakukan secara manual, cukup dengan menandai data class tersebut. Kelas `Drama` bertipe data class yang menyimpan informasi seperti `title`, `link`, `photo`, `plot`, `year`, dan `cast` yang masing-masing bertipe `String`. Kelas ini mengimplementasikan `Parcelable`, sehingga objeknya dapat dipaketkan dan ditransfer antar Activity atau Fragment.

3. HomeFragment.kt

Pada baris 1 `package com.example.recyclerview` menunjukkan bahwa file Kotlin ini berada dalam paket bernama `recyclerview` di bawah domain `com.example`. Baris 3-11 `import` yang mengikuti memungkinkan penggunaan berbagai komponen Android seperti `Intent`, `Uri`, dan `Fragment`. Baris 13 `class HomeFragment : Fragment()` mendeklarasikan bahwa `HomeFragment` adalah turunan dari kelas `Fragment`. Baris 15-16 `private var _binding: FragmentHomeBinding? = null` digunakan untuk menginisialisasi view binding guna mengakses elemen UI tanpa perlu `findViewById`. Baris 18-51 `onCreateView` meng-inflate layout dari `FragmentHomeBinding` dan mengembalikannya sebagai tampilan utama dari fragmen. Di dalam `onViewCreated`, `RecyclerView` disiapkan dengan `LinearLayoutManager` untuk menampilkan daftar secara vertikal dan ukuran tetap. Sebuah adapter `DramaAdapter` diinisialisasi dengan data dari `getDramaList()` serta dua lambda untuk aksi klik: satu membuka tautan Netflix menggunakan `Intent.ACTION_VIEW`, dan satu lagi mengganti fragmen saat tombol detail diklik dengan meneruskan data ke `DetailFragment` melalui `Bundle`. Fungsi pada baris 53-73 `getDramaList()` mengambil data dari resources (seperti judul, foto, plot, tahun, dan pemeran), lalu memasukkannya ke dalam list bertipe `Drama`. Terakhir, baris 75-78 `onDestroyView` bertugas untuk menghapus binding agar tidak terjadi memory leak saat fragment dihancurkan.

4. DetailFragment.kt

Pada baris 1 `package com.example.recyclerview` menunjukkan bahwa file Kotlin ini berada dalam paket bernama `recyclerview` di bawah domain `com.example`. Baris 3-8 `import` yang mengikuti memungkinkan penggunaan berbagai komponen Android seperti `Bundle`, `Fragment`, dan `ViewBinding`. Baris 10 `class DetailFragment : Fragment()` mendeklarasikan bahwa `DetailFragment` merupakan subclass dari

Fragment. Baris 12-13 `private var _binding: FragmentDetailBinding? = null` digunakan untuk mengelola binding agar bisa mengakses elemen UI dari `fragment_detail.xml` secara efisien. Baris 15-34 di dalam `onCreateView`, layout difungsikan melalui `FragmentDetailBinding.inflate`, lalu data yang dikirim melalui `arguments` diambil menggunakan kunci seperti `EXTRA_TITLE`, `EXTRA_PHOTO`, dan lainnya. Nilai-nilai tersebut kemudian diatur ke elemen UI yang sesuai seperti `tvName`, `tvPlot`, `tvYear`, `tvCast`, dan gambar `imgItemPhoto` apabila tersedia. Fungsi ini mengembalikan tampilan dari root binding. Baris 32-26 digunakan library Glide. Glide adalah library pihak ketiga yang efisien untuk menangani pemuatan gambar dari internet. Pada baris yang menggunakan `Glide.with(requireContext()).load(it).into(binding.imgItemPhoto)`, Glide digunakan untuk memuat gambar dari URL yang diperoleh dari variabel `photo`, kemudian menampilkannya ke dalam elemen `ImageView` bernama `imgItemPhoto`. Terakhir, pada baris 38-48 fungsi `onDestroyView` memastikan binding dihapus untuk mencegah kebocoran memori saat fragment dihancurkan.

5. DramaAdapter

Pada baris 1 `package com.example.recyclerview` menunjukkan bahwa file Kotlin ini berada dalam paket bernama `recyclerview` di bawah domain `com.example`. Baris 3-9 `import` yang mengikuti memungkinkan penggunaan komponen Android seperti `RecyclerView`, `View`, `ImageView`, dan `TextView`. Baris 11 class `DramaAdapter` mendefinisikan adapter untuk `RecyclerView` dengan parameter daftar data bertipe `Drama`, lambda `onWikiClick` untuk membuka tautan, dan `onDetailClick` untuk membuka detail drama. Pada baris 17-23 di dalam kelas ini terdapat `ViewHolder` yang merepresentasikan satu item tampilan dan menghubungkan komponen UI seperti gambar, judul, plot, dan dua tombol aksi dari `layout_item_list.xml`. Baris 25 Fungsi `onCreateViewHolder` bertugas untuk

meng-inflate layout item dan mengembalikannya sebagai objek ViewHolder. Baris 30 Fungsi getItemCount mengembalikan jumlah item dalam daftar data. Pada baris 33-43 onBindViewHolder, data drama diambil berdasarkan posisi lalu diatur ke elemen tampilan masing-masing, dan dua tombol diberi listener sesuai fungsi yang diterima melalui konstruktor adapter. Dalam baris ini terdapat `glide.Glide.with(holder.itemView.context).load(photo).into(holder.imgPhoto)` menunjukkan bahwa Glide akan menggunakan context dari item view saat ini, memuat gambar dari URL yang diberikan oleh variabel photo, lalu menampilkannya ke dalam ImageView yang memiliki ID `img_item_photo`.

6. Activity_Main.xml

Baris 1
`<androidx.constraintlayout.widget.ConstraintLayout>`
 adalah layout utama yang digunakan sebagai wadah seluruh elemen UI di halaman ini. Baris 2-4 adalah deklarasi namespace XML yang memungkinkan penggunaan atribut Android, app (untuk ConstraintLayout), dan tools. Baris 5-7 mendefinisikan ukuran layout agar memenuhi seluruh layar serta menetapkan konteks tools untuk keperluan preview di Android Studio. Baris 9-12 `<FrameLayout>` dengan id `frame_container` digunakan sebagai kontainer fragmen yang akan diganti-ganti secara dinamis, misalnya saat navigasi antar halaman. Terakhir, baris 14 menutup tag ConstraintLayout sebagai akhir dari struktur layout ini.

7. fragment_home.xml

Baris 1
`<androidx.constraintlayout.widget.ConstraintLayout>`
 digunakan sebagai layout utama yang memungkinkan penempatan elemen UI secara fleksibel dengan constraint. Baris 2-4 adalah deklarasi namespace XML standar agar atribut seperti `android:`, `app:`, dan `tools:` dapat digunakan dengan benar. Baris 5-7 mengatur layout agar mengisi seluruh layar dan

menyetel konteks preview di Android Studio ke HomeFragment. Baris 9-22 `<TextView>` dengan id `tvHeader` menampilkan teks dari resource `@string/header` sebagai judul halaman dengan ukuran 20sp, teks putih tebal, latar belakang ungu gelap, dan posisi berada di bagian atas layout serta disejajarkan secara horizontal di tengah. Baris 24-32 `<androidx.recyclerview.widget.RecyclerView>` dengan id `rv_Drama` digunakan untuk menampilkan daftar drama dalam bentuk list yang bisa discroll secara vertikal, ditempatkan tepat di bawah `tvHeader` dengan margin di sekelilingnya, dan memenuhi sisa ruang layar. Terakhir, baris 33 menutup tag `ConstraintLayout` sebagai akhir dari struktur layout ini.

8. fragment_detail.xml

Baris 1
`<androidx.constraintlayout.widget.ConstraintLayout>`
 digunakan sebagai layout utama yang memungkinkan penempatan elemen UI secara fleksibel dengan constraint. Baris 3-5 adalah deklarasi namespace XML standar agar atribut seperti `android:`, `app:`, dan `tools:` dapat digunakan dengan benar. Baris 6-9 mengatur layout agar mengisi seluruh layar, menambahkan latar belakang berupa gambar drawable, dan menyetel konteks preview ke `DetailFragment`. Baris 11-20 `<ImageView>` dengan id `img_item_photo` menampilkan gambar poster drama dengan ukuran tetap, skala `centerCrop`, serta diposisikan di bagian atas layout dengan margin atas 32dp dan posisi horizontal di tengah. Baris 22-32 `<TextView>` dengan id `tv_name` menampilkan judul drama dengan teks tebal berukuran 20sp, ditempatkan tepat di bawah gambar dan disejajarkan di tengah. Baris 34-43 `<TextView>` dengan id `tv_year` digunakan untuk menampilkan tahun rilis drama, dengan ukuran teks 16sp dan margin atas 8dp, diletakkan di bawah judul dan diposisikan secara tengah. Baris 45-56 `<TextView>` dengan id `tv_cast` menampilkan informasi pemeran dengan ukuran teks 16sp dan alignment tengah, diletakkan di bawah tahun dengan margin horizontal 16dp agar isi tidak menempel ke tepi layar. Baris 58-68 `<TextView>` dengan id `tv_plot` menampilkan sinopsis atau plot drama, ditempatkan di bawah informasi cast, dengan margin 16dp dan alignment tengah

agar rapi dibaca. Terakhir, baris 70 menutup tag `ConstraintLayout` sebagai akhir dari struktur layout ini.

9. `item_list.xml`

Baris 1 `<androidx.cardview.widget.CardView>` digunakan sebagai wadah utama dengan sudut membulat dan efek elevasi agar tampilan setiap item terlihat seperti kartu. Baris 3-5 adalah deklarasi namespace XML standar yang memungkinkan penggunaan atribut seperti `android:`, `app:`, dan `tools:`. Baris 5-7 mengatur lebar kartu agar memenuhi parent, tinggi menyesuaikan isi, dengan margin 8dp serta pengaturan radius sudut 8dp dan elevasi 4dp agar tampak sedikit melayang. Baris 12-15 `<androidx.constraintlayout.widget.ConstraintLayout>` digunakan sebagai layout dalam kartu untuk menyusun elemen-elemen UI secara terstruktur. Baris 17-25 `<ImageView>` dengan id `img_item_photo` digunakan untuk menampilkan gambar poster drama berukuran 70x100dp dengan skala `centerCrop`, diposisikan di sisi kiri dan vertikal tengah dalam kartu. Baris 27-37 `<TextView>` dengan id `tv_item_name` menampilkan judul drama dengan teks tebal berukuran 16sp, diletakkan di samping gambar dengan margin awal 12dp dan disejajarkan ke kanan. Baris 39-48 `<TextView>` dengan id `tv_item_plot` menampilkan deskripsi singkat atau plot drama di bawah judul, dengan ukuran teks lebih kecil 13sp dan sedikit margin atas. Baris 52-59 `<Button>` dengan id `btn_wiki` berfungsi untuk membuka tautan Netflix, diberi teks “Netflix”, latar belakang berwarna pink muda, dan diletakkan di bawah plot dengan margin awal 24dp. Baris 61-69 `<Button>` dengan id `button_detail` digunakan untuk menampilkan detail drama, posisinya tepat di samping kanan tombol Netflix dan tetap sejajar secara vertikal. Terakhir, baris 71 menutup tag `ConstraintLayout` dan baris 72 menutup tag `CardView` sebagai akhir dari struktur layout ini

10. string.xml

Baris 1 `<resources>` merupakan elemen root pada file XML ini yang berisi semua data string dan array yang digunakan dalam aplikasi. Baris 2-4 mendefinisikan string statis seperti nama aplikasi dan label pada tombol untuk detail serta wiki. Baris 6-15 `<string-array name="data_name">` menyimpan daftar judul drama Korea yang akan ditampilkan dalam RecyclerView. Baris 17-26 `<string-array name="data_photo">` menyimpan daftar URL gambar dari internet yang digunakan sebagai foto untuk setiap judul drama. Karena aplikasi menggunakan library Glide untuk memuat gambar. Baris 28-37 `<string-array name="data_link">` berisi link menuju halaman resmi masing-masing drama, kebanyakan dari Netflix dan satu dari Viu. Baris 39-48 `<string-array name="data_plot">` berisi deskripsi singkat atau sinopsis dari setiap drama, menjelaskan tema dan alur utama cerita. Baris 50-59 `<string-array name="data_year">` menyimpan tahun rilis masing-masing drama agar bisa ditampilkan pada tampilan detail. Baris 61-70 `<string-array name="data_cast">` memuat daftar pemeran utama dari masing-masing drama. Baris 73 dan 74 menambahkan string statis untuk keperluan default fragment dan judul header halaman. Terakhir, baris 75 menutup tag `<resources>` sebagai akhir dari seluruh deklarasi string dan array yang ada di file ini.

11. Build.gradle.kts

Pada gradle ini saya menambahkan baris 3 yaitu `id("kotlin-parcelize")` digunakan untuk mengaktifkan plugin Kotlin Parcelize yang memungkinkan penggunaan anotasi `@Parcelize` pada kelas data agar objeknya bisa dengan mudah dikirim antar komponen Android seperti Activity atau Fragment melalui Intent atau Bundle. Baris 4 Plugin `id("org.jetbrains.kotlin.kapt")` diaktifkan agar proyek dapat menggunakan **KAPT** (Kotlin Annotation Processing Tool), yang diperlukan ketika menggunakan library yang mengandalkan annotation processor **Glide**. Lalu pada `dependencies` menambahkan `implementation("androidx.recyclerview:recyclerview:1.`

3.1") yang digunakan untuk menyertakan library RecyclerView versi 1.3.1 untuk menampilkan daftar item secara efisien dan fleksibel dalam bentuk list atau grid yang dapat discroll. Dan `implementation("androidx.constraintlayout:constraintlayout:2.1.4")` digunakan untuk memasukkan library ConstraintLayout versi 2.1.4 yang memberikan kontrol layout yang lebih fleksibel dan efisien dengan sistem constraint antar elemen UI. Baris 54 `implementation("com.github.bumptech.glide:glide:4.15.1")` berfungsi untuk mengimpor library utama Glide versi 4.15.1, yang memungkinkan aplikasi Android memuat gambar dari berbagai sumber, termasuk URL di internet. baris `kapt("com.github.bumptech.glide:compiler:4.15.1")` digunakan untuk mengaktifkan dukungan annotation processor yang disediakan oleh Glide.

12. AndroidManifest.xml

Pada file ini, saya menambahkan `<intent-filter>` di dalam deklarasi `<activity android:name=".MainActivity">` yang berfungsi untuk menetapkan MainActivity sebagai titik masuk utama aplikasi. Baris `<action android:name="android.intent.action.MAIN" />` menunjukkan bahwa aktivitas ini adalah titik awal yang dijalankan saat aplikasi dibuka pertama kali, lalu `<category android:name="android.intent.category.LAUNCHER" />` menjadikan aktivitas ini muncul di daftar aplikasi (launcher) pada perangkat Android. Saya juga menambahkan baris `<uses-permission android:name="android.permission.INTERNET"/>` yang merupakan deklarasi izin (permission) yang ditulis di dalam file `AndroidManifest.xml` untuk memberi tahu sistem Android bahwa aplikasi membutuhkan akses ke jaringan internet.

SOAL 2

Mengapa RecyclerView masih digunakan, padahal RecyclerView memiliki kode yang panjang dan bersifat boiler-plate, dibandingkan LazyColumn dengan kode yang lebih singkat?

A. Jawaban

RecyclerView masih banyak digunakan karena sudah terbukti stabil dan fleksibel, terutama dalam proyek besar atau aplikasi lama yang masih menggunakan sistem View XML. Meskipun LazyColumn dari Jetpack Compose punya kode yang lebih ringkas dan mudah dibaca, RecyclerView tetap unggul dalam hal kontrol detail, seperti pengelolaan animasi kompleks, dukungan untuk berbagai jenis item dalam satu daftar, serta integrasi dengan fitur-fitur canggih seperti drag-and-drop atau sticky headers. Selain itu, dalam pengujian performa, RecyclerView menunjukkan waktu startup yang lebih cepat dibandingkan LazyColumn, meskipun perbedaan ini dapat diminimalkan dengan optimisasi tambahan. Jadi, meskipun LazyColumn lebih sederhana dan cocok untuk proyek baru dengan Jetpack Compose, RecyclerView tetap menjadi pilihan yang solid untuk kebutuhan yang lebih kompleks atau proyek yang sudah ada.

MODUL 4:

VIEWMODEL DAN DEBUGGING

SOAL 1

Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a. Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity.
- b. Gunakan ViewModelFactory dalam pembuatan ViewModel
- c. Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment
- d. gunakan logging untuk event berikut:
 - a. Log saat data item masuk ke dalam list
 - b. Log saat tombol Detail dan tombol Explicit Intent ditekan
 - c. Log data dari list yang dipilih ketika berpindah ke halaman Detail
- e. Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi. Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out

A. Source Code

1. MainActivity.kt

Tabel 22 Source Code Jawaban MainActivity.kt Modul 4

```
1 package com.example.viewmodel
2
3 import android.os.Bundle
4 import androidx.appcompat.app.AppCompatActivity
5 import com.example.viewmodel.R
6
7 class MainActivity : AppCompatActivity() {
8     override fun onCreate(savedInstanceState:
Bundle?) {
9         super.onCreate(savedInstanceState)
10        setContentView(R.layout.activity_main)
11
12        val fragmentManager =
supportFragmentManager
13        val homeFragment = HomeFragment()
14        val fragment =
fragmentManager.findFragmentByTag(HomeFragment:::
class.java.simpleName)
15        if (fragment != HomeFragment) {
16            fragmentManager.beginTransaction()
17                .add(R.id.frame_container,
homeFragment,
HomeFragment:::class.java.simpleName)
18                .commit()
19        }
20    }
21 }
```

2. Drama.kt

Tabel 23 Source Code Jawaban Drama.kt Modul 4

```
1 package com.example.recyclerview
2
3 import android.os.Parcelable
4 import kotlinx.parcelize.Parcelize
5
6 @Parcelize
7 data class Drama(
8     val title: String,
9     val link: String,
10    val photo: String,
11    val plot: String,
12    val year: String,
13    val cast: String
```

14	) : Parcelable
----	----------------

3. HomeFragment.kt

Tabel 24 Source Code Jawaban HomeFragment.kt Modul 4

1	package com.example.viewmodel
2	
3	import android.content.Intent
4	import android.net.Uri
5	import android.os.Bundle
6	import android.view.LayoutInflater
7	import android.view.View
8	import android.view.ViewGroup
9	import androidx.fragment.app.Fragment
10	import androidx.fragment.app.viewModels
11	import androidx.lifecycle.LifecycleScope
12	import
	androidx.recyclerview.widget.LinearLayoutManager
13	import
	com.example.viewmodel.databinding.FragmentHomeBi
	nding
14	import kotlinx.coroutines.flow.collectLatest
15	import kotlinx.coroutines.launch
16	
17	class HomeFragment : Fragment() {
18	
19	private var _binding: FragmentHomeBinding? =
	null
20	private val binding get() = _binding!!
21	
22	private val viewModel: DramaViewModel by
	viewModels {
23	DramaViewModelFactory(requireContext().resources
	)
24	}
25	
26	override fun onCreateView(
27	inflater: LayoutInflater, container:
	ViewGroup?,
28	savedInstanceState: Bundle?
29	): View {
30	_binding =
	FragmentHomeBinding.inflate(inflater, container,
	false)
31	return binding.root
32	}
33	
34	

```

        override fun onViewCreated(view: View,
savedInstanceState: Bundle?) {
35            binding.rvDrama.layoutManager =
LinearLayoutManager(requireContext())
36            binding.rvDrama.setHasFixedSize(true)
37
38            val adapter = DramaAdapter(
39                listDrama = emptyList(),
40                onWikiClick = { link ->
41                    val uri = Uri.parse(link)
42                    startActivity(Intent(Intent.ACTION_VIEW, uri))
43                },
44                onDetailClick = { drama ->
45                    viewModel.onDramaClicked(drama)
46                }
47            )
48
49            binding.rvDrama.adapter = adapter
50
51            lifecycleScope.launch {
52                viewModel.dramaList.collectLatest {
dramas ->
53                    adapter.updateData(dramas)
54                }
55            }
56
57            lifecycleScope.launch {
58                viewModel.eventClick.collectLatest {
drama ->
59                    android.util.Log.d(
60                        "HomeFragment",
61                        "Pindah ke Detail dengan
data: ${drama.title}, ${drama.year},
${drama.cast}"
62                    )
63
64                    val detailFragment =
DetailFragment().apply {
65                        arguments = Bundle().apply {
66                            putString("EXTRA_TITLE",
drama.title)
67                            putString("EXTRA_PHOTO",
drama.photo)
68                            putString("EXTRA_PLOT",
drama.plot)
69                            putString("EXTRA_YEAR",
drama.year)
70                            putString("EXTRA_CAST",
drama.cast)

```

71	}
72	}
73	parentFragmentManager.beginTransaction()
74	.replace(R.id.frame_container, detailFragment)
75	.addToBackStack(null)
76	.commit()
77	}
78	}
79	override fun onDestroyView() {
80	super.onDestroyView()
81	_binding = null
82	}
83	}
84	

4. DetailFragment

Tabel 25 Source Code Jawaban DetailFragment.kt Modul 4

1	package com.example.viewmodel
2	
3	import android.os.Bundle
4	import androidx.fragment.app.Fragment
5	import android.view.LayoutInflater
6	import android.view.View
7	import android.view.ViewGroup
8	import com.bumptech.glide.Glide
9	import
	com.example.viewmodel.databinding.FragmentDetail
	Binding
10	
11	class DetailFragment : Fragment() {
12	
13	private var _binding: FragmentDetailBinding?
	= null
14	private val binding get() = _binding!!
15	
16	override fun onCreateView(
17	inflater: LayoutInflater, container:
	ViewGroup?,
18	savedInstanceState: Bundle?,
19	): View {
20	_binding =
	FragmentDetailBinding.inflate(inflater,
	container, false)
21	
22	val title =
	arguments?.getString("EXTRA_TITLE")
23	

24	<pre> val photo = arguments?.getString("EXTRA_PHOTO") val plot = arguments?.getString("EXTRA_PLOT") 25 val year = arguments?.getString("EXTRA_YEAR") 26 val cast = arguments?.getString("EXTRA_CAST") 27 28 android.util.Log.d("DetailFragment", 29 "Data diterima: \$title, \$year, \$cast") 30 31 binding.tvName.text = title 32 binding.tvPlot.text = plot 33 binding.tvYear.text = "Year: \$year" 34 binding.tvCast.text = "Cast: \$cast" 35 photo?.let { 36 Glide.with(requireContext()) 37 .load(it) 38 .into(binding.imgItemPhoto) 39 } binding.buttonKembali.setOnClickListener 40 { 41 parentFragmentManager.popBackStack() 42 } 43 return binding.root 44 } 45 override fun onDestroyView() { 46 super.onDestroyView() 47 _binding = null 48 } 49 } </pre>
----	--

5. DramaAdapter.kt

Tabel 26 Source Code Jawaban DramaAdapter.kt Modul 4

1	package com.example.viewmodel
2	
3	import android.view.LayoutInflater
4	import android.view.View
5	import android.view.ViewGroup
6	import android.widget.Button
7	import android.widget.ImageView
8	import android.widget.TextView
9	import androidx.recyclerview.widget.RecyclerView
10	import com.bumptech.glide.Glide
11	import com.example.viewmodel.R
12	
13	class DramaAdapter(

```

14     private var listDrama: List<Drama>,
15     private val onWikiClick: (String) -> Unit,
16     private val onDetailClick: (Drama) -> Unit
17 ) :
    RecyclerView.Adapter<DramaAdapter.ViewHolder>()
    {
18
19         class ViewHolder(itemView: View) :
    RecyclerView.ViewHolder(itemView) {
20             val imgPhoto: ImageView =
    itemView.findViewById(R.id.img_item_photo)
21             val tvTitle: TextView =
    itemView.findViewById(R.id.tv_item_name)
22             val tvPlot: TextView =
    itemView.findViewById(R.id.tv_item_plot)
                val btnWiki: Button =
23 itemView.findViewById(R.id.btn_wiki)
                val btnDetail: Button =
24 itemView.findViewById(R.id.button_detail)
                }
25
26         override fun onCreateViewHolder(parent:
    ViewGroup, viewType: Int): ViewHolder {
27             val view: View =
28 LayoutInflater.from(parent.context).inflate(R.la
29 yout.item_list, parent, false)
                return ViewHolder(view)
30         }
31
32         override fun getItemCount(): Int =
33 listDrama.size
34
35         override fun onBindViewHolder(holder:
    ViewHolder, position: Int) {
                val drama = listDrama[position]
36
37             android.util.Log.d("DramaAdapter", "Data
38 masuk ke list: ${drama.title}")
39
                holder.tvTitle.text = drama.title
40             holder.tvPlot.text = "Plot:
41 ${drama.plot}"
42
                Glide.with(holder.itemView.context)
43                     .load(drama.photo)
44                     .into(holder.imgPhoto)
45
                holder.btnWiki.setOnClickListener {
46
47

```

48	android.util.Log.d("DramaAdapter",
	"Tombol Wiki ditekan untuk: \${drama.title}")
49	onWikiClick(drama.link)
50	}
51	
52	holder.btnDetail.setOnClickListener {
53	android.util.Log.d("DramaAdapter",
	"Tombol Detail ditekan untuk: \${drama.title}")
54	onDetailClick(drama)
55	}
56	}
57	fun updateData(newList: List<Drama>) {
58	listDrama = newList
59	notifyDataSetChanged()
60	}
61	}
62	fun updateData(dramas: List<Drama>) {
63	}

6. DramaViewModel

Tabel 27 Source Code Jawaban DramaViewModel.kt Modul 4

1	package com.example.viewmodel
2	
3	import android.content.res.Resources
4	import androidx.lifecycle.ViewModel
5	import androidx.lifecycle.viewModelScope
6	import kotlinx.coroutines.flow.MutableSharedFlow
7	import kotlinx.coroutines.flow.MutableStateFlow
8	import kotlinx.coroutines.flow.SharedFlow
9	import kotlinx.coroutines.flow.StateFlow
10	import kotlinx.coroutines.launch
11	
12	class DramaViewModel(private val resources:
	Resources) : ViewModel() {
13	
14	private val _dramaList =
	MutableStateFlow<List<Drama>>(emptyList())
15	val dramaList: StateFlow<List<Drama>> get() =
	_dramaList
16	
17	private val _eventClick =
	MutableSharedFlow<Drama>()
18	val eventClick: SharedFlow<Drama> get() =
	_eventClick
19	
20	init {
21	loadDramas()
22	}
23	

```

24     private fun getDramaFromResources():
List<Drama> {
        val dataTitle =
25 resources.getStringArray(R.array.data_name)
        val dataLink =
26 resources.getStringArray(R.array.data_link)
        val dataPhoto =
27 resources.getStringArray(R.array.data_photo)
        val dataPlot =
28 resources.getStringArray(R.array.data_plot)
        val dataYear =
29 resources.getStringArray(R.array.data_year)
        val dataCast =
30 resources.getStringArray(R.array.data_cast)
31
32     return dataTitle.indices.map { i ->
33         Drama(
34             title = dataTitle[i],
35             link = dataLink[i],
36             photo = dataPhoto[i],
37             plot = dataPlot[i],
38             year = dataYear[i],
39             cast = dataCast[i]
40         )
41     }
42 }
43 private fun loadDramas() {
44     viewModelScope.launch {
45         _dramaList.value =
getDramaFromResources()
46     }
47 }
48 fun onDramaClicked(drama: Drama) {
49     viewModelScope.launch {
50         _eventClick.emit(drama)
51     }
52 }
}

```

7. DramaViewModelFactory.kt

Tabel 28 Source Code Jawaban DramaViewModelFactory.kt Modul 4

```
1 package com.example.viewmodel
2
3 import android.content.res.Resources
4 import androidx.lifecycle.ViewModel
5 import androidx.lifecycle.ViewModelProvider
6
7 class DramaViewModelFactory(private val
  resources: Resources) :
  ViewModelProvider.Factory {
8     override fun <T : ViewModel>
  create(modelClass: Class<T>): T {
9         if
  (modelClass.isAssignableFrom(DramaViewModel::cla
  ss.java)) {
10             @Suppress("UNCHECKED_CAST")
11             return DramaViewModel(resources) as
  T
12         }
13         throw IllegalArgumentException("Unknown
  ViewModel class")
14     }
15 }
```

8. Activity_main.xml

Tabel 29 Source Code Jawaban Activity_main.xml Modul 4

```
1 <androidx.constraintlayout.widget.ConstraintLayo
  ut
2   xmlns:android="http://schemas.android.com/apk/re
  s/android"
3
4   xmlns:app="http://schemas.android.com/apk/res-
  auto"
5   xmlns:tools="http://schemas.android.com/tools"
6       android:layout_width="match_parent"
7       android:layout_height="match_parent"
8   tools:context="com.example.viewmodel.MainActivit
  y">
9       <FrameLayout
10           android:id="@+id/frame_container"
11           android:layout_width="match_parent"
12           android:layout_height="match_parent" />
13
14 </androidx.constraintlayout.widget.ConstraintLay
  out>
```

9. fragment_home.xml

Tabel 30 Source Code Jawaban fragment home Modul 4

1	<androidx.constraintlayout.widget.ConstraintLayout
2	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent"
7	tools:context="com.example.viewmodel.HomeFragment">
8	
9	<TextView
10	android:id="@+id/tvHeader"
11	android:layout_width="0dp"
12	android:layout_height="wrap_content"
13	android:text="@string/header"
14	android:textSize="20sp"
15	android:textStyle="bold"
16	android:background="#872852"
17	android:textColor="@android:color/white"
18	android:gravity="center"
19	android:padding="16dp"
20	app:layout_constraintTop_toTopOf="parent"
21	app:layout_constraintStart_toStartOf="parent"
22	app:layout_constraintEnd_toEndOf="parent" />
23	
24	<androidx.recyclerview.widget.RecyclerView
25	android:id="@+id/rv_Drama"
26	android:layout_width="0dp"
27	android:layout_height="0dp"
28	android:layout_margin="15dp"
29	
30	app:layout_constraintTop_toBottomOf="@id/tvHeader"
31	app:layout_constraintBottom_toBottomOf="parent"
	app:layout_constraintStart_toStartOf="parent"
32	app:layout_constraintEnd_toEndOf="parent" />
33	</androidx.constraintlayout.widget.ConstraintLayout>

10. fragment_detail.xml

Tabel 31 Source Code Jawaban fragment_detail.xml Modul 4

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	xmlns:tools="http://schemas.android.com/tools"
6	android:layout_width="match_parent"
7	android:layout_height="match_parent"
8	android:background="@drawable/background_image"
9	tools:context="com.example.viewmodel.DetailFragment">
10	
11	<ScrollView
12	android:id="@+id/scrollView"
13	android:layout_width="0dp"
14	android:layout_height="0dp"
15	android:fillViewport="true"
16	
17	app:layout_constraintTop_toTopOf="parent"
18	app:layout_constraintBottom_toBottomOf="parent"
19	app:layout_constraintStart_toStartOf="parent"
20	app:layout_constraintEnd_toEndOf="parent">
21	
22	<androidx.constraintlayout.widget.ConstraintLayout
23	android:layout_width="match_parent"
24	android:layout_height="wrap_content"
25	android:padding="16dp">
26	
27	<ImageView
28	android:id="@+id/img_item_photo"
29	android:layout_width="0dp"
30	android:layout_height="458dp"
31	android:scaleType="centerCrop"
32	app:layout_constraintTop_toTopOf="parent"
33	app:layout_constraintStart_toStartOf="parent"
34	app:layout_constraintEnd_toEndOf="parent" />
35	
36	<TextView
37	android:id="@+id/tv_name"
38	android:layout_width="wrap_content"
39	android:layout_height="wrap_content"
	android:textStyle="bold"

```

40         android:textSize="20sp"
41         android:layout_marginTop="16dp"
42 app:layout_constraintTop_toBottomOf="@id/img_i
tem_photo"
43 app:layout_constraintStart_toStartOf="parent"
44 app:layout_constraintEnd_toEndOf="parent"
45         android:text="Drama Title" />
46
47     <TextView
48         android:id="@+id/tv_year"
49 android:layout_width="wrap_content"
50 android:layout_height="wrap_content"
51         android:text="Year: "
52         android:textSize="16sp"
53         android:layout_marginTop="8dp"
54 app:layout_constraintTop_toBottomOf="@id/tv_na
me"
55 app:layout_constraintStart_toStartOf="parent"
56 app:layout_constraintEnd_toEndOf="parent" />
57
58     <TextView
59         android:id="@+id/tv_cast"
60         android:layout_width="0dp"
61 android:layout_height="wrap_content"
62         android:text="Cast: "
63         android:textSize="16sp"
64         android:layout_marginTop="8dp"
65         android:textAlignment="center"
66 app:layout_constraintTop_toBottomOf="@id/tv_ye
ar"
67 app:layout_constraintStart_toStartOf="parent"
68 app:layout_constraintEnd_toEndOf="parent" />
69
70     <TextView
71         android:id="@+id/tv_plot"
72         android:layout_width="0dp"
73 android:layout_height="wrap_content"
74         android:layout_marginTop="8dp"
75         android:textAlignment="center"
76         android:textSize="16sp"
77         android:text="Plot: "
78 app:layout_constraintTop_toBottomOf="@id/tv_ca
st"
79 app:layout_constraintStart_toStartOf="parent"
80 app:layout_constraintEnd_toEndOf="parent" />
81 </androidx.constraintlayout.widget.ConstraintL
ayout>
82     </ScrollView>

```


83	<TextView
84	android:id="@+id/button_kembali"
85	android:layout_width="match_parent"
86	android:layout_height="wrap_content"
87	android:layout_margin="16dp"
88	android:background="#d1cfe2"
89	android:elevation="10dp"
90	android:text="Kembali"
91	android:textAllCaps="false"
92	android:textColor="@color/black"
93	android:textSize="20sp"
94	android:gravity="center"
95	android:padding="16dp"
96	app:layout_constraintBottom_toBottomOf="parent"
97	"
	app:layout_constraintStart_toStartOf="parent"
98	app:layout_constraintEnd_toEndOf="parent" />
99	</androidx.constraintlayout.widget.ConstraintL
100	ayout>
	>

11. item_list.xml

Tabel 32 Source Code Jawaban item_list.xml Modul 4

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.cardview.widget.CardView
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:layout_width="match_parent"
6	android:layout_height="wrap_content"
7	xmlns:tools="http://schemas.android.com/tools"
8	android:layout_margin="8dp"
9	app:cardCornerRadius="8dp"
10	app:cardElevation="4dp">
11	<androidx.constraintlayout.widget.ConstraintLayout
12	android:layout_width="match_parent"
13	android:layout_height="wrap_content"
14	android:padding="8dp">
15	<ImageView
16	android:id="@+id/img_item_photo"
17	android:layout_width="70dp"
18	android:layout_height="100dp"
19	android:scaleType="centerCrop"
20	app:layout_constraintStart_toStartOf="parent"

```

21 app:layout_constraintTop_toTopOf="parent"
22 app:layout_constraintBottom_toBottomOf="parent"
23
24     <TextView
25         android:id="@+id/tv_item_name"
26         android:layout_width="0dp"
27         android:layout_height="wrap_content"
28         android:textSize="16sp"
29         android:textStyle="bold"
30 app:layout_constraintStart_toEndOf="@id/img_item
31 _photo"
32 app:layout_constraintTop_toTopOf="parent"
33 app:layout_constraintEnd_toEndOf="parent"
34         android:layout_marginStart="12dp"
35         tools:text="True Beauty" />
36
37     <TextView
38         android:id="@+id/tv_item_plot"
39         android:layout_width="0dp"
40         android:layout_height="wrap_content"
41         android:layout_marginTop="4dp"
42         android:textSize="13sp"
43 app:layout_constraintTop_toBottomOf="@id/tv_item
44 _name"
45 app:layout_constraintStart_toStartOf="@id/tv_item_name"
46 app:layout_constraintEnd_toEndOf="parent"
47         tools:text="Plot: " />
48
49     <Button
50         android:id="@+id/btn_wiki"
51         android:layout_width="wrap_content"
52         android:layout_height="wrap_content"
53         android:layout_marginStart="24dp"
54         android:layout_marginTop="8dp"
55         android:backgroundTint="#ffc5d3"
56         android:text="IMDB"
57 app:layout_constraintStart_toStartOf="@id/tv_item_plot"
58 app:layout_constraintTop_toBottomOf="@id/tv_item_plot" />
59
60     <Button
61         android:id="@+id/button_detail"
62         android:layout_width="wrap_content"
63         android:layout_height="wrap_content"
64         android:layout_marginStart="8dp"
65         android:backgroundTint="#ffc5d3"

```

65	android:text="Detail"
66	app:layout_constraintStart_toEndOf="@id/btn_wiki
67	"
	app:layout_constraintTop_toTopOf="@id/btn_wiki"
68	/>
	</androidx.constraintlayout.widget.ConstraintLayout>
69	out>
	</androidx.cardview.widget.CardView>
70	

12. string.xml

Tabel 33 Source Code Jawaban string.xml Modul 4

1	<resources>
2	<string name="app_name">modul3</string>
3	<string name="btn_detail">Detail</string>
4	<string name="btn_wiki">Wiki</string>
5	
6	<string-array name="data_name">
7	<item>True Beauty</item>
8	<item>Business Proposal</item>
9	<item>King the Land</item>
10	<item>Welcome to Samdalri</item>
11	<item>Love Next Door</item>
12	<item>Family by Choice</item>
13	<item>Lovely Runner</item>
14	<item>Melo Movie</item>
15	</string-array>
16	
17	<string-array name="data_photo">
18	<item>https://upload.wikimedia.org/wikipedia/id/6/6b/True_Beauty_main_poster.jpg</item>
19	<item>https://upload.wikimedia.org/wikipedia/id/1/19/A_Business_Proposal.jpg</item>
20	<item>https://upload.wikimedia.org/wikipedia/id/d/da/King_the_Land.png</item>
21	<item>https://upload.wikimedia.org/wikipedia/id/5/54/Welcome_to_Samdal-ri.jpg</item>
22	<item>https://upload.wikimedia.org/wikipedia/id/a/a6/Love_Next_Door_poster.png</item>
23	<item>https://upload.wikimedia.org/wikipedia/id/a/a6/Family_by_Choice_poster.png</item>
24	<item>https://upload.wikimedia.org/wikipedia/id/6/67/Lovely_Runner.png</item>
25	<item>https://upload.wikimedia.org/wikipedia/id/7/7a/Melo_Movie_poster.png</item>
26	</string-array>
27	
28	

29	<string-array name="data_link">
30	<item>https://www.netflix.com/id/title/81410834</item>
31	<item>https://www.netflix.com/id-en/title/81509440</item>
32	<item>https://www.netflix.com/id/title/81671440</item>
33	<item>https://www.netflix.com/id/title/81697983</item>
34	<item>https://www.netflix.com/id-en/title/81739044</item>
35	<item>https://www.viu.com/ott/id/id/vod/2435994/Family-By-Choice</item>
36	<item>https://www.netflix.com/id-en/title/81914565</item>
37	<item>https://www.netflix.com/id-en/title/81720593</item>
38	</string-array>
39	
40	<string-array name="data_plot">
41	<item>True Beauty is a high school drama about a girl who becomes a beauty through makeup and faces challenges when her secret is revealed.</item>
42	<item>Business Proposal tells the story of a woman who participates in a business proposal pretending to be her friend, only to find out the man is her company s CEO</item>
43	<item>King the Land is about a CEO who tries to open a new hotel and a woman who works there while struggling with her own love story.</item>
44	<item>Welcome to Samdalri is about a group of individuals who meet in a small village and form bonds through humorous and touching moments.</item>
45	<item>Love Next Door is a romantic drama revolving around a couple living next door and their developing relationship.</item>
46	<item>Family by Choice is a heartwarming drama about a group of people who form a family despite not being related by blood.</item>
47	<item>Lovely Runner follows a runner who is trying to make it in the world of competitive sports while balancing her personal life.</item>
	<item>Melo Movie is a romantic comedy about a woman trying to find love while

48	struggling with her career in the film
49	industry.</item>
50	</string-array>
51	
52	<string-array name="data_year">
53	<item>2020</item>
54	<item>2022</item>
55	<item>2023</item>
56	<item>2023</item>
57	<item>2024</item>
58	<item>2024</item>
59	<item>2024</item>
60	<item>2025</item>
61	</string-array>
62	
	<string-array name="data_cast">
63	<item>Moon Ga-young, Cha Eun-woo, Hwang
	In-yeop</item>
64	<item>Kim Se-jeong, Ahn Hyo-seop, Kim
65	Min Kyu, Seol In Ah</item>
66	<item>YoonA, Lee Jun-ho</item>
	<item>Ji Chang-wook, Shin Hye-sun</item>
67	<item>Jung Hae In, Jung So Min, Kim Ji
	Eun, Yun Ji On</item>
68	<item>Hwang In Yeop, Jung Chae Yeon, Bae
	Hyun Sung</item>
69	<item>Byeon Woo-seok, Kim Hye-
	yeon</item>
70	<item>Choi Woo Shik, Park Bo Young, Lee
71	Jun Young, Jeon So Nee</item>
72	</string-array>
73	
74	
75	<string name="hello_blank_fragment">Hello
	blank fragment</string>
	</resources>

13. build.gradle.kts (module:app)

Tabel 34 Source Code Jawaban build.gradle.kts Modul 4

1	plugins {
2	alias(libs.plugins.android.application)
3	alias(libs.plugins.kotlin.android)
4	id("kotlin-parcelize")
5	id("org.jetbrains.kotlin.kapt")
6	}
7	

```

8  android {
9      namespace = "com.example.viewmodel"
10     compileSdk = 35
11
12     defaultConfig {
13         applicationId = "com.example.viewmodel"
14         minSdk = 30
15         targetSdk = 35
16         versionCode = 1
17         versionName = "1.0"
18
19         testInstrumentationRunner =
20         "androidx.test.runner.AndroidJUnitRunner"
21     }
22
23     buildTypes {
24         release {
25             isMinifyEnabled = false
26             proguardFiles(
27                 getDefaultProguardFile("proguard-android-
28                 optimize.txt"),
29                 "proguard-rules.pro"
30             )
31         }
32     }
33     compileOptions {
34         sourceCompatibility =
35         JavaVersion.VERSION_11
36         targetCompatibility =
37         JavaVersion.VERSION_11
38     }
39     kotlinOptions {
40         jvmTarget = "11"
41     }
42     buildFeatures {
43         viewBinding = true
44     }
45 }
46
47 dependencies {
48     implementation(libs.androidx.core.ktx)
49     implementation(libs.androidx.appcompat)
50     implementation(libs.material)
51     implementation(libs.androidx.activity)
52     implementation("androidx.recyclerview:recyclerview:1.3.1")

```

	implementation
51	("androidx.constraintlayout:constraintlayout:2.1.4")
52	testImplementation(libs.junit)
53	androidTestImplementation(libs.androidx.junit)
54	androidTestImplementation(libs.androidx.espresso
	.core)
55	implementation("com.github.bumptech.glide:glide:4.15.1")
56	kapt("com.github.bumptech.glide:compiler:4.15.1")
57	implementation("androidx.lifecycle:lifecycle-viewmodel-ktx:2.7.0")
58	implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core:1.7.3")
59	implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.7.3")
60	implementation("androidx.fragment:fragment-ktx:1.6.0")
	}

14. AndroidManifest.xml

Tabel 35 Source Code Jawaban AndroidManifest.xml Modul 4

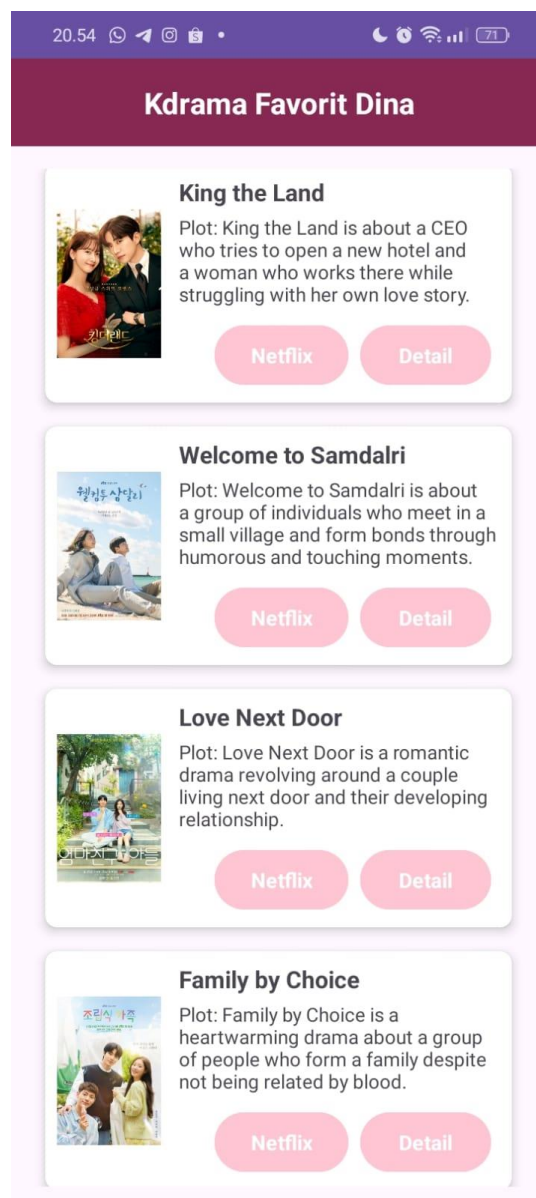
1	<?xml version="1.0" encoding="utf-8"?>
2	<manifest
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:tools="http://schemas.android.com/tools">
4	
5	<uses-permission
6	android:name="android.permission.INTERNET"/>
	<application
7	android:allowBackup="true"
8	android:dataExtractionRules="@xml/data_extraction_rules"
9	android:fullBackupContent="@xml/backup_rules"
10	android:icon="@mipmap/ic_launcher"
11	android:label="@string/app_name"
12	android:roundIcon="@mipmap/ic_launcher_round"
13	android:supportRtl="true"
14	android:theme="@style/Theme.ViewModel"
15	tools:targetApi="31">
16	<activity
17	android:name="com.example.viewmodel.MainActivity"
18	android:exported="true">
19	<intent-filter>
20	

```

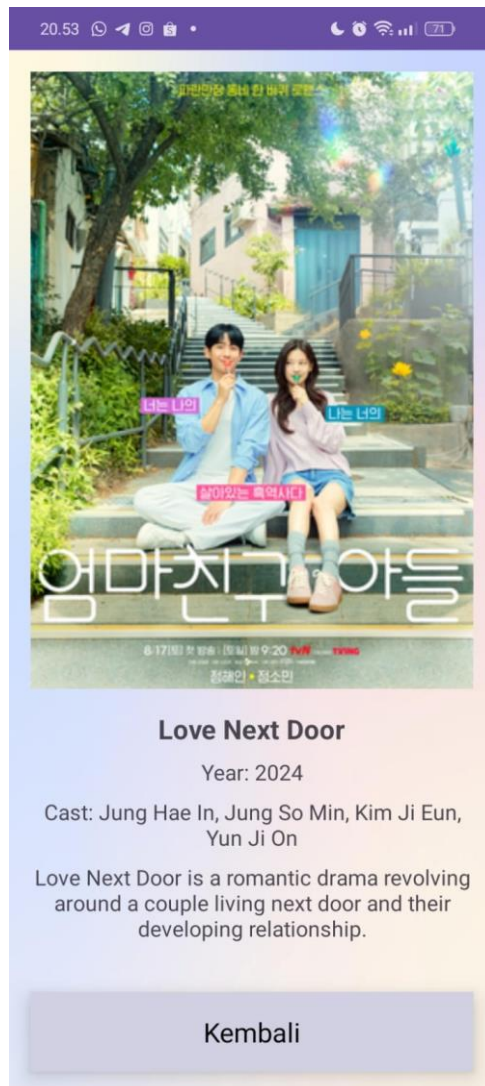
21         <action
22         android:name="android.intent.action.MAIN" />
23         <category
24         android:name="android.intent.category.LAUNCHER"
25         />
26     </intent-filter>
27 </activity>
28 </application>
29 </manifest>

```

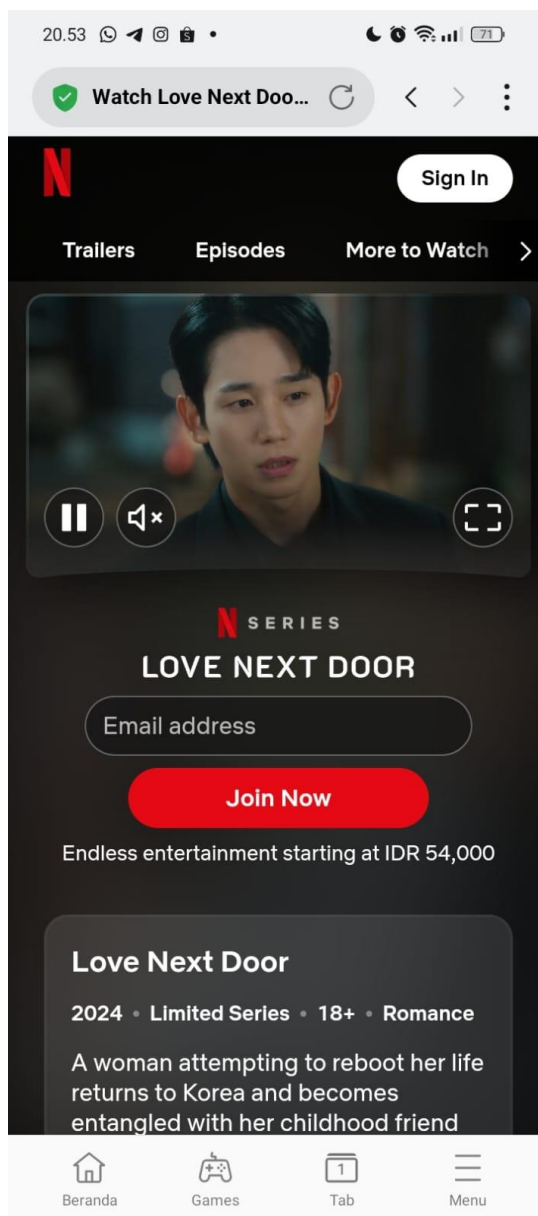
B. Output Program



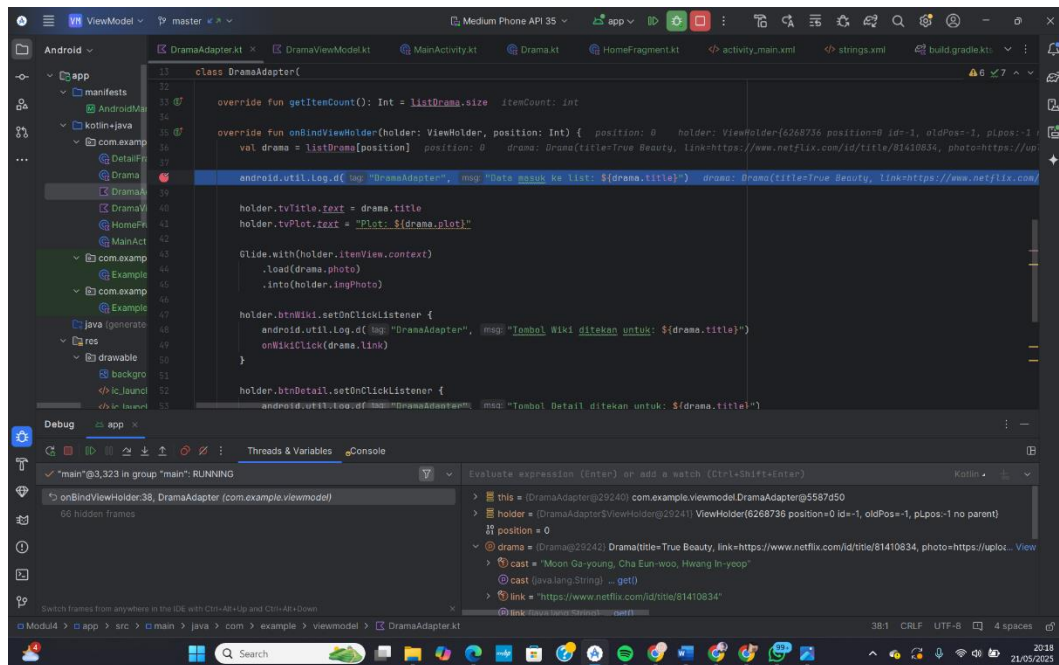
Gambar 16 Screenshot Hasil Tampilan Halaman Home



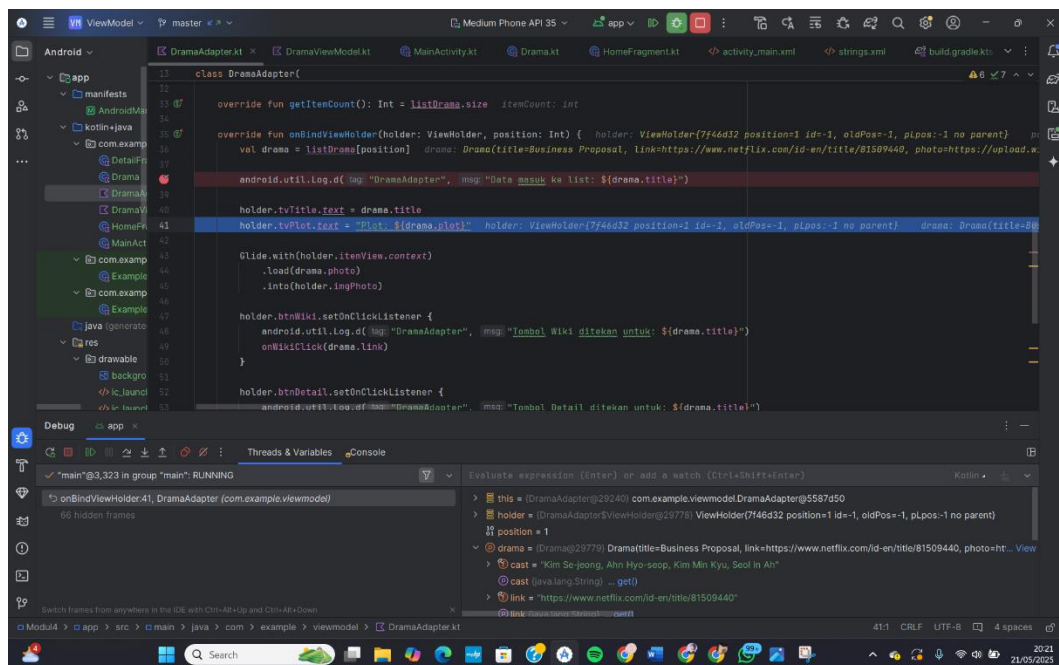
Gambar 17 Screenshot Hasil Tampilan Halaman Detail



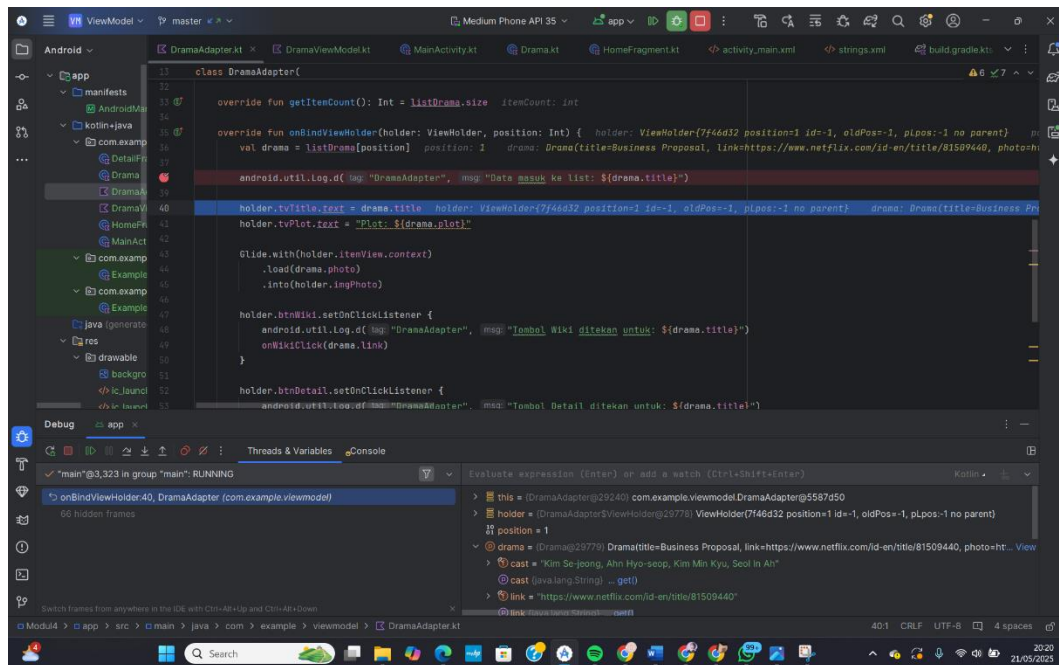
Gambar 18 Screenshot Hasil Tampilan Link Netflix



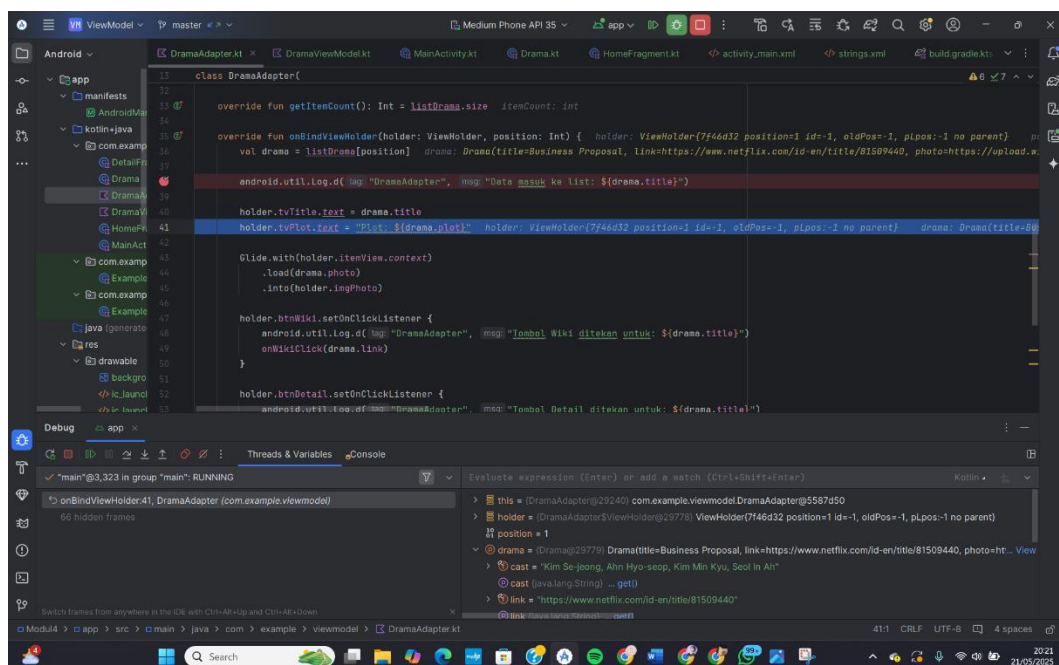
Gambar 19 Penggunaan Debugger



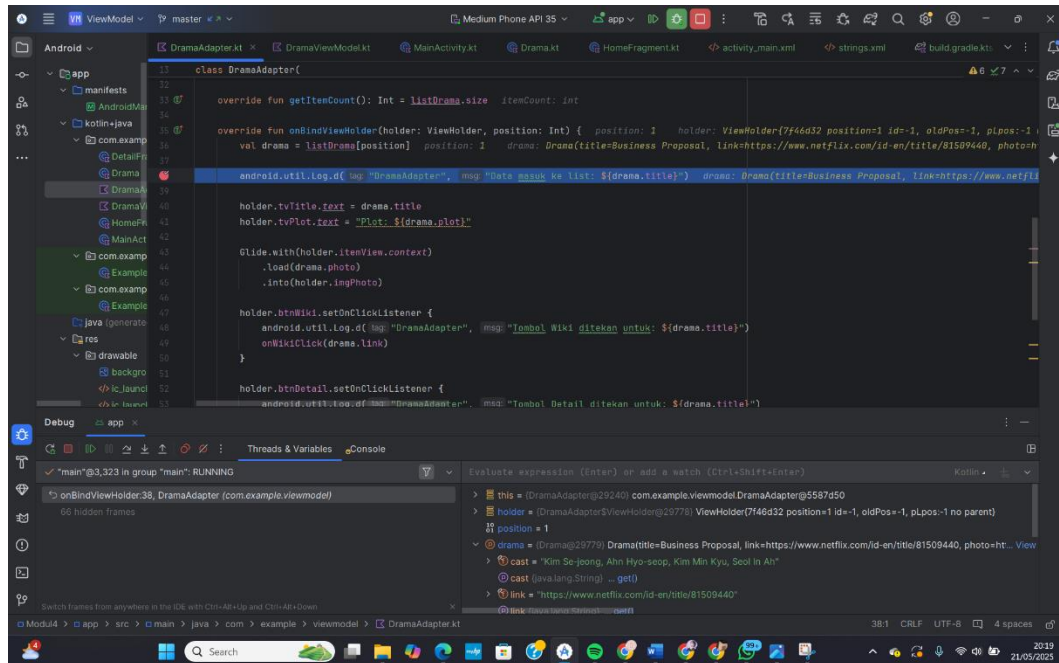
Gambar 20 Penggunaan Fitur f7 (Step Into) pada Debug



Gambar 21 Penggunaan Fitur f8 (Step Over) pada Debug



Gambar 22 Penggunaan Fitur shift+f8 (Step Out) pada Debug



Gambar 23 Penggunaan Fitur f9 (Resume Program) pada Debug

C. Pembahasan

1. MainActivity.kt

Pada baris 1 package `com.example.viewmodel` menunjukkan bahwa file Kotlin ini berada di dalam paket bernama `viewmodel` di bawah domain `com.example`. Baris 3–5 merupakan impor agar file ini bisa menggunakan `AppCompatActivity` dan resource dari `R`. Baris 7 mendefinisikan kelas `MainActivity` yang merupakan activity utama aplikasi ini. Baris 8 memanggil fungsi `onCreate()` yang akan dijalankan saat activity dibuat. Baris 10 menjalankan `setContentView(R.layout.activity_main)` untuk menampilkan layout `activity_main` sebagai antarmuka pengguna. Baris 12 membuat objek `FragmentManager` untuk menangani manajemen fragmen di activity ini. Baris 13 membuat instance dari `HomeFragment`. Baris 14–15 memeriksa apakah fragmen dengan tag `HomeFragment` sudah ada sebelumnya, dan jika belum maka fragmen baru akan ditambahkan ke dalam `frame_container` menggunakan `beginTransaction()` lalu disimpan dengan `commit()`.

2. Drama.kt

Pada bagian awal, kode `package com.example.recyclerview` digunakan untuk mengelompokkan file ini ke dalam paket `recyclerview` agar lebih terorganisir dalam struktur proyek. Baris `import android.os.Parcelable` dan `import kotlinx.parcelize.Parcelize` memungkinkan kelas `Drama` dapat dikirim antar komponen Android dengan cara efisien menggunakan `parcel`. Anotasi `@Parcelize` digunakan supaya proses implementasi `Parcelable` tidak perlu dilakukan secara manual, cukup dengan menandai data class tersebut. Kelas `Drama` bertipe data class yang menyimpan informasi seperti `title`, `link`, `photo`, `plot`, `year`, dan `cast` yang masing-masing bertipe `String`. Kelas ini mengimplementasikan `Parcelable`, sehingga objeknya dapat dipaketkan dan ditransfer antar `Activity` atau `Fragment`.

3. HomeFragment.kt

Pada baris 1 `package com.example.viewmodel` menunjukkan bahwa file Kotlin ini berada dalam paket bernama `viewmodel` di bawah domain `com.example`. Baris 3–15 adalah bagian `import` yang memungkinkan kelas ini menggunakan berbagai komponen seperti `Intent`, `Uri`, `Fragment`, `ViewModel`, `RecyclerView`, dan `coroutine` dari `lifecycle`. Baris 17 `class HomeFragment : Fragment()` mendefinisikan bahwa `HomeFragment` merupakan subclass dari `Fragment`. Baris 19–20 mendeklarasikan variabel binding dengan `FragmentHomeBinding` untuk mengakses elemen UI dari layout `fragment_home` tanpa perlu menggunakan `findViewById` secara manual. Baris 22–24 mendeklarasikan `viewModel` menggunakan `delegate by viewModels` dan memanfaatkan `DramaViewModelFactory` yang mengambil resource dari `context`. Baris 26–32 mengimplementasikan fungsi `onCreateView` yang meng-inflate layout XML menggunakan binding dan mengembalikan root view sebagai tampilan `fragment`. Di dalam `onViewCreated`, `RecyclerView` disiapkan dengan `LinearLayoutManager` agar daftar drama ditampilkan secara vertikal, dan `setHasFixedSize(true)` digunakan

untuk meningkatkan performa. Sebuah instance `DramaAdapter` dibuat dengan parameter kosong untuk `listDrama`, dan dua aksi klik: `onWikiClick` membuka link dengan `Intent`, sementara `onDetailClick` memicu event dari `ViewModel` saat item diklik. Adapter ini kemudian diset ke `RecyclerView`. `Coroutine` pertama di dalam `lifecycleScope.launch` berfungsi untuk mengamati aliran data `dramaList` dari `ViewModel` dan memperbarui isi adapter setiap kali data berubah. `Coroutine` kedua mengamati `eventClick`, yaitu event klik item, dan jika ada data yang diterima, akan dibuat instance `DetailFragment` dengan mengisi `Bundle` berisi informasi drama yang diklik. Fragment tersebut kemudian ditampilkan menggunakan transaksi fragment dan ditambahkan ke back stack. Terakhir, fungsi `onDestroyView` digunakan untuk menghapus nilai binding menjadi null agar mencegah memory leak setelah view dihancurkan.

4. `DetailFragment.kt`

Pada baris 1 `package com.example.viewmodel` menunjukkan bahwa file Kotlin ini berada dalam paket bernama `viewModel` di bawah domain `com.example`. Baris 3–9 adalah bagian import yang memungkinkan penggunaan berbagai komponen Android seperti `Fragment`, `Bundle`, `View`, `ViewGroup`, serta library eksternal `Glide` untuk pemrosesan gambar. Baris 11 `class DetailFragment : Fragment()` mendefinisikan bahwa `DetailFragment` adalah turunan dari `Fragment`. Baris 13–14 mendeklarasikan binding menggunakan `FragmentDetailBinding` agar bisa mengakses elemen UI dari layout `fragment_detail` secara langsung. Baris 16 sampai akhir di dalam fungsi `onCreateView`, layout di-inflate menggunakan binding lalu mengambil data dari `arguments` dengan key seperti `EXTRA_TITLE`, `EXTRA_PHOTO`, `EXTRA_PLOT`, `EXTRA_YEAR`, dan `EXTRA_CAST`, kemudian diset ke `TextView` dan `ImageView` yang sesuai. `Glide` digunakan untuk memuat gambar dari URL ke dalam `imgItemPhoto` secara efisien. Tombol kembali disiapkan untuk memanggil `popBackStack` yang akan menutup fragment ini dan kembali ke sebelumnya. Fungsi ini mengembalikan tampilan dari root binding. Terakhir, fungsi

`onDestroyView` digunakan untuk menghapus binding agar tidak terjadi memory leak saat fragment dihancurkan.

5. DramaAdapter

Pada baris 1 `package com.example.viewmodel` menunjukkan bahwa file Kotlin ini berada dalam paket bernama `viewModel` di bawah domain `com.example`. Baris 3–10 adalah bagian import yang memungkinkan penggunaan komponen Android seperti `View`, `RecyclerView`, `TextView`, `ImageView`, `Button`, dan library `Glide` untuk memuat gambar dari URL. Baris 12 `class DramaAdapter` mendefinisikan adapter untuk `RecyclerView` yang menerima data berupa `List<Drama>` serta dua lambda, yaitu `onWikiClick` dan `onDetailClick`, yang masing-masing akan dijalankan ketika tombol Wiki dan tombol Detail ditekan. Baris 19–25 mendefinisikan `ViewHolder` sebagai class dalam yang bertanggung jawab menginisialisasi dan mereferensikan elemen UI dalam item, seperti gambar, teks judul, teks plot, dan dua tombol dari file layout `item_list.xml`. Baris 27 `onCreateViewHolder` bertugas untuk meng-inflate layout `item_list` dan mengembalikannya sebagai objek `ViewHolder`. Baris 33 `getItemCount` akan mengembalikan jumlah data yang ada di dalam `listDrama` untuk menentukan berapa banyak item yang akan ditampilkan. Baris 35 `onBindViewHolder` menghubungkan data drama berdasarkan posisi ke komponen UI dalam item, memuat gambar menggunakan `Glide`, menampilkan teks judul dan plot, serta memberikan aksi ketika tombol Wiki ditekan (memanggil `onWikiClick`) dan tombol Detail ditekan (memanggil `onDetailClick`). Baris 57–61 adalah fungsi `updateData` yang digunakan untuk memperbarui isi daftar drama dengan data baru dan memanggil `notifyDataSetChanged()` agar tampilan `RecyclerView` ikut diperbarui. Baris 62–63 adalah deklarasi fungsi `updateData` kosong yang tidak memiliki isi dan bisa dihapus karena tidak digunakan.

6. DramaViewModel

Pada baris 1 `package com.example.viewmodel` menunjukkan bahwa file Kotlin ini berada dalam paket bernama `viewModel` di bawah domain `com.example`. Baris 3–9 adalah bagian `import` yang diperlukan untuk menggunakan kelas `ViewModel`, `coroutine`, serta `StateFlow` dan `SharedFlow` dari Kotlin. Baris 11 `class DramaViewModel` mendefinisikan kelas `ViewModel` yang bertugas menyediakan dan mengelola data list drama, dengan menerima parameter `Resources` untuk mengambil data dari file arrays di `resources`. Baris 14–15 mendeklarasikan `_dramaList` sebagai `MutableStateFlow` yang berisi list dari objek `Drama`, lalu membungkusnya menjadi `StateFlow` agar hanya dapat diamati dari luar tanpa bisa dimodifikasi langsung. Baris 17–18 mendeklarasikan `_eventClick` sebagai `MutableSharedFlow` untuk menangani event klik item yang akan dibagikan ke observer, lalu dibungkus sebagai `SharedFlow`. Baris 20–22 memanggil fungsi `loadDramas()` saat `ViewModel` pertama kali dibuat untuk memuat data dari `resources`. Fungsi `getDramaFromResources()` pada baris 23–41 mengambil data dari array string seperti judul, link, foto, plot, tahun, dan pemain dari `resources`, lalu menggabungkannya menjadi list berisi objek `Drama`. Baris 42–46 merupakan implementasi fungsi `loadDramas()` yang menjalankan `coroutine` menggunakan `viewModelScope` untuk mengisi nilai `_dramaList` dengan hasil dari `getDramaFromResources()`. Terakhir, baris 47–52 adalah fungsi `onDramaClicked()` yang juga dijalankan dalam `coroutine` untuk mengirimkan objek `Drama` yang diklik melalui `_eventClick` agar bisa diproses di `fragment`.

7. DramaViewModelFactory.kt

Pada baris 1 `package com.example.viewmodel` menunjukkan bahwa file Kotlin ini berada dalam paket bernama `viewModel` di bawah domain `com.example`. Baris 3–5 adalah bagian `import` yang digunakan untuk mengakses `Resources`, `ViewModel`, dan `ViewModelProvider` dari `Android`. Baris 7 `class`

`DramaViewModelFactory` mendefinisikan sebuah kelas factory yang mewarisi `ViewModelProvider.Factory`, dan bertugas untuk membuat instance dari `DramaViewModel` dengan membawa parameter `resources`. Baris 8–12 mengimplementasikan fungsi `create` yang akan dipanggil saat `ViewModel` diminta, lalu memeriksa apakah class yang diminta adalah `DramaViewModel`. Jika benar, maka akan dikembalikan instance `DramaViewModel` dengan cast ke tipe generic `T`, sedangkan jika bukan, maka akan dilempar `IllegalArgumentException` untuk menangani kasus class yang tidak dikenali.

8. Activity_Main.xml

Baris 1
`<androidx.constraintlayout.widget.ConstraintLayout`
`>` adalah layout utama yang digunakan sebagai wadah seluruh elemen UI di halaman ini. Baris 2-4 adalah deklarasi namespace XML yang memungkinkan penggunaan atribut `Android`, `app` (untuk `ConstraintLayout`), dan `tools`. Baris 5-7 mendefinisikan ukuran layout agar memenuhi seluruh layar serta menetapkan konteks `tools` untuk keperluan preview di `Android Studio`. Baris 9-12 `<FrameLayout>` dengan id `frame_container` digunakan sebagai kontainer fragmen yang akan diganti-ganti secara dinamis, misalnya saat navigasi antar halaman. Terakhir, baris 14 menutup tag `ConstraintLayout` sebagai akhir dari struktur layout ini.

9. fragment_home.xml

Baris 1
`<androidx.constraintlayout.widget.ConstraintLayout`
`>` digunakan sebagai layout utama yang memungkinkan penempatan elemen UI secara fleksibel dengan `constraint`. Baris 2-4 adalah deklarasi namespace XML standar agar atribut seperti `android:`, `app:`, dan `tools:` dapat digunakan dengan benar. Baris 5-7 mengatur layout agar mengisi seluruh layar dan menyetel konteks preview di `Android Studio` ke `HomeFragment`. Baris 9-22 `<TextView>` dengan id `tvHeader`

menampilkan teks dari resource `@string/header` sebagai judul halaman dengan ukuran 20sp, teks putih tebal, latar belakang ungu gelap, dan posisi berada di bagian atas layout serta disejajarkan secara horizontal di tengah. Baris 24-32 `<androidx.recyclerview.widget.RecyclerView>` dengan id `rv_Drama` digunakan untuk menampilkan daftar drama dalam bentuk list yang bisa discroll secara vertikal, ditempatkan tepat di bawah `tvHeader` dengan margin di sekelilingnya, dan memenuhi sisa ruang layar. Terakhir, baris 33 menutup tag `ConstraintLayout` sebagai akhir dari struktur layout ini.

10. fragment_detail.xml

Baris 1 `<androidx.constraintlayout.widget.ConstraintLayout>` digunakan sebagai layout utama yang memungkinkan penempatan elemen UI secara fleksibel dengan constraint. Baris 2–5 adalah deklarasi namespace XML standar agar atribut seperti `android:`, `app:`, dan `tools:` dapat digunakan dengan benar dalam layout. Baris 6–9 mengatur ukuran layout agar memenuhi layar penuh, memberikan latar belakang gambar dari `drawable`, serta mengatur konteks pratinjau layout ke `DetailFragment`. Baris 10–19 adalah elemen `ScrollView` dengan `fillViewport=true` agar konten dapat discroll jika melebihi tinggi layar, dan dikunci dengan constraint ke seluruh sisi parent. Baris 21–81 adalah nested `ConstraintLayout` di dalam `ScrollView` untuk menampung konten vertikal seperti gambar dan teks dengan pengaturan padding 16dp. Baris 26–33 `<ImageView>` dengan id `img_item_photo` digunakan untuk menampilkan gambar poster drama dengan ukuran tetap 458dp dan skala `centerCrop`, serta diletakkan di bagian atas layout. Baris 35–45 `<TextView>` dengan id `tv_name` menampilkan judul drama dengan ukuran teks 20sp dan gaya tebal, diletakkan di bawah gambar dan diposisikan di tengah. Baris 47–56 `<TextView>` dengan id `tv_year` menampilkan informasi tahun rilis drama dengan ukuran 16sp, margin atas

8dp, dan posisi di bawah judul. Baris 58–68 `<TextView>` dengan id `tv_cast` berisi daftar pemain drama, dengan lebar penuh, alignment tengah, dan margin atas 8dp, diposisikan di bawah tahun. Baris 70–680 `<TextView>` dengan id `tv_plot` menampilkan sinopsis atau alur cerita dengan alignment tengah dan ukuran teks 16sp, diletakkan di bawah informasi cast. Baris 84-99 `<TextView>` dengan id `button_kembali` berfungsi sebagai tombol kembali dengan latar warna ungu muda, teks rata tengah, ukuran 20sp, dan dikunci ke bagian bawah layout agar selalu tampil tetap meskipun konten di-scroll.

11. item_list.xml

Baris 1 `<androidx.cardview.widget.CardView>` digunakan sebagai wadah utama dengan sudut membulat dan efek elevasi agar tampilan setiap item terlihat seperti kartu. Baris 3-5 adalah deklarasi namespace XML standar yang memungkinkan penggunaan atribut seperti `android:`, `app:`, dan `tools:`. Baris 5-7 mengatur lebar kartu agar memenuhi parent, tinggi menyesuaikan isi, dengan margin 8dp serta pengaturan radius sudut 8dp dan elevasi 4dp agar tampak sedikit melayang. Baris 12-15

`<androidx.constraintlayout.widget.ConstraintLayout>` digunakan sebagai layout dalam kartu untuk menyusun elemen-elemen UI secara terstruktur. Baris 17-24 `<ImageView>` dengan id `img_item_photo` digunakan untuk menampilkan gambar poster drama berukuran 70x100dp dengan skala `centerCrop`, diposisikan di sisi kiri dan vertikal tengah dalam kartu. Baris 26-36 `<TextView>` dengan id `tv_item_name` menampilkan judul drama dengan teks tebal berukuran 16sp, diletakkan di samping gambar dengan margin awal 12dp dan disejajarkan ke kanan. Baris 38-47 `<TextView>` dengan id `tv_item_plot` menampilkan deskripsi singkat atau plot drama di bawah judul, dengan ukuran teks lebih kecil 13sp dan sedikit margin atas. Baris 49-58 `<Button>` dengan id `btn_wiki` berfungsi untuk membuka tautan Netflix, diberi teks “Netflix”, latar belakang berwarna pink muda, dan

diletakkan di bawah plot dengan margin awal 24dp. Baris 60-68 `<Button>` dengan id `button_detail` digunakan untuk menampilkan detail drama, posisinya tepat di samping kanan tombol Netflix dan tetap sejajar secara vertikal. Terakhir, baris 71 menutup tag `ConstraintLayout` dan baris 72 menutup tag `CardView` sebagai akhir dari struktur layout ini

12. string.xml

Baris 1 `<resources>` merupakan elemen root pada file XML ini yang berisi semua data string dan array yang digunakan dalam aplikasi. Baris 2-4 mendefinisikan string statis seperti nama aplikasi dan label pada tombol untuk detail serta wiki. Baris 6-15 `<string-array name="data_name">` menyimpan daftar judul drama Korea yang akan ditampilkan dalam `RecyclerView`. Baris 17-26 `<string-array name="data_photo">` menyimpan daftar URL gambar dari internet yang digunakan sebagai foto untuk setiap judul drama. Karena aplikasi menggunakan library `Glide` untuk memuat gambar. Baris 28-37 `<string-array name="data_link">` berisi link menuju halaman resmi masing-masing drama, kebanyakan dari Netflix dan satu dari Viu. Baris 39-48 `<string-array name="data_plot">` berisi deskripsi singkat atau sinopsis dari setiap drama, menjelaskan tema dan alur utama cerita. Baris 50-59 `<string-array name="data_year">` menyimpan tahun rilis masing-masing drama agar bisa ditampilkan pada tampilan detail. Baris 61-70 `<string-array name="data_cast">` memuat daftar pemeran utama dari masing-masing drama. Baris 73 dan 74 menambahkan string statis untuk keperluan default fragment dan judul header halaman. Terakhir, baris 75 menutup tag `<resources>` sebagai akhir dari seluruh deklarasi string dan array yang ada di file ini.

13. Build.gradle.kts

Pada gradle ini saya menambahkan baris 3 yaitu `id("kotlin-parcelize")` digunakan untuk mengaktifkan plugin Kotlin Parcelize yang memungkinkan penggunaan anotasi `@Parcelize` pada kelas data agar objeknya bisa dengan mudah dikirim antar komponen Android seperti Activity atau Fragment melalui Intent atau Bundle. Baris 4 Plugin `id("org.jetbrains.kotlin.kapt")` diaktifkan agar proyek dapat menggunakan KAPT (Kotlin Annotation Processing Tool), yang diperlukan ketika menggunakan library yang mengandalkan annotation processor Glide. Lalu pada `dependencies` menambahkan `implementation("androidx.recyclerview:recyclerview:1.3.1")` yang digunakan untuk menyertakan library RecyclerView versi 1.3.1 untuk menampilkan daftar item secara efisien dan fleksibel dalam bentuk list atau grid yang dapat discroll. Dan `implementation("androidx.constraintlayout:constraintlayout:2.1.4")` digunakan untuk memasukkan library ConstraintLayout versi 2.1.4 yang memberikan kontrol layout yang lebih fleksibel dan efisien dengan sistem constraint antar elemen UI. Baris 54 `implementation("com.github.bumptech.glide:glide:4.15.1")` berfungsi untuk mengimpor library utama Glide versi 4.15.1, yang memungkinkan aplikasi Android memuat gambar dari berbagai sumber, termasuk URL di internet. baris 55 `kapt("com.github.bumptech.glide:compiler:4.15.1")` digunakan untuk mengaktifkan dukungan **annotation processor** yang disediakan oleh Glide. Baris 56 `implementation("androidx.lifecycle:lifecycle-viewmodel-ktx:2.7.0")` digunakan untuk menyertakan library Lifecycle ViewModel versi KTX, yang mendukung penggunaan arsitektur MVVM pada Android. Dengan ekstensi `ktx`, ViewModel dapat digunakan secara lebih idiomatik dengan Kotlin, termasuk dukungan terhadap coroutine melalui `viewModelScope` agar tugas asinkron dapat dijalankan dengan aman dan efisien tanpa menyebabkan memory leak. Baris 57

implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core:1.7.3") digunakan untuk menambahkan library Kotlin Coroutines Core versi 1.7.3, yang merupakan pustaka inti untuk pemrograman asynchronous dan concurrency di Kotlin. Dengan ini, kita dapat menggunakan fungsi suspend, launch, async, dan coroutine flow untuk menjalankan operasi non-blocking secara efisien. Baris 58 implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.7.3") digunakan untuk menambahkan dukungan coroutine pada platform Android, yang memungkinkan eksekusi coroutine pada Dispatchers.Main atau thread utama. Library ini mempermudah pengelolaan tugas latar belakang yang perlu memperbarui UI setelah selesai. Baris 59 implementation("androidx.fragment:fragment-ktx:1.6.0") digunakan untuk menyertakan library ekstensi Kotlin (KTX) pada Fragment, yang menyediakan API berbasis Kotlin agar lebih ringkas dan efisien. Dengan ini, kita bisa menggunakan fitur seperti findNavController(), lifecycleScope, dan setFragmentResultListener() secara langsung dan lebih idiomatik.

14. AndroidManifest.xml

Pada file ini, saya menambahkan baris <uses-permission android:name="android.permission.INTERNET"/> yang merupakan deklarasi izin (permission) untuk memberi tahu sistem Android bahwa aplikasi membutuhkan akses ke jaringan internet. Izin ini sangat penting terutama jika aplikasi memuat konten dari sumber eksternal seperti gambar dari URL menggunakan library Glide. Selain itu, saya juga menyertakan deklarasi <activity android:name="com.example.viewmodel.MainActivity"> yang dilengkapi dengan <intent-filter> untuk menetapkan MainActivity sebagai titik masuk utama aplikasi. Baris <action android:name="android.intent.action.MAIN" />

menandakan bahwa aktivitas ini akan dijalankan pertama kali saat aplikasi dibuka, `android:launchMode="singleTop"` sedangkan `<category android:name="android.intent.category.LAUNCHER" />` memastikan bahwa aktivitas ini akan muncul di daftar aplikasi (launcher) pada perangkat Android, sehingga pengguna dapat meluncurkan aplikasi melalui ikon di layar utama. Atribut tambahan seperti `android:theme`, `android:icon`, dan `android:label` digunakan untuk menentukan tampilan dan nama aplikasi di perangkat. Sementara itu, `android:allowBackup`, `android:fullBackupContent`, dan `android:dataExtractionRules` mengatur perilaku pencadangan data aplikasi, dan `android:supportsRtl="true"` memungkinkan aplikasi mendukung tata letak kanan ke kiri (RTL) untuk bahasa tertentu.

15. Pembahasan Debugging

Debugger adalah alat penting dalam pengembangan aplikasi Android yang memungkinkan pemeriksaan eksekusi kode secara mendetail saat aplikasi berjalan. Fungsi utamanya adalah membantu menemukan dan memperbaiki bug atau kesalahan dalam kode dengan cara menghentikan sementara jalannya program pada titik tertentu (breakpoint), memeriksa nilai variabel, dan melacak alur eksekusi kode secara real-time. Dengan debugger, pengembang dapat memahami bagaimana aplikasi bekerja secara internal dan mengidentifikasi penyebab masalah dengan lebih cepat dan akurat.

Cara Menggunakan Debugger

i. Pasang Breakpoint:

Tentukan titik di kode (breakpoint) tempat yang ingin program berhenti sementara. Caranya, klik di sebelah kiri baris kode yang ingin dicek.

ii. Jalankan Program dalam Mode Debug:

Tekan tombol Debug 'app' di Android Studio. Program akan berjalan seperti biasa sampai menemukan breakpoint, lalu berhenti di sana.

iii. Periksa Nilai Variabel dan Alur Eksekusi:

Setelah program berhenti, lihat nilai variabel, array, atau objek di panel Variables. Juga bisa lihat urutan pemanggilan fungsi di panel Frames.

iv. Menggunakan Fitur Step (Step Into, Step Over, Step Out):

Untuk menelusuri kode lebih detail, gunakan fitur step yang disediakan oleh debugger

Fungsi Step Into, Step Over, Step Out, dan Resume Program di Debug

a. Step Into

Fitur ini digunakan untuk masuk ke dalam detail fungsi atau metode yang sedang dipanggil pada baris kode saat ini. Jika pada baris tersebut terdapat pemanggilan fungsi, Step Into akan membawa proses debug masuk ke dalam fungsi tersebut dan menjalankan kode di dalamnya secara langkah demi langkah. Fitur ini sangat berguna untuk memahami secara mendalam apa yang terjadi di dalam fungsi tersebut, bukan hanya melihat hasil akhirnya saja.

b. Step Over

Step Over menjalankan baris kode saat ini secara utuh, termasuk pemanggilan fungsi, tanpa masuk ke dalam fungsi tersebut. Setelah baris tersebut selesai dieksekusi, proses debug akan berhenti di baris berikutnya. Fitur ini cocok digunakan ketika ingin mengetahui hasil dari fungsi tanpa perlu menelusuri langkah-langkah di dalamnya secara detail.

c. Step Out

Step Out digunakan ketika proses debug sudah berada di dalam sebuah fungsi, namun ingin keluar dan kembali ke baris kode setelah pemanggilan fungsi tersebut di bagian pemanggil. Debugger akan menjalankan sisa kode dalam fungsi tersebut sampai selesai, kemudian berhenti di baris setelah pemanggilan fungsi. Fitur ini membantu mempercepat proses debugging dengan keluar dari fungsi yang sudah cukup diperiksa.

d. Resume Program

Resume Program berfungsi untuk melanjutkan eksekusi program secara normal hingga menemukan breakpoint berikutnya atau sampai program

selesai berjalan. Fitur ini memungkinkan proses debug berjalan tanpa berhenti di setiap baris kode, sehingga debugging dapat dilanjutkan dengan lebih cepat setelah pemeriksaan tertentu selesai.

Referensi: <https://developer.android.com/codelabs/basic-android-kotlin-compose-intro-debugger?hl=id#0>
https://google-developer-training.github.io/android-developer-fundamentals-course-concepts/idn/Unit%201/31_c_the_android_studio_debugger.html

SOAL 2

Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya

A. Jawaban

Application class di Android adalah kelas dasar yang menjadi pondasi utama dari sebuah aplikasi Android. Berfungsi sebagai titik awal (entry point) aplikasi dan bertanggung jawab untuk mengelola status global aplikasi, serta menjalankan inisialisasi penting sebelum komponen lain seperti Activity, Service, atau BroadcastReceiver dibuat. Application class dibuat secara otomatis oleh sistem Android saat aplikasi diluncurkan, dan tetap aktif sampai aplikasi dihentikan atau dikeluarkan dari memori.

Fungsi Application Class antara lain:

1. Menjaga Status Global

Application class sangat berguna untuk menyimpan data atau status yang ingin diakses secara global oleh seluruh komponen aplikasi. Misalnya, bisa menyimpan objek singleton, konfigurasi global, atau resource yang dibutuhkan di banyak tempat dalam aplikasi.

2. Inisialisasi Awal

Salah satu fungsi utama Application class adalah menjalankan proses inisialisasi yang hanya perlu dilakukan sekali selama siklus hidup aplikasi. Contohnya, inisialisasi library pihak ketiga (seperti database, analytics, atau dependency injection), setup konfigurasi, atau registrasi receiver global. Semua proses ini biasanya diletakkan di dalam metode onCreate() pada subclass Application.

3. Manajemen Resource dan Lifecycle

Application class juga menyediakan beberapa metode lifecycle seperti onCreate(), onConfigurationChanged(), dan onLowMemory(). Dengan override metode-metode ini, developer bisa mengatur perilaku aplikasi saat terjadi perubahan konfigurasi (misal: perubahan orientasi layar) atau ketika sistem mendeteksi memori yang hampir habis.

Contoh sederhana Application Class seperti berikut:

kotlin

```

class MyApplication : Application() {
    override fun onCreate() {
        super.onCreate()
        // Inisialisasi global di sini
    }
}

```

Dan AndroidManifest.xml:

xml

```

<application
    android:name=".MyApplication"
    ... >
</application>

```

Application class merupakan pusat pengelolaan status dan inisialisasi global pada aplikasi Android. Sangat penting dalam arsitektur aplikasi modern karena memungkinkan pengaturan resource dan proses yang berlaku untuk seluruh aplikasi, bukan hanya pada satu Activity atau Service saja.

Referensi: <https://www.linkedin.com/pulse/application-class-android-amit-nadiger/>

https://github.com/codepath/android_guides/wiki/Understanding-the-Android-Application-Class

MODUL 5: FUNCTION DAN DATABASE

SOAL 1

Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
- b Gunakan KotlinX Serialization sebagai library JSON.
- c Gunakan library seperti Coil atau Glide untuk image loading.
- d API yang digunakan pada modul ini bebas, contoh API gratis The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API: <https://developer.themoviedb.org/docs/getting-started>
- e Implementasikan konsep data persistence (misalnya offline-first app, pengaturan dark/light mode, fitur favorite, dll)
- f Gunakan caching strategy pada Room.
- g Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya.

B. Source Code

1. data/local/dao/DramaDao.kt

Tabel 36 Source Code Jawaban DramaDao.kt Modul 5

1	package com.example.connectapi.data.local.dao
2	
3	import androidx.room.Dao
4	import androidx.room.Insert
5	import androidx.room.OnConflictStrategy
6	import androidx.room.Query
7	import
8	com.example.connectapi.data.local.entities.Drama
	Entity
9	import kotlinx.coroutines.flow.Flow
10	
11	@Dao
12	interface DramaDao {
13	@Insert(onConflict =
	OnConflictStrategy.REPLACE)
14	suspend fun insertDramas(dramas:
	List<DramaEntity>)

15	
16	@Query("SELECT * FROM dramas")
17	fun getAllDramas(): Flow<List<DramaEntity>>
18	
19	@Query("DELETE FROM dramas")
20	suspend fun deleteAllDramas()
21	}

2. data/local/Database/DramaDatabase

Tabel 37 Source Code Jawaban DramaDatabase.kt Modul 5

1	package
	com.example.connectapi.data.local.Database
2	import android.content.Context
3	import androidx.room.Database
4	import androidx.room.Room
5	import androidx.room.RoomDatabase
6	import
	com.example.connectapi.data.local.dao.DramaDao
7	import
	com.example.connectapi.data.local.entities.Drama
	Entity
8	
9	@Database(entities = [DramaEntity::class],
10	version = 1, exportSchema = false)
11	abstract class DramaDatabase : RoomDatabase() {
12	
13	abstract fun dramaDao(): DramaDao
14	
15	companion object {
16	@Volatile
17	private var INSTANCE: DramaDatabase? =
	null
18	
19	fun getDatabase(context: Context):
	DramaDatabase {
20	return INSTANCE ?:
	synchronized(this) {
21	val instance =
	Room.databaseBuilder(
22	context.applicationContext,
23	DramaDatabase::class.java,
24	"drama_db"
25	).build()
26	INSTANCE = instance
27	instance
28	}
29	}
30	}

31	}
----	---

3. data/local/entities/DramaEntity.kt

Tabel 38 Source Code Jawaban DramaEntity.kt Modul 5

1	package
	com.example.connectapi.data.local.entities
2	
3	import androidx.room.Entity
4	import androidx.room.PrimaryKey
5	
6	@Entity(tableName = "dramas")
7	data class DramaEntity(
8	@PrimaryKey val id: Int,
9	val name: String,
10	val overview: String,
11	val posterPath: String?,
12	val firstAirDate: String?
13	
14	)

4. data/remote/api/DramaApiService.kt

Tabel 39 Source Code Jawaban DramaApiService.kt Modul 5

1	package
	com.example.connectapi.data.local.entities
2	
3	import androidx.room.Entity
4	import androidx.room.PrimaryKey
5	
6	@Entity(tableName = "dramas")
7	data class DramaEntity(
8	@PrimaryKey val id: Int,
9	val name: String,
10	val overview: String,
11	val posterPath: String?,
12	val firstAirDate: String?
13	
14	)

5. data/remote/model/DramaDto.kt

Tabel 40 Source Code Jawaban DramaDto.kt Modul 5

1	package com.example.connectapi.data.remote.model
2	
3	import kotlinx.serialization.SerialName
4	import kotlinx.serialization.Serializable
5	
6	@Serializable

7	data class DramaDto(
8	val id: Int,
9	val name: String,
10	val overview: String,
11	@SerializedName("poster_path")
12	val posterPath: String?,
13	@SerializedName("first_air_date")
14	val firstAirDate: String?
15	)

6. data/remote/model/PopularDramasResponse.kt

Tabel 41 Source Code Jawaban PopularDramasResponse.kt Modul 5

1	package com.example.connectapi.data.remote.model
2	import kotlinx.serialization.SerialName
3	import kotlinx.serialization.Serializable
4	
5	@Serializable
6	data class PopularDramasResponse(
7	@SerializedName("page")
8	val page: Int,
9	@SerializedName("results")
10	val results: List<DramaDto>,
11	@SerializedName("total_pages")
12	val totalPages: Int,
13	@SerializedName("total_results")
14	val totalResults: Int
15	)

7. data/remote/ApiClient.kt

Tabel 42 Source Code Jawaban ApiClient.kt Modul 5

1	package com.example.connectapi.data.remote
2	
3	import android.util.Log
4	import com.example.connectapi.BuildConfig
5	import
	com.example.connectapi.data.remote.api.DramaApiService
6	import
	com.jakewharton.retrofit2.converter.kotlinx.serialization.AsConverterFactory
7	import kotlinx.serialization.json.Json
8	import okhttp3.MediaType.Companion.toMediaType
9	import okhttp3.OkHttpClient
10	import okhttp3.logging.HttpLoggingInterceptor
11	import retrofit2.Retrofit
12	import java.util.concurrent.TimeUnit
13	


```

14 object ApiClient {
15     private const val BASE_URL =
16     "https://api.themoviedb.org/3/"
17     private const val TAG = "ApiClient"
18
19     private val json = Json {
20         ignoreUnknownKeys = true
21         prettyPrint = true
22         isLenient = true
23         coerceInputValues = true
24     }
25
26     private val okHttpClient: OkHttpClient by
27     lazy {
28         val builder = OkHttpClient.Builder()
29         .connectTimeout(30,
30         TimeUnit.SECONDS)
31         .readTimeout(30, TimeUnit.SECONDS)
32         .writeTimeout(30, TimeUnit.SECONDS)
33
34         try {
35             val loggingInterceptor =
36             HttpLoggingInterceptor().apply {
37                 setLevel(HttpLoggingInterceptor.Level.BODY)
38             }
39
40             builder.addInterceptor(loggingInterceptor)
41         } catch (e: NoClassDefFoundError) {
42             Log.e(TAG, "HttpLoggingInterceptor
43             class not found. This might be due to missing
44             dependency.", e)
45         } catch (e: Exception) {
46             Log.e(TAG, "Error initializing
47             HttpLoggingInterceptor: ${e.message}", e)
48         }
49
50         builder.build()
51     }
52
53     val dramaApiService: DramaApiService by lazy
54     {
55         val apiKey: String
56         try {
57             apiKey = BuildConfig.TMDB_API_KEY
58             Log.d(TAG, "DEBUG: TMDB_API_KEY yang
59             DIBACA dari BuildConfig: '$apiKey'")
60         }
61     }

```

51	if (apiKey.isNullOrEmpty() apiKey == "null" apiKey.contains("null", ignoreCase = true)) {
52	Log.e(TAG, "FATAL ERROR: TMDB_API_KEY masih kosong atau 'null' saat runtime. Nilai yang dibaca: '\$apiKey'")
53	throw IllegalStateException("TMDB_API_KEY tidak ditemukan atau tidak valid di BuildConfig. Nilai yang dibaca: '\$apiKey'")
54	}
55	Retrofit.Builder()
56	.baseUrl(BASE_URL)
57	.client(okHttpClient)
58	.addConverterFactory(json.asConverterFactory("application/json".toMediaType()))
59	.build()
60	.create(DramaApiService::class.java)
61	} catch (e: IllegalStateException) {
62	Log.e(TAG, "Error in ApiClient initialization due to invalid API Key: \${e.message}", e)
63	throw e
64	} catch (e: Exception) {
65	Log.e(TAG, "Error initializing DramaApiService unexpectedly: \${e.message}", e)
66	throw RuntimeException("Gagal menginisialisasi layanan API. Detail: \${e.localizedMessage}", e)
67	}
68	}
69	}

8. data/remote/repository/DramaRepositoryImpl.kt

Tabel 43 Source Code Jawaban DramaRepositoryImpl.kt Modul 5

1	package com.example.connectapi.data.repository
2	
3	import android.util.Log
4	import com.example.connectapi.data.local.dao.DramaDao
5	import com.example.connectapi.data.local.entities.DramaEntity
6	import com.example.connectapi.data.remote.api.DramaApiService
7	import com.example.connectapi.domain.model.Drama

```

8  import
   com.example.connectapi.domain.repository.DramaRe
   pository
9  import com.example.connectapi.utils.Resource
10 import
   com.example.connectapi.utils.toDomainModel
11 import com.example.connectapi.utils.toEntity
12 import kotlinx.coroutines.Dispatchers
13 import kotlinx.coroutines.flow.Flow
14 import kotlinx.coroutines.flow.firstOrNull
15 import kotlinx.coroutines.flow.flow
16 import kotlinx.coroutines.flow.map
17 import kotlinx.coroutines.withContext
18 import retrofit2.HttpException
19 import java.io.IOException
20
21 class DramaRepositoryImpl(
22     private val dramaDao: DramaDao,
23     private val dramaApiService:
   DramaApiService,
24     private val apiKey: String
25 ) : DramaRepository {
26
27     override fun getPopularDramas():
   Flow<Resource<List<Drama>>> = flow {
28         emit(Resource.Loading())
29
30         val cachedDramasFlow =
   dramaDao.getAllDramas().map { entities ->
31             entities.map { it.toDomainModel() }
32         }
33         val cachedDramas =
   cachedDramasFlow.firstOrNull()
34
35         if (!cachedDramas.isNullOrEmpty()) {
36             emit(Resource.Success(cachedDramas))
37             Log.d("DramaRepositoryImpl",
   "Berhasil mengambil data dari database lokal
   (cached).")
38         } else {
39             Log.d("DramaRepositoryImpl",
   "Database lokal kosong atau tidak ada cached
   data.")
40         }
41         try {
42             val response =
   dramaApiService.getPopularDramas(apiKey =
   apiKey)
43             if (response.isSuccessful) {

```

```

43         val dramaDtos =
44 response.body()?.results ?: emptyList()
         val domainDramas = dramaDtos.map
45 { it.toDomainModel() }

46         withContext(Dispatchers.IO) {
47             dramaDao.deleteAllDramas()
48 dramaDao.insertDramas(dramaDtos.map {
49 it.toEntity() })
         }
50         Log.d("DramaRepositoryImpl",
51 "Berhasil mengambil data dari API dan
memperbarui database.")
emit(Resource.Success(domainDramas))
52     } else {
53         val errorMessage = "API Error:
54 ${response.code()} ${response.message()}"
         Log.e("DramaRepositoryImpl",
55 errorMessage)
emit(Resource.Error(errorMessage, cachedDramas))
56     }
57     } catch (e: HttpException) {
58         val errorMessage = "Network Error
59 (HTTP ${e.code()}) : ${e.message()}"
         Log.e("DramaRepositoryImpl",
60 errorMessage, e)
emit(Resource.Error(errorMessage,
cachedDramas))
61     } catch (e: IOException) {
62         val errorMessage = "No Internet
62 Connection or API Timeout: ${e.message}"
63         Log.e("DramaRepositoryImpl",
64 errorMessage, e)
emit(Resource.Error(errorMessage,
cachedDramas))
65     } catch (e: Exception) {
66         val errorMessage = "An unexpected
66 error occurred: ${e.localizedMessage}"
67         Log.e("DramaRepositoryImpl",
68 errorMessage, e)
emit(Resource.Error(errorMessage,
cachedDramas))
69     }
    }
70     suspend fun insertDramas(dramas:
71 List<Drama>) {
72         withContext(Dispatchers.IO) {
73             dramaDao.insertDramas(dramas.map {
74 it.toEntity() })

```

74	Log.d("DramaRepositoryImpl", "Data drama berhasil dimasukkan ke database.")
75	}
	}
76	suspend fun clearAllDramas() {
77	withContext(Dispatchers.IO) {
78	dramaDao.deleteAllDramas()
79	Log.d("DramaRepositoryImpl", "Semua data drama di database telah dihapus.")
80	}
81	}
82	}
83	
84	

9. data/remote/Utils/DomainMappers.kt

Tabel 44 Source Code Jawaban DramaMappers.kt Modul 5

1	package com.example.connectapi.utils
2	
3	import
	com.example.connectapi.data.local.entities.Drama
	Entity
4	import
	com.example.connectapi.data.remote.model.DramaDt
	o
5	import com.example.connectapi.domain.model.Drama
6	
7	fun DramaDto.toDomainModel(): Drama {
8	return Drama(
9	id = id,
10	name = name,
11	overview = overview,
12	posterPath = posterPath,
13	firstAirDate = firstAirDate
14	)
15	}
16	fun DramaEntity.toDomainModel(): Drama {
17	return Drama(
18	id = id,
19	name = name,
20	overview = overview,
21	posterPath = posterPath,
22	firstAirDate = firstAirDate
23	)
24	}
25	fun DramaDto.toEntity(): DramaEntity {
26	return DramaEntity(
27	id = id,

28	name = name,
29	overview = overview,
30	posterPath = posterPath,
31	firstAirDate = firstAirDate
32	)
33	}
34	fun Drama.toEntity(): DramaEntity {
35	return DramaEntity(
36	id = id,
37	name = name,
38	overview = overview,
39	posterPath = posterPath,
40	firstAirDate = firstAirDate
41	)
42	}

10. domain/model/Drama.kt

Tabel 45 Source Code Jawaban Drama.kt Modul 5

1	package com.example.connectapi.domain.model
2	
3	import android.os.Parcelable
4	import kotlinx.parcelize.Parcelize
5	
6	@Parcelize
7	data class Drama(
8	val id: Int,
9	val name: String,
10	val overview: String,
11	val posterPath: String?,
12	val firstAirDate: String?
13	) : Parcelable

11. domain/repository/DramaRepository.kt

Tabel 46 Source Code Jawaban DramaRepository.kt Modul 5

1	package com.example.connectapi.domain.repository
2	
3	import com.example.connectapi.domain.model.Drama
4	import com.example.connectapi.utils.Resource
5	import kotlinx.coroutines.flow.Flow
6	
7	interface DramaRepository {
8	fun getPopularDramas():
	Flow<Resource<List<Drama>>>
9	}

12. domain/usecase/GetDramaUseCase

Tabel 47 Source Code Jawaban GetDramaUseCase.kt Modul 5

1	package com.example.connectapi.domain.usecase
2	
3	import com.example.connectapi.domain.model.Drama
4	import
	com.example.connectapi.domain.repository.DramaRe
	pository
5	import com.example.connectapi.utils.Resource
6	import kotlinx.coroutines.flow.Flow
7	
8	class GetDramaUseCase(private val
	dramaRepository: DramaRepository) {
9	operator fun invoke():
	Flow<Resource<List<Drama>>> {
10	return
	dramaRepository.getPopularDramas()
11	}
12	}

13. presentation/ui/DetailFragment

Tabel 48 Source Code Jawaban DetailFragment.kt Modul 5

1	package com.example.connectapi.presentation.ui
2	
3	import android.os.Bundle
4	import android.view.LayoutInflater
5	import android.view.View
6	import android.view.ViewGroup
7	import androidx.fragment.app.Fragment
8	import coil.load
	import
	com.example.connectapi.databinding.FragmentDetailBinding
9	import com.example.connectapi.domain.model.Drama
10	class DetailFragment : Fragment() {
11	private var _binding: FragmentDetailBinding?
12	= null
13	private val binding get() = _binding!!
14	override fun onCreateView(
15	inflater: LayoutInflater, container:
16	ViewGroup?,
17	savedInstanceState: Bundle?
	): View {
18	_binding =
19	FragmentDetailBinding.inflate(inflater,
20	container, false)

	return binding.root
	}
21	override fun onCreateView(view: View,
22	savedInstanceState: Bundle?) {
23	super.onCreateView(view,
	savedInstanceState)
24	
	val drama =
25	arguments?.getParcelable<Drama>("drama")
26	drama?.let {
	binding.tvDetailTitle.text = it.name
27	binding.tvDetailOverview.text =
28	it.overview
29	binding.tvDetailReleaseDate.text =
	it.firstAirDate
30	binding.ivDetailPoster.load("https://image.tmdb.
	org/t/p/w500\${it.posterPath}") {
31	crossfade(true)
	}
32	}
33	}
34	override fun onDestroyView() {
35	super.onDestroyView()
36	_binding = null
37	}
38	}
39	
40	

14. presentation/ui/DramaAdapter.kt

Tabel 49 Source Code Jawaban DramaAdapter.kt Modul 5

1	package com.example.connectapi.presentation.ui
2	
3	import android.view.LayoutInflater
4	import android.view.ViewGroup
5	import androidx.recyclerview.widget.DiffUtil
6	import androidx.recyclerview.widget.ListAdapter
7	import androidx.recyclerview.widget.RecyclerView
8	import coil.load
9	import
	com.example.connectapi.databinding.ItemDramaBind
	ing
10	import com.example.connectapi.domain.model.Drama
11	
12	class DramaAdapter : ListAdapter<Drama,
	DramaAdapter.DramaViewHolder>(DramaDiffCallback(
	)) {
13	


```

14     var onDetailClickListener: ((Drama) ->
    Unit)? = null
15     var onNetflixClickListener: ((Drama) ->
    Unit)? = null
16
17     override fun onCreateViewHolder(parent:
    ViewGroup, viewType: Int): DramaViewHolder {
18         val binding =
    ItemDramaBinding.inflate(LayoutInflater.from(par
    ent.context), parent, false)
19         return DramaViewHolder(binding)
20     }
21
22     override fun onBindViewHolder(holder:
    DramaViewHolder, position: Int) {
23         val drama = getItem(position)
24         holder.bind(drama)
25     }
26
27     inner class DramaViewHolder(private val
    binding: ItemDramaBinding) :
    RecyclerView.ViewHolder(binding.root) {
28         fun bind(drama: Drama) {
29             binding.tvTitle.text = drama.name
30             binding.tvOverview.text =
    drama.overview
31             binding.tvReleaseDate.text =
    drama.firstAirDate
32
    binding.ivPoster.load("https://image.tmbd.org/t/
    p/w500${drama.posterPath}") {
33                 crossfade(true)
34             }
35
36             binding.btnDetail.setOnClickListener
    {
37                 onDetailClickListener?.invoke(drama)
38             }
39         }
40
41         class DramaDiffCallback :
    DiffUtil.ItemCallback<Drama>() {
42             override fun areItemsTheSame(oldItem:
    Drama, newItem: Drama): Boolean {
43                 return oldItem.id == newItem.id
44             }
45
46

```

47	override fun areContentsTheSame(oldItem:
48	Drama, newItem: Drama): Boolean {
	return oldItem == newItem
49	}
50	}
51	}
52	

15. presentation/ui/DramaListFragment.kt

Tabel 50 Source Code Jawaban DramaListFragment.kt Modul 5

1	package com.example.connectapi.presentation.ui
2	
3	import android.os.Bundle
4	import android.util.Log
5	import android.view.LayoutInflater
6	import android.view.View
7	import android.view.ViewGroup
8	import android.widget.Toast
9	import androidx.fragment.app.Fragment
10	import androidx.fragment.app.viewModels
11	import androidx.lifecycle.Lifecycle
12	import androidx.lifecycle.LifecycleScope
13	import androidx.lifecycle.repeatOnLifecycle
14	import
	androidx.navigation.fragment.findNavController
15	import
	androidx.recyclerview.widget.LinearLayoutManager
	er
16	import com.example.connectapi.BuildConfig
17	import com.example.connectapi.R
18	import
	com.example.connectapi.data.local.Database.DramaDatabase
19	import
	com.example.connectapi.data.remote.ApiClient
20	import
	com.example.connectapi.data.repository.DramaRepositoryImpl
21	import
	com.example.connectapi.databinding.FragmentDramaListBinding
22	import
	com.example.connectapi.domain.usecase.GetDramaUseCase
23	import com.example.connectapi.utils.Resource
24	import
	com.example.connectapi.utils.AppPreferences
25	import
	androidx.appcompat.app.AppCompatActivity

```

26 import kotlinx.coroutines.launch
27
28 class DramaListFragment : Fragment() {
29     private var _binding:
30     FragmentDramaListBinding? = null
31     private val binding get() = _binding!!
32
33     private val dramaViewModel: DramaViewModel
34     by viewModels {
35         val dramaDao =
36         DramaDatabase.getDatabase(requireContext()).dr
37         amaDao()
38         val dramaApiService =
39         ApiClient.dramaApiService
40         val apiKey = BuildConfig.TMDB_API_KEY
41
42         val dramaRepository =
43         DramaRepositoryImpl(dramaDao, dramaApiService,
44         apiKey)
45         val getDramaUseCase =
46         GetDramaUseCase(dramaRepository)
47
48         DramaViewModelFactory(requireActivity().applic
49         ation, getDramaUseCase)
50     }
51     private lateinit var dramaAdapter:
52     DramaAdapter
53     private lateinit var appPreferences:
54     AppPreferences
55
56     override fun onCreateView(
57         inflater: LayoutInflater, container:
58         ViewGroup?,
59         savedInstanceState: Bundle?
60     ): View {
61         _binding =
62         FragmentDramaListBinding.inflate(inflater,
63         container, false)
64         return binding.root
65     }
66     override fun onViewCreated(view: View,
67         savedInstanceState: Bundle?) {
68         super.onViewCreated(view,
69         savedInstanceState)
70
71         appPreferences =
72         AppPreferences(requireContext())
73
74         setupRecyclerView()

```

```

58         observeViewModel()
59
60         val darkModeSwitch =
61             binding.darkModeSwitch
62
63             darkModeSwitch.isChecked =
64                 appPreferences.getDarkModeState()
65
66                 darkModeSwitch.setOnCheckedChangeListener {_,
67                     isChecked ->
68                         appPreferences.saveDarkModeState(isChecked)
69
70                         applyTheme(isChecked)
71                     }
72                 binding.btnRefreshApi.setOnClickListener {
73                     dramaViewModel.refreshDramasFromLocal()
74                 }
75                 binding.btnClearLocalData.setOnClickListener {
76                     clearLocalData()
77                 }
78                 private fun setupRecyclerView() {
79                     dramaAdapter = DramaAdapter()
80                     binding.rvDrama.apply {
81                         layoutManager =
82                             LinearLayoutManager(context)
83                         adapter = dramaAdapter
84                     }
85                     dramaAdapter.onDetailClickListener = {
86                         drama ->
87                             val bundle = Bundle().apply {
88                                 putParcelable("drama", drama)
89                             }
90                             findNavController().navigate(R.id.action_drama
91                                 ListFragment_to_detailFragment, bundle)
92                         }
93                     dramaAdapter.onNetflixClickListener =
94                     { drama ->
95                         Toast.makeText(context, "Tombol
96                             Netflix untuk ${drama.name} diklik!",
97                             Toast.LENGTH_SHORT).show()
98                     }
99                 }
100                 private fun observeViewModel() {
101                     viewLifecycleOwner.lifecycleScope.launch {
102                         repeatOnLifecycle(Lifecycle.State.STARTED) {
103                             dramaViewModel.popularDramasState.collect {
104                                 resource ->
105                                     when (resource) {

```

```

97                                     is Resource.Loading ->
98     {
99         if
100     (resource.data.isNullOrEmpty()) {
101     binding.progressBar.visibility = View.VISIBLE
102     binding.rvDrama.visibility = View.GONE
103     binding.tvError.visibility = View.GONE
104     } else {
105     binding.progressBar.visibility = View.GONE
106     binding.rvDrama.visibility = View.VISIBLE
107     binding.tvError.visibility = View.GONE
108     dramaAdapter.submitList(resource.data)
109     }
110     }
111     is Resource.Success ->
112     {
113     binding.progressBar.visibility = View.GONE
114     binding.rvDrama.visibility = View.VISIBLE
115     binding.tvError.visibility = View.GONE
116     resource.data?.let
117     {
118     dramaAdapter.submitList(it)
119     }
120     }
121     is Resource.Error -> {
122     binding.progressBar.visibility = View.GONE
123     binding.rvDrama.visibility = View.GONE
124     binding.tvError.visibility = View.VISIBLE
125     binding.tvError.text = resource.message ?:
126     "Terjadi kesalahan tidak diketahui"
127
128     Toast.makeText(context, "Error:
129     ${resource.message}",
130     Toast.LENGTH_LONG).show()
131     Log.e("DramaListFragment", "Error:
132     ${resource.message}")
133     resource.data?.let
134     {
135         if
136     (it.isNotEmpty()) {
137     dramaAdapter.submitList(it)
138     binding.rvDrama.visibility = View.VISIBLE
139     binding.tvError.visibility = View.GONE
140     }
141     }
142     }
143     }
144     }

```

135	}
136	}
137	private fun clearLocalData() {
138	lifecycleScope.launch {
139	try {
140	dramaViewModel.clearLocalData()
141	Toast.makeText(context, "Data lokal berhasil dihapus!", Toast.LENGTH_SHORT).show()
142	} catch (e: Exception) {
143	Toast.makeText(context, "Gagal menghapus data lokal: \${e.localizedMessage}", Toast.LENGTH_SHORT).show()
144	Log.e("DramaListFragment", "Error clearing local data: \${e.localizedMessage}", e)
145	}
146	}
147	}
148	private fun applyTheme(isDarkMode: Boolean) {
149	if (isDarkMode) {
150	AppCompatDelegate.setDefaultNightMode(AppCompa tDelegate.MODE_NIGHT_YES)
151	} else {
152	AppCompatDelegate.setDefaultNightMode(AppCompa tDelegate.MODE_NIGHT_NO)
153	}
154	requireActivity().recreate()
155	}
156	override fun onDestroyView() {
157	super.onDestroyView()
158	_binding = null
159	}
160	}

16. presentation/ui/DramaViewModel.kt

Tabel 51 Source Code Jawaban DramaViewModel.kt Modul 5

1	package com.example.connectapi.presentation.ui
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.viewModelScope
5	import com.example.connectapi.domain.model.Drama
6	import com.example.connectapi.domain.repository.DramaRe pository
7	

```

import
8  com.example.connectapi.domain.usecase.GetDramaUs
9  eCase
10 import com.example.connectapi.utils.Resource
11 import kotlinx.coroutines.flow.MutableStateFlow
12 import kotlinx.coroutines.flow.StateFlow
13 import kotlinx.coroutines.flow.asStateFlow
14 import kotlinx.coroutines.launch
15
16 class DramaViewModel(
17     private val getDramaUseCase:
18     GetDramaUseCase,
19     private val dramaRepository: DramaRepository
20 ) : ViewModel() {
21
22     private val _popularDramasState =
23     MutableStateFlow<Resource<List<Drama>>>(Resource
24     .Loading())
25     val popularDramasState:
26     StateFlow<Resource<List<Drama>>> =
27     _popularDramasState.asStateFlow()
28
29     init {
30         fetchPopularDramas()
31     }
32     private fun fetchPopularDramas() {
33         viewModelScope.launch {
34             getDramaUseCase().collect { resource
35 ->
36                 _popularDramasState.value =
37 resource
38             }
39         }
40     }
41     fun refreshDramasFromLocal() {
42         fetchPopularDramas()
43     }
44
45     fun clearLocalData() {
46         viewModelScope.launch {
47             (dramaRepository as?
48 com.example.connectapi.data.repository.DramaRepo
49 sitoryImpl)?.clearAllDramas()
50             fetchPopularDramas()
51         }
52     }
53 }

```

17. presentation/ui/DramaViewModelFactory.kt

Tabel 52 Source Code Jawaban DramaViewModelFactory.kt Modul 5

```
1 package com.example.connectapi.presentation.ui
2
3 import android.app.Application
4 import androidx.lifecycle.ViewModel
5 import androidx.lifecycle.ViewModelProvider
6 import
  com.example.connectapi.data.local.Database.Drama
  Database
7 import
  com.example.connectapi.data.remote.ApiClient
8 import
  com.example.connectapi.data.repository.DramaRepo
  sitoryImpl
9 import
  com.example.connectapi.domain.usecase.GetDramaUs
  eCase
10 import com.example.connectapi.BuildConfig
11
12 class DramaViewModelFactory(
13     private val application: Application,
14     private val getPopularDramaUseCase:
  GetDramaUseCase
15 ) : ViewModelProvider.Factory {
16     override fun <T : ViewModel>
  create(modelClass: Class<T>): T {
17         if
  (modelClass.isAssignableFrom(DramaViewModel::cla
  ss.java)) {
18
19             val dramaDao =
  DramaDatabase.getDatabase(application).dramaDao(
  )
20             val dramaApiService =
  ApiClient.dramaApiService
21             val apiKey =
  BuildConfig.TMDB_API_KEY
22             val dramaRepository =
  DramaRepositoryImpl(dramaDao, dramaApiService,
  apiKey)
23             val getDramaUseCase =
  GetDramaUseCase(dramaRepository)
24
25             @Suppress("UNCHECKED_CAST")
26             return
  DramaViewModel(getDramaUseCase, dramaRepository)
  as T
27         }
```


28	throw IllegalArgumentException("Unknown ViewModel class")
29	}
30	}

18. MainActivity.kt

Tabel 53 Source Code Jawaban MainActivity.kt Modul 5

1	package com.example.connectapi.presentation.ui
2	
3	import android.os.Bundle
4	import androidx.appcompat.app.AppCompatActivity
5	import androidx.appcompat.app.AppCompatActivityDelegate
6	import
	androidx.navigation.fragment.NavHostFragment
7	import
	androidx.navigation.ui.setupActionBarWithNavController
8	import
	com.example.connectapi.databinding.ActivityMainB
	inding
9	import
10	com.example.connectapi.utils.AppPreferences
11	
12	class MainActivity : AppCompatActivity() {
13	private lateinit var binding:
	ActivityMainBinding
14	private lateinit var appPreferences:
	AppPreferences
15	override fun onCreate(savedInstanceState:
	Bundle?) {
16	super.onCreate(savedInstanceState)
17	binding =
	ActivityMainBinding.inflate(layoutInflater)
18	setContentView(binding.root)
19	
20	appPreferences = AppPreferences(this)
21	
22	applySavedTheme()
23	
24	setSupportActionBar(binding.toolbar)
25	
26	val navHostFragment =
	supportFragmentManager
27	.findFragmentById(binding.navHostFragment.id) as
	NavHostFragment
28	val navController =
	navHostFragment.navController
29	setupActionBarWithNavController(navController)

```

30         }
31         override fun onSupportNavigateUp(): Boolean
32     {
33         val navHostFragment =
34             supportFragmentManager
35             .findFragmentById(binding.navHostFragment.id) as
36             NavHostFragment
37         val navController =
38             navHostFragment.navController
39         return navController.navigateUp() ||
40             super.onSupportNavigateUp()
41     }
42     private fun applySavedTheme() {
43         val isDarkMode =
44             appPreferences.getDarkModeState()
45             AppCompatActivity.setDefaultNightMode(
46                 if (isDarkMode)
47                     AppCompatActivity.MODE_NIGHT_YES
48                 else AppCompatActivity.MODE_NIGHT_NO
49             )
50     }
51 }

```

19. Utils/AppPreferences.kt

Tabel 54 Source Code Jawaban AppPreferences.kt Modul 5

```

1 package com.example.connectapi.utils
2
3 import android.content.Context
4 import android.content.SharedPreferences
5
6 class AppPreferences(context: Context) {
7     private val PREFS_NAME = "app_prefs"
8     private val KEY_DARK_MODE = "dark_mode"
9     private val sharedPrefs: SharedPreferences
10
11     init {
12         sharedPrefs =
13             context.getSharedPreferences(PREFS_NAME,
14             Context.MODE_PRIVATE)
15     }
16     fun saveDarkModeState(isDarkMode: Boolean) {
17         sharedPrefs.edit().putBoolean(KEY_DARK_MODE,
18             isDarkMode).apply()
19     }
20     fun getDarkModeState(): Boolean {
21

```

19	return
20	sharedPrefs.getBoolean(KEY_DARK_MODE, false)
	}

20. Utils/Resource.kt

Tabel 55 Source Code Jawaban Resource.kt Modul 5

1	package com.example.connectapi.utils
2	
3	sealed class Resource<T>(val data: T? = null, val message: String? = null) {
4	class Loading<T>(data: T? = null) : Resource<T>(data)
5	class Success<T>(data: T) : Resource<T>(data)
6	class Error<T>(message: String, data: T? = null) : Resource<T>(data, message)
7	}

21. Activity_main.xml

Tabel 56 Source Code Jawaban Activity_main.xml Modul 5

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent"
7	tools:context=".presentation.ui.MainActivity">
8	
9	<com.google.android.material.appbar.AppBarLayout
10	android:id="@+id/app_bar_layout"
11	android:layout_width="match_parent"
12	android:layout_height="wrap_content"
13	app:layout_constraintTop_toTopOf="parent">
14	
15	<androidx.appcompat.widget.Toolbar
16	android:id="@+id/toolbar"
17	android:layout_width="match_parent"
18	android:layout_height="53dp"
19	android:background="?attr/colorPrimary"
20	app:titleTextColor="@android:color/white" />

21	</com.google.android.material.appbar.AppBarLayout>
22	<androidx.fragment.app.FragmentContainerView
23	android:id="@+id/nav_host_fragment"
24	android:name="androidx.navigation.fragment.NavHostFragment"
25	android:layout_width="0dp"
26	android:layout_height="0dp"
27	app:defaultNavHost="true"
28	app:layout_constraintBottom_toBottomOf="parent"
29	app:layout_constraintEnd_toEndOf="parent"
30	app:layout_constraintStart_toStartOf="parent"
31	app:layout_constraintTop_toBottomOf="@id/app_bar_layout"
32	app:navGraph="@navigation/nav_graph" />
33	
34	</androidx.constraintlayout.widget.ConstraintLayout>

22. fragment_detail.xml

Tabel 57 Source Code Jawaban fragment_detail.xml Modul 5

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	xmlns:tools="http://schemas.android.com/tools"
6	android:layout_width="match_parent"
7	android:layout_height="match_parent"
8	tools:context=".presentation.ui.DetailFragment">
9	
10	<ImageView
11	android:id="@+id/iv_detail_poster"
12	android:layout_width="match_parent"
13	android:layout_height="250dp"
14	android:scaleType="centerCrop"
15	android:src="@drawable/ic_launcher_background"
16	app:layout_constraintTop_toTopOf="parent"
17	app:layout_constraintStart_toStartOf="parent"
18	app:layout_constraintEnd_toEndOf="parent" />
19	
20	<TextView
21	android:id="@+id/tv_detail_title"
22	android:layout_width="0dp"

```

23         android:layout_height="wrap_content"
24         android:layout_marginTop="16dp"
25         android:layout_marginStart="16dp"
26         android:layout_marginEnd="16dp"
27         android:text="Judul Drama"
28         android:textSize="24sp"
29         android:textStyle="bold"
30     app:layout_constraintTop_toBottomOf="@id/iv_de
tail_poster"
31     app:layout_constraintStart_toStartOf="parent"
32     app:layout_constraintEnd_toEndOf="parent" />
33
34     <TextView
35     android:id="@+id/tv_detail_release_date"
36         android:layout_width="0dp"
37         android:layout_height="wrap_content"
38         android:layout_marginTop="8dp"
39         android:layout_marginStart="16dp"
40         android:layout_marginEnd="16dp"
41         android:text="Tanggal Rilis: YYYY-MM-
DD"
42         android:textSize="16sp"
43         android:alpha="0.7"
44     app:layout_constraintTop_toBottomOf="@id/tv_de
tail_title"
45     app:layout_constraintStart_toStartOf="parent"
46     app:layout_constraintEnd_toEndOf="parent" />
47
48     <TextView
49         android:id="@+id/tv_detail_overview"
50         android:layout_width="0dp"
51         android:layout_height="0dp"
52         android:layout_marginTop="16dp"
53         android:layout_marginStart="16dp"
54         android:layout_marginEnd="16dp"
55         android:layout_marginBottom="16dp"
56         android:text="Deskripsi panjang
drama..."
57         android:textSize="14sp"
58         android:scrollbars="vertical"
59     app:layout_constraintTop_toBottomOf="@id/tv_de
tail_release_date"
60     app:layout_constraintBottom_toBottomOf="parent"
61     "
62     app:layout_constraintStart_toStartOf="parent"
63     app:layout_constraintEnd_toEndOf="parent" />
64 </androidx.constraintlayout.widget.ConstraintL
ayout>

```

23. fragment_drama_list.xml

Tabel 58Source Code Jawaban fragment drama list.xml Modul 5

1	<androidx.constraintlayout.widget.ConstraintLayo
2	ut
3	xmlns:android="http://schemas.android.com/apk/re
4	s/android"
5	xmlns:app="http://schemas.android.com/apk/res-
6	auto"
7	xmlns:tools="http://schemas.android.com/tools"
8	android:layout_width="match_parent"
9	android:layout_height="match_parent"
10	tools:context=".presentation.ui.DramaListFragmen
11	t">
12	
13	<Button
14	android:id="@+id/btn_refresh_api"
15	android:layout_width="0dp"
16	android:layout_height="wrap_content"
17	android:text="Refresh dari API"
18	app:layout_constraintTop_toTopOf="parent"
19	app:layout_constraintStart_toStartOf="parent"
20	app:layout_constraintEnd_toStartOf="@id/guidelin
21	e_center"
22	android:layout_marginTop="16dp"
23	android:layout_marginStart="16dp"
24	android:layout_marginEnd="8dp" />
25	
26	<Button
27	android:id="@+id/btn_clear_local_data"
28	android:layout_width="0dp"
29	android:layout_height="wrap_content"
30	android:text="Hapus Data Lokal"
31	app:layout_constraintTop_toTopOf="@id/btn_refres
32	h_api"
33	app:layout_constraintStart_toEndOf="@id/guidelin
34	e_center"
35	app:layout_constraintEnd_toEndOf="parent"
36	android:layout_marginStart="8dp"
37	android:layout_marginEnd="16dp" />
38	
39	<androidx.constraintlayout.widget.Guideline
40	android:id="@+id/guideline_center"
41	android:layout_width="wrap_content"
42	android:layout_height="wrap_content"
43	android:orientation="vertical"
44	app:layout_constraintGuide_percent="0.5"
45	/>

```

40 <com.google.android.material.switchmaterial.Swit
41 chMaterial
42     android:id="@+id/darkModeSwitch"
43     android:layout_width="wrap_content"
44     android:layout_height="wrap_content"
45     android:layout_marginTop="8dp"
46     android:padding="8dp"
47     android:text="Aktifkan Dark Mode"
48 android:textColor="?attr/colorOnBackground"
49     android:textSize="16sp"
50     app:layout_constraintEnd_toEndOf="parent"
51 app:layout_constraintHorizontal_bias="0.862"
52 app:layout_constraintStart_toStartOf="parent"
53 app:layout_constraintTop_toBottomOf="@id/btn_cle
54 ar_local_data" />
55     <ProgressBar
56         android:id="@+id/progressBar"
57         android:layout_width="wrap_content"
58         android:layout_height="wrap_content"
59         android:visibility="gone"
60 android:indeterminateTint="?attr/colorPrimary"
61 app:layout_constraintBottom_toBottomOf="parent"
62 app:layout_constraintEnd_toEndOf="parent"
63 app:layout_constraintStart_toStartOf="parent"
64 app:layout_constraintTop_toBottomOf="@id/darkMod
65 eSwitch"
66         tools:visibility="visible" />
67     <androidx.recyclerview.widget.RecyclerView
68         android:id="@+id/rvDrama"
69         android:layout_width="0dp"
70         android:layout_height="0dp"
71         android:paddingTop="8dp"
72 app:layout_constraintBottom_toBottomOf="parent"
73 app:layout_constraintEnd_toEndOf="parent"
74 app:layout_constraintStart_toStartOf="parent"
75 app:layout_constraintTop_toBottomOf="@id/darkMod
76 eSwitch"
77         tools:listitem="@layout/item_drama" />
78     <TextView
79         android:id="@+id/tvError"
80         android:layout_width="wrap_content"
81         android:layout_height="wrap_content"
82         android:text="Terjadi kesalahan!"

```

83	android:textColor="@color/red_error"
84	android:textSize="16sp"
85	android:visibility="gone"
86	app:layout_constraintBottom_toBottomOf="parent"
87	app:layout_constraintEnd_toEndOf="parent"
88	app:layout_constraintStart_toStartOf="parent"
89	app:layout_constraintTop_toBottomOf="@id/darkMod eSwitch"
90	tools:visibility="visible" /> </androidx.constraintlayout.widget.ConstraintLayo ut>

24. item_drama.xml

Tabel 59 Source Code Jawaban item_drama.xml Modul 5

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.cardview.widget.CardView
3	xmlns:android="http://schemas.android.com/apk/re s/android"
4	xmlns:app="http://schemas.android.com/apk/res- auto"
5	xmlns:tools="http://schemas.android.com/tools"
6	android:layout_width="match_parent"
7	android:layout_height="wrap_content"
8	android:layout_margin="8dp"
9	app:cardCornerRadius="8dp"
10	app:cardElevation="4dp">
11	
12	<androidx.constraintlayout.widget.ConstraintLayo ut
13	android:layout_width="match_parent"
14	android:layout_height="wrap_content"
15	android:padding="16dp">
16	
17	<ImageView
18	android:id="@+id/iv_poster"
19	android:layout_width="100dp"
20	android:layout_height="150dp"
21	android:scaleType="centerCrop"
22	android:src="@drawable/ic_launcher_background"
23	app:layout_constraintStart_toStartOf="parent"
24	app:layout_constraintTop_toTopOf="parent"
25	app:layout_constraintBottom_toBottomOf="parent" >
26	
27	<TextView
28	android:id="@+id/tv_title"
29	android:layout_width="0dp"
30	android:layout_height="wrap_content"


```

31         android:layout_marginStart="16dp"
32         android:layout_marginEnd="8dp"
33         android:text="Judul Drama"
34         android:textSize="18sp"
35         android:textStyle="bold"
36 app:layout_constraintTop_toTopOf="parent"
37 app:layout_constraintStart_toEndOf="@id/iv_poster"
38 app:layout_constraintEnd_toEndOf="parent" />
39
40     <TextView
41         android:id="@+id/tv_release_date"
42         android:layout_width="0dp"
43         android:layout_height="wrap_content"
44         android:layout_marginStart="16dp"
45         android:layout_marginTop="4dp"
46         android:layout_marginEnd="8dp"
47         android:text="Tanggal Rilis: YYYY-
MM-DD"
48         android:textSize="14sp"
49         android:alpha="0.7"
50 app:layout_constraintTop_toBottomOf="@id/tv_title"
51 app:layout_constraintStart_toEndOf="@id/iv_poster"
52 app:layout_constraintEnd_toEndOf="parent" />
53
54     <TextView
55         android:id="@+id/tv_overview"
56         android:layout_width="0dp"
57         android:layout_height="wrap_content"
58         android:layout_marginStart="16dp"
59         android:layout_marginTop="8dp"
60         android:layout_marginEnd="8dp"
61         android:maxLines="3"
62         android:ellipsize="end"
63         android:text="Ini adalah ringkasan
64 singkat dari drama..."
65         android:textSize="14sp"
66 app:layout_constraintTop_toBottomOf="@id/tv_release_date"
67 app:layout_constraintStart_toEndOf="@id/iv_poster"
68 app:layout_constraintEnd_toEndOf="parent" />
69
70     <LinearLayout
71         android:layout_width="0dp"
72         android:layout_height="wrap_content"
73         android:orientation="horizontal"

```

74	android:layout_marginTop="8dp"
75	app:layout_constraintTop_toBottomOf="@id/tv_over view"
76	app:layout_constraintStart_toEndOf="@id/iv_poste r"
77	app:layout_constraintEnd_toEndOf="parent"
78	app:layout_constraintBottom_toBottomOf="parent"
79	android:layout_marginStart="16dp">
80	
81	<Button
82	android:id="@+id/btn_detail"
83	android:layout_width="0dp"
	android:layout_height="wrap_content"
84	android:layout_weight="1"
85	android:text="Detail"
86	android:layout_marginEnd="4dp"/>
87	
88	</LinearLayout>
89	
90	</androidx.constraintlayout.widget.ConstraintLay out>
91	</androidx.cardview.widget.CardView>

25. main_menu.xml

Tabel 60 Source Code Jawaban main_menu.xml Modul 5

1	<?xml version="1.0" encoding="utf-8"?>
2	<menu
	xmlns:android="http://schemas.android.com/apk/re s/android"
3	xmlns:app="http://schemas.android.com/apk/res- auto">
4	<item
5	android:id="@+id/action_dark_mode_switch"
6	android:title="Dark Mode"
7	app:actionViewClass="com.google.android.material .switchmaterial.SwitchMaterial"
8	app:showAsAction="always" />
9	</menu>

26. nav_graph.xml

Tabel 61 Source Code Jawaban nav_graph.xml Modul 5

1	<?xml version="1.0" encoding="utf-8"?>
2	<navigation
	xmlns:android="http://schemas.android.com/apk/re s/android"
3	xmlns:app="http://schemas.android.com/apk/res-

4	auto"
5	xmlns:tools="http://schemas.android.com/tools"
6	android:id="@+id/nav_graph"
7	app:startDestination="@id/dramaListFragment">
8	
9	<fragment
10	android:id="@+id/dramaListFragment"
11	android:name="com.example.connectapi.presentation.ui.DramaListFragment"
12	android:label="Series Lawas Populer"
13	tools:layout="@layout/fragment_drama_list">
14	<action
15	android:id="@+id/action_dramaListFragment_to_detailFragment"
16	app:destination="@id/detailFragment"
17	/>
18	</fragment>
19	<fragment
20	android:id="@+id/detailFragment"
21	android:name="com.example.connectapi.presentation.ui.DetailFragment"
22	android:label="Drama Detail"
23	tools:layout="@layout/fragment_detail">
24	<argument
25	android:name="drama"
26	app:argType="com.example.connectapi.domain.model.Drama" />
27	</argument>
28	</fragment>
29	</navigation>
30	

27. colors.xml

Tabel 62 Source Code Jawaban colors.xml Modul 5

1	<?xml version="1.0" encoding="utf-8"?>
2	<resources>
3	
4	<color name="black">#FF000000</color>
5	<color name="white">#FFFFFFFF</color>
6	<color name="red_error">#FFDA4433</color>
7	
8	<color
9	name="md_theme_light_primary">#6750A4</color>
10	<color
11	name="md_theme_light_onPrimary">#FFFFFF</color>
12	<color
13	name="md_theme_light_primaryContainer">#EADDFD</color>
14	</color>
15	

12	<color name="md_theme_light_onPrimaryContainer">#21005D </color>
13	<color name="md_theme_light_secondary">#625B71</color> <color
14	name="md_theme_light_onSecondary">#FFFFFF</color > <color
15	name="md_theme_light_secondaryContainer">#E8DEF8 </color> <color
16	name="md_theme_light_onSecondaryContainer">#1D19 2B</color> <color
17	name="md_theme_light_tertiary">#7D5260</color> <color
18	name="md_theme_light_onTertiary">#FFFFFF</color> <color
19	name="md_theme_light_tertiaryContainer">#FFD8E4< </color> <color
20	name="md_theme_light_onTertiaryContainer">#31111 D</color> <color
21	name="md_theme_light_error">#BA1A1A</color> <color
22	name="md_theme_light_onError">#FFFFFF</color> <color
23	name="md_theme_light_errorContainer">#FFDAD6</co lor> <color
24	name="md_theme_light_onErrorContainer">#410002</ color> <color
25	name="md_theme_light_background">#FFFBFE</color> <color
26	name="md_theme_light_onBackground">#1C1B1F</colo r> <color
27	name="md_theme_light_surface">#FFFBFE</color> <color
28	name="md_theme_light_onSurface">#1C1B1F</color> <color
29	name="md_theme_light_surfaceVariant">#E7E0EC</co lor> <color
30	name="md_theme_light_onSurfaceVariant">#49454F</ color>

31	<color name="md_theme_light_outline">#7A757F</color>
32	<color name="md_theme_light_shadow">#000000</color>
33	<color name="md_theme_light_inverseSurface">#313034</color>
34	<color name="md_theme_light_inverseOnSurface">#F4EFF4</color>
35	<color name="md_theme_light_inversePrimary">#D0BCFF</color>
36	<color name="md_theme_light_surfaceTint">#6750A4</color>
37	>
38	<color name="md_theme_dark_primary">#D0BCFF</color>
39	<color name="md_theme_dark_onPrimary">#381E72</color>
40	<color name="md_theme_dark_primaryContainer">#4F378B</color>
41	<color name="md_theme_dark_onPrimaryContainer">#EADDDFF</color>
42	<color name="md_theme_dark_secondary">#CCC2DC</color>
43	<color name="md_theme_dark_onSecondary">#332D41</color>
44	<color name="md_theme_dark_secondaryContainer">#4A4458</color>
45	<color name="md_theme_dark_onSecondaryContainer">#E8DEF8</color>
46	<color name="md_theme_dark_tertiary">#EFB8C8</color>
47	<color name="md_theme_dark_onTertiary">#492532</color>
48	<color name="md_theme_dark_tertiaryContainer">#633B48</color>
49	<color name="md_theme_dark_onTertiaryContainer">#FFD8E4</color>
50	<color name="md_theme_dark_error">#FFB4AB</color>

51	<code><color</code> name="md_theme_dark_onError">#690005</color>
52	<code><color</code> name="md_theme_dark_errorContainer">#93000A</color>
53	<code><color</code> name="md_theme_dark_onErrorContainer">#FFDAD6</color>
54	<code><color</code> name="md_theme_dark_background">#1C1B1F</color>
55	<code><color</code> name="md_theme_dark_onBackground">#E6E1E5</color>
56	<code><color</code> name="md_theme_dark_surface">#1C1B1F</color>
57	<code><color</code> name="md_theme_dark_onSurface">#E6E1E5</color>
58	<code><color</code> name="md_theme_dark_surfaceVariant">#49454F</color>
59	<code><color</code> name="md_theme_dark_onSurfaceVariant">#CCC4D4</color>
60	<code><color</code> name="md_theme_dark_outline">#948F99</color>
61	<code><color</code> name="md_theme_dark_shadow">#000000</color>
62	<code><color</code> name="md_theme_dark_inverseSurface">#E6E1E5</color>
63	<code><color</code> name="md_theme_dark_inverseOnSurface">#1C1B1F</color>
64	<code><color</code> name="md_theme_dark_inversePrimary">#6750A4</color>
65	<code><color</code> name="md_theme_dark_surfaceTint">#D0BCFF</color>
	<code></resources></code>

28. themes.xml (light)

Tabel 63Source Code Jawaban themes.xml Modul 5

1	<code><resources</code> xmlns:tools="http://schemas.android.com/tools">
2	<code><!-- Base application theme. --></code>
3	<code><style name="Base.Theme.ConnectAPI"</code> parent="Theme.Material3.DayNight.NoActionBar">
4	<code><!-- Customize your light theme here. --</code>

	>
5	<code> <item name="colorPrimary">@color/md_theme_light_primary</item> </code>
6	<code> <item name="colorOnPrimary">@color/md_theme_light_onPrimary</item> </code>
7	<code> <item name="colorPrimaryContainer">@color/md_theme_light_primaryContainer</item> </code>
8	<code> <item name="colorOnPrimaryContainer">@color/md_theme_light_onPrimaryContainer</item> </code>
9	<code> <item name="colorSecondary">@color/md_theme_light_secondary</item> </code>
10	<code> <item name="colorOnSecondary">@color/md_theme_light_onSecondary</item> </code>
11	<code> <item name="colorSecondaryContainer">@color/md_theme_light_secondaryContainer</item> </code>
12	<code> <item name="colorOnSecondaryContainer">@color/md_theme_light_onSecondaryContainer</item> </code>
13	<code> <item name="colorTertiary">@color/md_theme_light_tertiary</item> </code>
14	<code> <item name="colorOnTertiary">@color/md_theme_light_onTertiary</item> </code>
15	<code> <item name="colorTertiaryContainer">@color/md_theme_light_tertiaryContainer</item> </code>
16	<code> <item name="colorOnTertiaryContainer">@color/md_theme_light_onTertiaryContainer</item> </code>
17	<code> <item name="colorError">@color/md_theme_light_error</item> </code>
18	<code> <item name="colorOnError">@color/md_theme_light_onError</item> </code>
19	<code> <item name="colorErrorContainer">@color/md_theme_light_errorContainer</item> </code>
20	<code> <item name="colorOnErrorContainer">@color/md_theme_light_onErrorContainer</item> </code>

21	<code><item</code>
	<code>name="android:colorBackground">@color/md_theme_l</code>
	<code>ight_background</item></code>
22	<code><item</code>
	<code>name="colorOnBackground">@color/md_theme_light_o</code>
	<code>nBackground</item></code>
23	<code><item</code>
	<code>name="colorSurface">@color/md_theme_light_surfac</code>
	<code>e</item></code>
24	<code><item</code>
	<code>name="colorOnSurface">@color/md_theme_light_onSu</code>
	<code>rface</item></code>
25	<code><item</code>
	<code>name="colorSurfaceVariant">@color/md_theme_light</code>
	<code>_surfaceVariant</item></code>
26	<code><item</code>
	<code>name="colorOnSurfaceVariant">@color/md_theme_lig</code>
	<code>ht_onSurfaceVariant</item></code>
27	<code><item</code>
	<code>name="colorOutline">@color/md_theme_light_outlin</code>
	<code>e</item></code>
28	<code><item</code>
	<code>name="colorPrimaryInverse">@color/md_theme_light</code>
	<code>_inversePrimary</item></code>
29	<code><item name="android:statusBarColor"</code>
	<code>tools:targetApi="1">?attr/colorPrimary</item></code>
30	<code><item name="android:navigationBarColor"</code>
	<code>tools:targetApi="1">?attr/colorSurface</item></code>
31	<code></style></code>
32	
33	<code><style name="Theme.ConnectAPI"</code>
	<code>parent="Base.Theme.ConnectAPI" /></code>
34	<code></resources></code>

29. themes.xml (night)

Tabel 64 Source Code Jawaban themes.xml Modul 5

1	<code><resources</code>
	<code>xmlns:tools="http://schemas.android.com/tools"></code>
2	<code><!-- Base application theme. --></code>
3	<code><style name="Base.Theme.ConnectAPI"</code>
	<code>parent="Theme.Material3.DayNight.NoActionBar"></code>
4	<code><!-- Customize your dark theme here. --></code>
5	<code><item</code>
	<code>name="colorPrimary">@color/md_theme_dark_primary</code>
	<code></item></code>
6	<code><item</code>
	<code>name="colorOnPrimary">@color/md_theme_dark_onPri</code>
	<code>mary</item></code>

7	<item name="colorPrimaryContainer">@color/md_theme_dar k_primaryContainer</item>
8	<item name="colorOnPrimaryContainer">@color/md_theme_d ark_onPrimaryContainer</item>
9	<item name="colorSecondary">@color/md_theme_dark_secon dary</item>
10	<item name="colorOnSecondary">@color/md_theme_dark_onS econdary</item>
11	<item name="colorSecondaryContainer">@color/md_theme_d ark_secondaryContainer</item>
12	<item name="colorOnSecondaryContainer">@color/md_theme _dark_onSecondaryContainer</item>
13	<item name="colorTertiary">@color/md_theme_dark_tertia ry</item>
14	<item name="colorOnTertiary">@color/md_theme_dark_onTe rtiary</item>
15	<item name="colorTertiaryContainer">@color/md_theme_da rk_tertiaryContainer</item>
16	<item name="colorOnTertiaryContainer">@color/md_theme_ dark_onTertiaryContainer</item>
17	<item name="colorError">@color/md_theme_dark_error</it em>
18	<item name="colorOnError">@color/md_theme_dark_onError </item>
19	<item name="colorErrorContainer">@color/md_theme_dark_ errorContainer</item>
20	<item name="colorOnErrorContainer">@color/md_theme_dar k_onErrorContainer</item>
21	<item name="android:colorBackground">@color/md_theme_d ark_background</item>
22	<item name="colorOnBackground">@color/md_theme_dark_on Background</item>

23	<item
	name="colorSurface">@color/md_theme_dark_surface
	</item>
24	<item
	name="colorOnSurface">@color/md_theme_dark_onSur
	face</item>
25	<item
	name="colorSurfaceVariant">@color/md_theme_dark_
	surfaceVariant</item>
26	<item
	name="colorOnSurfaceVariant">@color/md_theme_dar
	k_onSurfaceVariant</item>
27	<item
	name="colorOutline">@color/md_theme_dark_outline
	</item>
28	<item
	name="colorPrimaryInverse">@color/md_theme_dark_
	inversePrimary</item>
29	<item name="android:statusBarColor"
	tools:targetApi="1">?attr/colorPrimary</item>
30	<item name="android:navigationBarColor"
	tools:targetApi="1">?attr/colorSurface</item>
31	</style>
32	</resources>

30. build.gradle.kts (module:app)

Tabel 65 Source Code Jawaban build.gradle.kts Modul 5

1	plugins {
2	alias(libs.plugins.android.application)
3	alias(libs.plugins.kotlin.android)
4	id("kotlin-parcelize")
5	id("org.jetbrains.kotlin.plugin.serialization")
	version "2.0.0"
6	id("kotlin-kapt")
7	}
8	
9	android {
10	namespace = "com.example.connectapi"
11	compileSdk = 35
12	
13	defaultConfig {
14	applicationId = "com.example.connectapi"
15	minSdk = 30
16	targetSdk = 35
17	versionCode = 1
18	versionName = "1.0"
19	testInstrumentationRunner =
	"androidx.test.runner.AndroidJUnitRunner"

```

20         buildConfigField("String",
    "TMDB_API_KEY",
    "\"c85206126e56c036e856104caa98a28a\"")
21     }
22
23     buildTypes {
24         release {
25             isMinifyEnabled = false
26             proguardFiles(
27 getDefaultValue("proguard-android-
optimize.txt"),
28                 "proguard-rules.pro"
29             )
30         }
31     }
32
33     compileOptions {
34         sourceCompatibility =
JavaVersion.VERSION_11
35         targetCompatibility =
JavaVersion.VERSION_11
36     }
37
38     kotlinOptions {
39         jvmTarget = "11"
40     }
41
42     buildFeatures {
43         viewBinding = true
44         buildConfig = true
45     }
46 }
47
48 dependencies {
49     implementation(libs.androidx.core.ktx)
50     implementation(libs.androidx.appcompat)
51     implementation(libs.material)
52     implementation(libs.androidx.activity)
53     implementation(libs.androidx.constraintlayout)
54     implementation("androidx.navigation:navigation-
fragment-ktx:2.7.7")
55     implementation("androidx.navigation:navigation-
ui-ktx:2.7.7")
56     implementation("io.coil-kt:coil:2.6.0")
57     testImplementation(libs.junit)
58     androidTestImplementation(libs.androidx.junit)
59     androidTestImplementation(libs.androidx.espresso
.core)
60 }

```

61	testImplementation("androidx.room:room- compiler-processing-testing:2.7.1")
62	implementation("androidx.room:room- runtime:2.6.1")
63	kapt("androidx.room:room-compiler:2.6.1") implementation("androidx.room:room- ktx:2.6.1")
64	implementation("com.squareup.retrofit2:retrofit: 2.9.0")
65	implementation("com.squareup.okhttp3:okhttp:4.12 .0")
66	implementation("com.squareup.okhttp3:logging- interceptor:4.12.0")
67	implementation("org.jetbrains.kotlinx:kotlinx- serialization-json:1.6.2")
68	implementation("com.jakewharton.retrofit:retrofi t2-kotlinx-serialization-converter:1.0.0")
69	implementation("org.jetbrains.kotlinx:kotlinx- coroutines-android:1.7.3")
70	implementation("androidx.lifecycle:lifecycle- viewmodel-ktx:2.7.0")
71	implementation("androidx.lifecycle:lifecycle- runtime-ktx:2.7.0")
72	implementation("androidx.lifecycle:lifecycle- livedata-ktx:2.7.0")
73	}

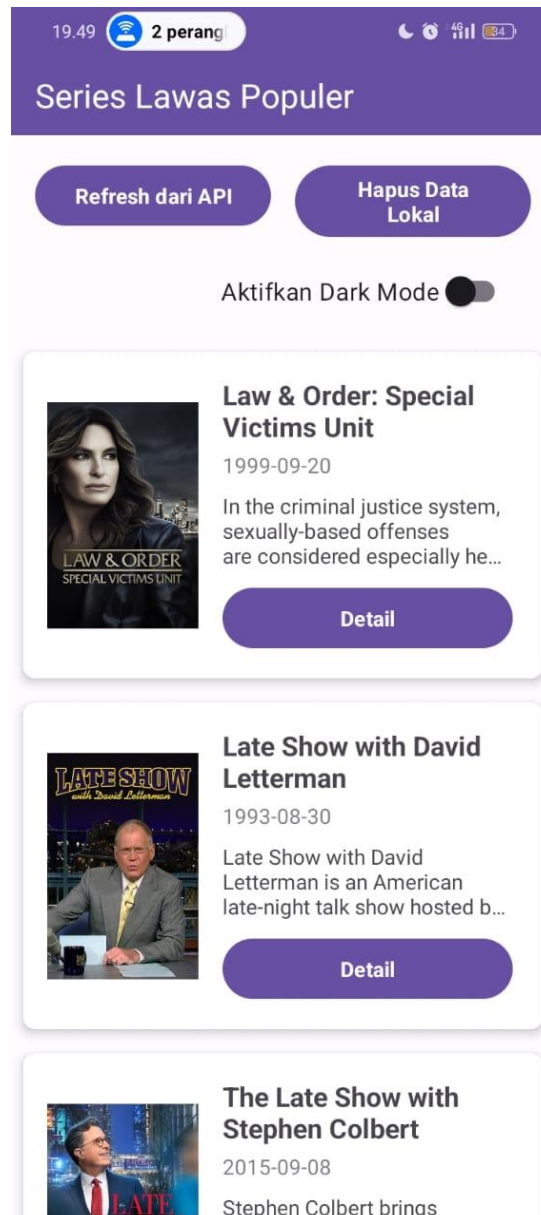
31. AndroidManifest.xml

Tabel 66 Source Code Jawaban AndroidManifest.xml Modul 5

1	<?xml version="1.0" encoding="utf-8"?>
2	<manifest xmlns:android="http://schemas.android.com/apk/re s/android"
3	xmlns:tools="http://schemas.android.com/tools">
4	
5	<uses-permission android:name="android.permission.INTERNET" />
6	<application android:allowBackup="true"
7	android:dataExtractionRules="@xml/data_extractio n_rules"
8	android:fullBackupContent="@xml/backup_rules"
9	android:icon="@mipmap/ic_launcher"
10	android:label="@string/app_name"
11	android:roundIcon="@mipmap/ic_launcher_round"
12	android:supportsRtl="true"
13	android:theme="@style/Theme.ConnectAPI"
14	tools:targetApi="31">
15	

```
16         <activity
17     android:name=".presentation.ui.MainActivity"
18         android:exported="true">
19         <intent-filter>
20             <action
21     android:name="android.intent.action.MAIN" />
22             <category
23     android:name="android.intent.category.LAUNCHER"
24     />
25         </intent-filter>
26     </activity>
27 </application>
28 </manifest>
```

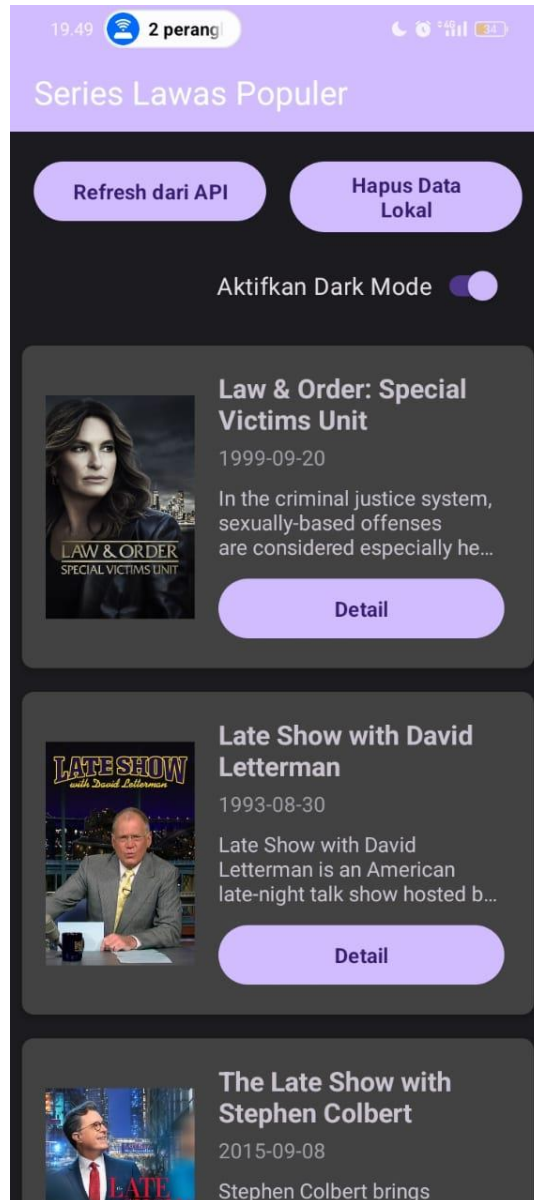
C. Output Program



Gambar 24 Screenshot Hasil Tampilan Halaman Utama



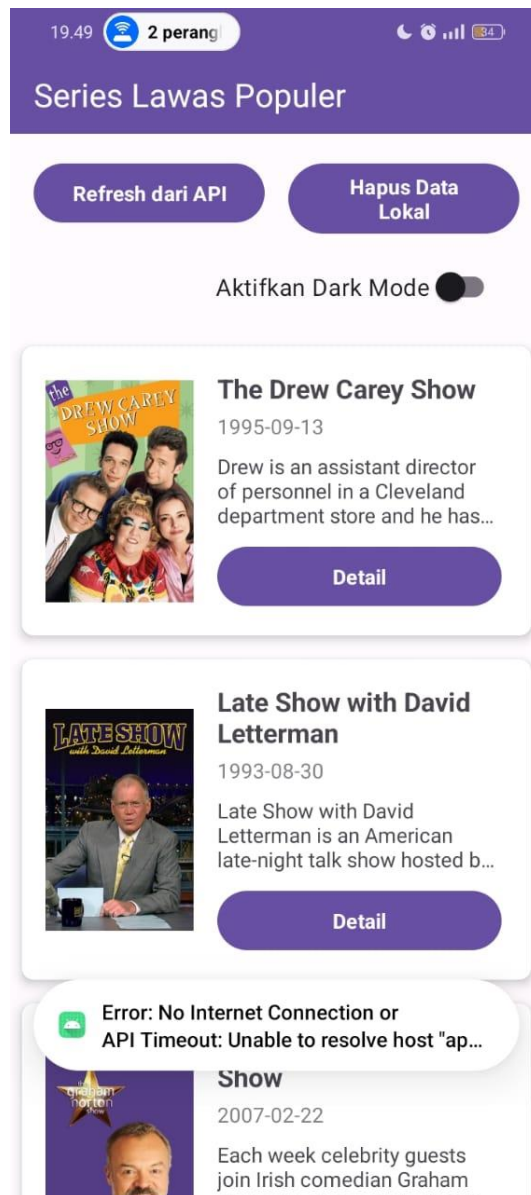
Gambar 25 Screenshot Hasil Tampilan Halaman Detail



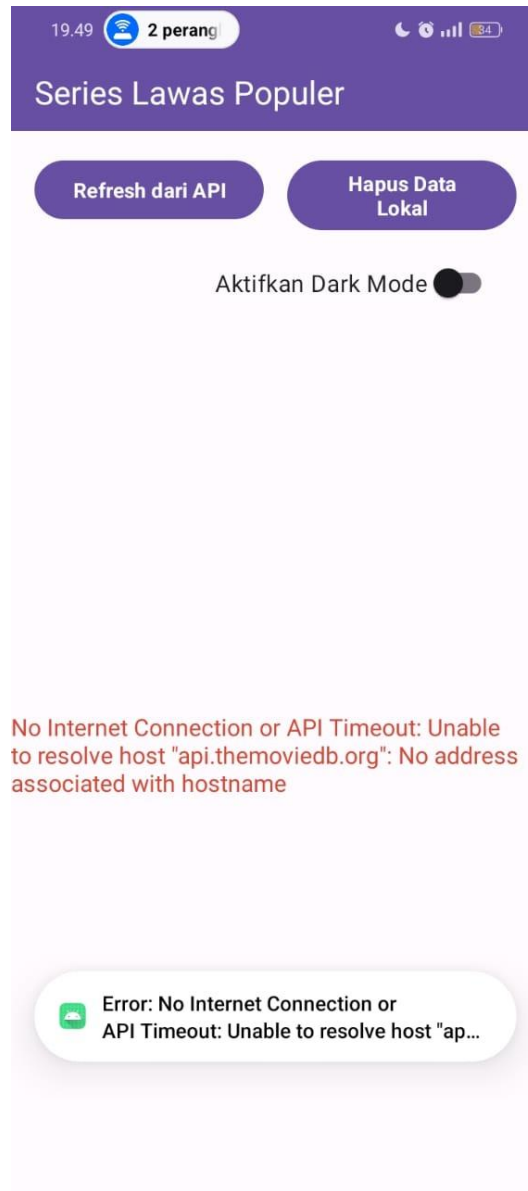
Gambar 26 Screenshot Hasil Tampilan Halaman Utama DarkMode



*Gambar 27 Screenshot Hasil Tampilan Halaman Detail
DarkMode*



Gambar 28 Screenshot Hasil Tampilan Halaman Utama Jika Offline



Gambar 29 Screenshot Hasil Tampilan Halaman Detail Jika Offline

D. Pembahasan

1. DramaDao.kt

Pada baris 1 package com.example.connectapi.data.local.dao menunjukkan bahwa file Kotlin ini berada di dalam paket dao di bawah domain com.example.connectapi.data.local. Baris 3-8 merupakan impor agar file ini bisa menggunakan kelas-kelas yang diperlukan dari Android Jetpack Room (seperti Dao, Insert, OnConflictStrategy, dan Query) serta Flow dari Kotlin Coroutines. Baris 11 mendefinisikan antarmuka (interface) bernama DramaDao yang akan digunakan sebagai Data Access Object (DAO) untuk berinteraksi dengan database Room. Baris 12-13 adalah fungsi insertDramas yang berfungsi untuk menyisipkan daftar DramaEntity ke dalam database. Anotasi @Insert menandai fungsi ini sebagai operasi penyisipan, dan onConflict = OnConflictStrategy.REPLACE berarti jika ada konflik (misalnya, data dengan primary key yang sama sudah ada), data yang lama akan diganti dengan yang baru. Baris 15-16 adalah fungsi getAllDramas yang berfungsi untuk mendapatkan semua data drama dari tabel dramas. Anotasi @Query("SELECT * FROM dramas") menunjukkan bahwa fungsi ini akan menjalankan kueri SQL untuk memilih semua kolom dari tabel dramas. Fungsi ini mengembalikan Flow<List<DramaEntity>>, yang berarti data akan dikirim secara asinkron dan aplikasi akan menerima pembaruan setiap kali data di tabel berubah. Baris 18-19 adalah fungsi deleteAllDramas yang berfungsi untuk menghapus semua data dari tabel dramas. Anotasi @Query("DELETE FROM dramas") menunjukkan bahwa fungsi ini akan menjalankan kueri SQL untuk menghapus semua baris dari tabel dramas.

2. DramaDatabase.kt

Pada baris 1 package com.example.connectapi.data.local.Database menunjukkan bahwa file Kotlin ini berada di dalam paket Database di

bawah domain `com.example.connectapi.data.local`. Baris 2-7 merupakan impor agar file ini bisa menggunakan kelas-kelas yang diperlukan dari Android (`Context`), Android Jetpack Room (`Database`, `Room`, `RoomDatabase`), serta DAO (`DramaDao`) dan entitas (`DramaEntity`) yang telah didefinisikan sebelumnya. Baris 9 mendefinisikan kelas abstrak bernama `DramaDatabase` yang merupakan database Room. Anotasi `@Database` mengonfigurasi database ini, menyebutkan `DramaEntity` sebagai entitas yang digunakan, versi database adalah 1, dan `exportSchema` diatur ke `false` (yang berarti skema database tidak akan diekspor). Baris 12 adalah fungsi abstrak `dramaDao()` yang mengembalikan instance dari `DramaDao`. Room akan secara otomatis menyediakan implementasi untuk fungsi ini. Baris 14-27 adalah bagian `companion object` yang berisi logika untuk membuat dan mengelola instance tunggal dari `DramaDatabase` (pola Singleton). Baris 15-16 mendeklarasikan variabel `INSTANCE` yang bersifat `@Volatile` untuk memastikan visibilitas di seluruh thread. Baris 18-26 adalah fungsi `getDatabase` yang menerima `Context` dan mengembalikan instance dari `DramaDatabase`. Jika `INSTANCE` sudah ada, itu akan langsung dikembalikan; jika tidak, blok `synchronized` akan membuat instance baru menggunakan `Room.databaseBuilder`, membangun database dengan nama `"drama_db"`, dan menyimpannya ke `INSTANCE` sebelum mengembalikannya.

3. DramaEntity.kt

Pada baris 1 package `com.example.connectapi.data.local.entities` menunjukkan bahwa file Kotlin ini berada di dalam paket `entities` di bawah domain `com.example.connectapi.data.local`. Baris 3-4 merupakan impor agar file ini bisa menggunakan anotasi `Entity` dan `PrimaryKey` dari Android Jetpack Room. Baris 6 adalah anotasi `@Entity(tableName = "dramas")` yang menandai kelas

`DramaEntity` sebagai entitas database Room. Ini berarti setiap instance dari `DramaEntity` akan sesuai dengan sebuah baris dalam tabel bernama "dramas" di database. Baris 7 mendefinisikan data class `DramaEntity`, sebuah kelas data Kotlin yang secara otomatis menyediakan fungsionalitas seperti `equals()`, `hashCode()`, dan `toString()`. Kelas ini merepresentasikan struktur data untuk setiap drama yang akan disimpan dalam database. Baris 8 mendefinisikan properti `id` dengan anotasi `@PrimaryKey`. Ini menandai `id` sebagai kunci utama (primary key) dari tabel "dramas", memastikan setiap entri memiliki identifikasi unik. Baris 9-12 mendefinisikan properti-properti lain dari entitas drama, yaitu `name`, `overview`, `posterPath`, dan `firstAirDate`. Properti `posterPath` dan `firstAirDate` ditandai dengan `?` yang berarti mereka bisa bernilai null.

4. DramaApiService.kt

Pada baris 1 `package com.example.connectapi.data.remote.api` menunjukkan bahwa file Kotlin ini berada di dalam paket `api` di bawah domain `com.example.connectapi.data.remote`. Baris 3-6 merupakan impor agar file ini bisa menggunakan kelas-kelas yang diperlukan dari Retrofit (`Response` dan `http.GET`, `http.Query`) serta model data `PopularDramasResponse` yang telah didefinisikan. Baris 8 mendefinisikan antarmuka (interface) bernama `DramaApiService`. Antarmuka ini digunakan oleh Retrofit untuk mendefinisikan *endpoint* API dan bagaimana aplikasi akan berinteraksi dengannya. Baris 10-14 adalah fungsi `getPopularDramas` yang merupakan metode *suspending function*, artinya bisa dipanggil dari *coroutine*. Fungsi ini ditandai dengan `@GET("tv/popular")`, yang menunjukkan bahwa ia akan melakukan permintaan HTTP GET ke *endpoint* "tv/popular" dari URL dasar API. Fungsi ini menerima tiga parameter: `apiKey` yang wajib (`@Query("api_key")`), `language` dengan nilai *default* "en-US"

(@Query("language")), dan page dengan nilai *default* 1 (@Query("page")). Semua parameter ini akan ditambahkan sebagai *query parameter* ke URL permintaan. Fungsi ini mengembalikan Response<PopularDramasResponse>, yang merupakan objek respons dari Retrofit yang berisi data drama populer.

5. DramaDto.kt

Pada baris 1 package com.example.connectapi.data.remote.model menunjukkan bahwa file Kotlin ini berada di dalam paket model di bawah domain com.example.connectapi.data.remote. Baris 3-4 merupakan impor agar file ini bisa menggunakan anotasi SerialName dan Serializable dari library KotlinX Serialization. Baris 6 adalah anotasi @Serializable yang menandai kelas DramaDto agar bisa di-*serialize* dan di-*deserialize* (diubah dari/ke format data seperti JSON). Baris 7 mendefinisikan data class DramaDto, sebuah kelas data Kotlin yang merepresentasikan objek data untuk drama yang diterima dari API. Baris 8-10 mendefinisikan properti id, name, dan overview yang akan langsung dipetakan dari data JSON. Baris 11-14 adalah properti posterPath dan firstAirDate yang menggunakan anotasi @SerializedName. Anotasi ini digunakan untuk memetakan nama properti di JSON ("poster_path" dan "first_air_date") ke nama properti di kelas Kotlin (posterPath dan firstAirDate) yang menggunakan *camelCase*. Kedua properti ini juga ditandai dengan ? yang berarti mereka bisa bernilai null.

6. PopularDramasResponse.kt

Pada baris 1 package com.example.connectapi.data.remote.model menunjukkan bahwa file Kotlin ini berada di dalam paket model di bawah domain com.example.connectapi.data.remote. Baris 2-3 merupakan

import agar file ini bisa menggunakan anotasi `SerializedName` dan `Serializable` dari library `KotlinX Serialization`. Baris 5 adalah anotasi `@Serializable` yang menandai kelas `PopularDramasResponse` agar bisa di-*serialize* dan di-*deserialize* (diubah dari/ke format data seperti JSON). Baris 6 mendefinisikan data class `PopularDramasResponse`, sebuah kelas data Kotlin yang merepresentasikan struktur respons lengkap dari API ketika meminta daftar drama populer. Baris 7-8 mendefinisikan properti `page` yang akan memetakan ke nama `page` dari JSON. Baris 9-10 mendefinisikan properti `results` yang akan memetakan ke nama `results` dari JSON, dan ini adalah daftar objek `DramaDto`. Baris 11-12 mendefinisikan properti `totalPages` yang memetakan ke `total_pages` dari JSON. Terakhir, baris 13-14 mendefinisikan properti `totalResults` yang memetakan ke `total_results` dari JSON. Semua properti ini menggunakan anotasi `@SerializedName` untuk memetakan nama properti di JSON ke nama properti di kelas Kotlin yang menggunakan *camelCase*.

7. `ApiClient.kt`

Pada baris 1 package `com.example.connectapi.data.remote` menunjukkan bahwa file Kotlin ini berada di dalam paket `remote` di bawah domain `com.example.connectapi.data.remote`. Baris 3-12 merupakan import agar file ini bisa menggunakan kelas-kelas seperti `Log` dari `Android`, `BuildConfig` (yang berisi variabel konfigurasi proyek), `DramaApiService` (antarmuka API yang telah didefinisikan), `asConverterFactory` dari `Retrofit KotlinX Serialization`, `Json` dari `KotlinX Serialization`, `MediaType` dan `OkHttpClient` dari `OkHttp`, `HttpLoggingInterceptor` dari `OkHttp Logging`, `Retrofit` dari `Retrofit`, dan `TimeUnit` dari `Java`. Baris 14 mendefinisikan objek `ApiClient`, yang berfungsi sebagai `Singleton` untuk menyediakan instance `Retrofit` dan `DramaApiService` yang telah dikonfigurasi. Baris 15-16 mendeklarasikan `BASE_URL` sebagai alamat dasar API dan `TAG`

untuk keperluan *logging*. Baris 18-23 menginisialisasi objek `Json` dari `KotlinX Serialization` dengan konfigurasi khusus seperti `ignoreUnknownKeys`, `prettyPrint`, `isLenient`, dan `coerceInputValues` untuk menangani parsing JSON secara fleksibel. Baris 25-29 mendefinisikan `okHttpClient` secara *lazy*. `OkHttpClient` ini dikonfigurasi dengan *timeout* 30 detik untuk koneksi, baca, dan tulis. Terdapat blok `try-catch` baris 31-43 untuk menambahkan `HttpLoggingInterceptor` yang berfungsi untuk *logging* permintaan dan respons HTTP, namun penanganan *error* disertakan jika *interceptor* ini tidak ditemukan atau terjadi kesalahan lain. Baris 45-69 mendefinisikan `dramaApiService` juga secara *lazy*. Di sini, kunci API (`TMDB_API_KEY`) diambil dari `BuildConfig`. Ada pemeriksaan untuk memastikan kunci API tidak kosong atau "null", dan jika ada masalah, `IllegalStateException` akan dilemparkan. Jika kunci API valid, `Retrofit` dibangun dengan `BASE_URL`, `okHttpClient` yang telah dikonfigurasi, dan `json.asConverterFactory` untuk menangani konversi JSON. Akhirnya, instance dari `DramaApiService` dibuat dan dikembalikan. Terdapat juga penanganan *error* yang komprehensif untuk menangani `IllegalStateException` dan `Exception` lainnya selama inisialisasi API.

8. DramaRepositoryImpl.kt

Pada baris 1 package `com.example.connectapi.data.repository` menunjukkan bahwa file Kotlin ini berada di dalam paket `repository` di bawah domain `com.example.connectapi.data..` Baris 3-19 merupakan impor agar file ini bisa menggunakan kelas-kelas seperti `Log` dari `Android`, `DramaDao` dan `DramaEntity` dari lokal database, `DramaApiService` dari remote API, `Drama` dari domain model, `DramaRepository` dari domain repository, `Resource` dari utilitas,

`toDomainModel` dan `toEntity` untuk pemetaan data, serta berbagai fungsi `Flow` dan `Dispatchers` dari Kotlin `Coroutines` dan `Retrofit`.

Baris 21 mendefinisikan kelas `DramaRepositoryImpl` yang mengimplementasikan antarmuka `DramaRepository`. Kelas ini bertanggung jawab untuk mengelola data drama, baik dari sumber lokal (database `Room`) maupun remote (API `The Movie Database`). Konstruktor kelas ini menerima `dramaDao` untuk akses database, `dramaApiService` untuk panggilan API, dan `apiKey` yang diperlukan untuk otentikasi API. Baris 27 adalah implementasi fungsi `getPopularDramas()` yang mengembalikan `Flow` dari `Resource<List<Drama>>`. Fungsi ini dimulai dengan memancarkan `Resource.Loading()`. Kemudian, mencoba mengambil data drama dari database lokal (cache). Jika data cache ditemukan dan tidak kosong, ia akan memancarkan `Resource.Success` dengan data cache tersebut dan mencatatnya. Jika cache kosong, ia akan mencatatnya juga. Selanjutnya, fungsi ini mencoba mengambil data drama terbaru dari API. Kemudian, ia memanggil `getPopularDramas` dari `dramaApiService`. Jika respons berhasil (`response.isSuccessful`), data DTO dari respons dikonversi ke model domain, kemudian database lokal dihapus dan diisi ulang dengan data terbaru dari API menggunakan `withContext(Dispatchers.IO)` untuk operasi I/O asinkron. Setelah itu, `Resource.Success` dipancarkan dengan data terbaru dan dicatat pada baris 52. Jika respons API tidak berhasil baris 58-70 `Resource.Error` akan dipancarkan dengan pesan kesalahan API, dan jika ada, data cache akan disertakan kembali. Terakhir, terdapat blok `catch` yang menangani berbagai jenis kesalahan seperti `HttpException` (kesalahan jaringan HTTP), `IOException` (masalah koneksi internet atau *timeout* API), dan `Exception` umum lainnya, di mana `Resource.Error` akan dipancarkan dengan pesan kesalahan yang relevan dan data cache jika tersedia. Baris 72-77 adalah fungsi `insertDramas` yang merupakan *suspending function* untuk menyisipkan

daftar Drama ke database lokal, mengubahnya menjadi `DramaEntity` dan mencatat keberhasilannya. Ini dilakukan di `Dispatchers.IO`. Baris 78-84 adalah fungsi `clearAllDramas` yang juga merupakan *suspending function* untuk menghapus semua data drama dari database lokal dan mencatatnya. Ini juga dijalankan di `Dispatchers.IO`.

9. DomainMappers.kt

Pada baris 1 `package com.example.connectapi.utils` menunjukkan bahwa file Kotlin ini berada di dalam paket `utils` di bawah domain `com.example.connectapi`. Baris 3-5 merupakan impor agar file ini bisa menggunakan kelas-kelas seperti `DramaEntity` dari entitas lokal, `DramaDto` dari model remote, dan `Drama` dari domain model.

Baris 7-15 mendefinisikan fungsi ekstensi `toDomainModel()` untuk objek `DramaDto`. Fungsi ini bertugas mengonversi objek `DramaDto` (yang merepresentasikan data dari API) menjadi objek `Drama` (model domain aplikasi), dengan memetakan properti-propertinya secara langsung. Baris 16-24 mendefinisikan fungsi ekstensi `toDomainModel()` kedua untuk objek `DramaEntity`. Fungsi ini bertugas mengonversi objek `DramaEntity` (yang merepresentasikan data dari database lokal) menjadi objek `Drama` (model domain aplikasi), juga dengan memetakan properti-propertinya secara langsung. Baris 25-33 mendefinisikan fungsi ekstensi `toEntity()` untuk objek `DramaDto`. Fungsi ini bertanggung jawab untuk mengonversi objek `DramaDto` menjadi objek `DramaEntity` agar data dari API dapat disimpan ke dalam database lokal. Baris 34-42 mendefinisikan fungsi ekstensi `toEntity()` terakhir untuk objek `Drama`. Fungsi ini bertugas mengonversi objek `Drama` (model domain aplikasi) menjadi objek `DramaEntity`, yang diperlukan ketika data dari domain akan disimpan atau diperbarui di database lokal.

10. Drama.kt

Pada baris 1 `package com.example.connectapi.domain.model` menunjukkan bahwa

file Kotlin ini berada di dalam paket `model` di bawah domain `com.example.connectapi.domain`. Baris 3-4 merupakan impor agar file ini bisa menggunakan antarmuka `Parcelable` dari Android OS dan anotasi `Parcelize` dari Kotlin `Parcelize`. Baris 6 adalah anotasi `@Parcelize` yang memungkinkan objek dari kelas `Drama` untuk dilewatkan antar komponen Android (seperti `Activity` atau `Fragment`) dengan mudah, tanpa perlu mengimplementasikan metode `Parcelable` secara manual. Baris 7 mendefinisikan data class `Drama`, sebuah kelas data Kotlin yang merepresentasikan model domain untuk sebuah drama. Ini adalah representasi bersih dari data drama yang digunakan di seluruh lapisan domain aplikasi. Baris 8-12 mendefinisikan properti-properti utama dari drama: `id` sebagai identifikasi unik, `name` sebagai judul, `overview` sebagai ringkasan cerita, `posterPath` sebagai jalur gambar poster (yang bisa bernilai `null`), dan `firstAirDate` sebagai tanggal tayang perdana (yang juga bisa bernilai `null`). Terakhir, `:` `Parcelable` pada baris 13 menunjukkan bahwa kelas `Drama` mengimplementasikan antarmuka `Parcelable`, memungkinkan objeknya untuk di-*serialize* dan di-*deserialize* dengan efisien.

11. DramaRepository.kt

Pada baris 1 `package com.example.connectapi.domain.repository` menunjukkan bahwa file Kotlin ini berada di dalam paket `repository` di bawah domain `com.example.connectapi.domain`. Baris 3-5 merupakan impor agar file ini bisa menggunakan kelas `Drama` dari model domain, kelas `Resource` dari utilitas, dan `Flow` dari Kotlin `Coroutines`.

Baris 7 mendefinisikan sebuah **antarmuka** (interface) bernama `DramaRepository`. Antarmuka ini berfungsi sebagai kontrak yang mendefinisikan operasi-operasi yang harus disediakan oleh repositori drama. Baris 8-9 adalah deklarasi fungsi `getPopularDramas()`. Fungsi ini mengembalikan `Flow<Resource<List<Drama>>>`, yang berarti ia

akan menyediakan *stream* data drama populer yang dibungkus dalam objek `Resource`. Objek `Resource` ini memungkinkan representasi berbagai status (seperti *loading*, sukses, atau *error*) selama pengambilan data, dan `Flow` memungkinkan pengamatan perubahan data secara asinkron.

12. GetDramaUseCase

Pada baris 1 package `com.example.connectapi.domain.usecase` menunjukkan bahwa file Kotlin ini berada di dalam paket `usecase` di bawah domain `com.example.connectapi.domain`. Baris 3-5 merupakan impor agar file ini bisa menggunakan kelas `Drama` dari model domain, `DramaRepository` dari repository domain, dan `Resource` dan `Flow` dari Kotlin `Coroutines`. Baris 8 mendefinisikan kelas `GetDramaUseCase`. Kelas ini adalah *use case* atau *interactor* yang berfungsi sebagai jembatan antara lapisan UI/ViewModel dan lapisan repository. Kelas ini menerima `dramaRepository` sebagai dependensinya melalui konstruktor. Baris 9-10 mendefinisikan fungsi `invoke()` dengan *operator* `fun invoke()`. Ini memungkinkan objek `GetDramaUseCase` untuk dipanggil seperti sebuah fungsi (misalnya, `getDramaUseCase()`). Fungsi ini mengembalikan `Flow<Resource<List<Drama>>>`, yang diperoleh langsung dari `dramaRepository.getPopularDramas()`. Dengan begitu, `GetDramaUseCase` hanya meneruskan permintaan pengambilan drama populer dari repository, menjadikannya sebuah *use case* yang bersih dan tunggal dalam fungsinya.

13. DetailFragment.kt

Pada baris 1 package `com.example.connectapi.presentation.ui` menunjukkan bahwa file Kotlin ini berada di dalam paket `ui` di bawah domain `com.example.connectapi.presentation`. Baris 3-10

merupakan impor agar file ini bisa menggunakan kelas-kelas seperti Bundle, LayoutInflater, View, ViewGroup dari Android, Fragment dari AndroidX, load dari Coil, FragmentDetailBinding (binding class yang dihasilkan dari View Binding), dan Drama dari model domain. Baris 12 mendefinisikan kelas DetailFragment yang merupakan turunan dari Fragment, berfungsi untuk menampilkan detail sebuah drama. Baris 13-14 mendeklarasikan properti `_binding` sebagai variabel pembantu untuk menyimpan instance dari FragmentDetailBinding yang bisa bernilai null, serta properti `binding` sebagai *getter* yang tidak-null untuk memudahkan akses ke binding. Baris 16-22 adalah *override* dari fungsi `onCreateView()`. Fungsi ini bertanggung jawab untuk meng-inflasi (membuat) tampilan antarmuka pengguna fragment. Di sini, `FragmentDetailBinding.inflate()` digunakan untuk meng-inflasi layout dan mengikat tampilan ke objek binding, yang kemudian disimpan ke `_binding`, dan akar tampilan dari binding (`binding.root`) dikembalikan. Baris 23-35 adalah *override* dari fungsi `onViewCreated()`. Fungsi ini dipanggil setelah `onCreateView()` selesai dan tampilan fragment telah dibuat dan mencoba mengambil objek Drama yang dikirimkan sebagai argumen ke fragment dengan kunci "drama". Jika objek drama ditemukan dan tidak null properti-properti drama seperti `name`, `overview`, dan `firstAirDate` akan diatur ke TextView yang sesuai. `posterPath` digunakan bersama library Coil (`coil.load`) untuk memuat gambar poster dari URL dan menampilkannya di ImageView dengan efek *crossfade*. Terakhir, baris 33-36 adalah *override* dari fungsi `onDestroyView()`. Fungsi ini dipanggil ketika tampilan fragment akan dihancurkan. Pada baris 36, `_binding` diatur kembali menjadi null untuk membantu menghindari *memory leak* karena referensi ke tampilan tidak lagi dibutuhkan.

14. DramaAdapter.kt

Pada baris 1 package com.example.connectapi.presentation.ui menunjukkan bahwa file Kotlin ini berada di dalam paket ui di bawah com.example.connectapi.presentation. Baris 3-10 mengimpor kelas-kelas penting seperti LayoutInflater, ViewGroup, DiffUtil, ListAdapter, RecyclerView, coil.load untuk memuat gambar, ItemDramaBinding yang dihasilkan dari View Binding, serta Drama sebagai model data. Baris 12 mendefinisikan kelas DramaAdapter yang berfungsi sebagai jembatan antara data drama dan RecyclerView untuk menampilkannya. Adapter ini mewarisi ListAdapter yang secara efisien mengelola daftar data dan menggunakan DramaDiffCallback untuk mengoptimalkan pembaruan daftar. Baris 14-15 mendeklarasikan dua *callback* opsional, onDetailClickListener dan onNetflixClickListener, yang memungkinkan aksi tertentu dijalankan ketika tombol detail atau Netflix diklik untuk sebuah drama. Baris 17-20 mengimplementasikan onCreateViewHolder, yang bertanggung jawab untuk membuat dan mengembalikan sebuah DramaViewHolder baru dengan meng-inflasi tata letak ItemDramaBinding untuk setiap item daftar. Baris 22-25 mengimplementasikan onBindViewHolder, yang bertugas mengikat data Drama tertentu ke tampilan di dalam DramaViewHolder pada posisi yang diberikan. Baris 27-42 mendefinisikan kelas internal DramaViewHolder. Kelas ini memegang referensi ke tampilan setiap item di RecyclerView. Fungsi bind mengatur teks untuk tvTitle, tvOverview, dan tvReleaseDate dari objek Drama. Selain itu, ia menggunakan Coil untuk memuat gambar poster ke ivPoster dan mengatur OnClickListener pada btnDetail untuk memicu onDetailClickListener ketika tombol diklik. Terakhir, baris 43 mendefinisikan DramaDiffCallback, sebuah kelas utilitas yang membantu ListAdapter menentukan perbedaan antara dua daftar

drama. Baris 44-46 fungsi `areItemsTheSame`, membandingkan `id` dari dua objek `Drama` untuk mengetahui apakah mereka merujuk pada item yang sama. Baris 4852, fungsi `areContentsTheSame`, membandingkan seluruh isi objek `Drama` untuk mengetahui apakah ada perubahan pada data item tersebut.

15. `DramaListFragment.kt`

Pada baris 1 `package com.example.connectapi.presentation.ui` menunjukkan bahwa file Kotlin ini berada di dalam paket `ui` di bawah `com.example.connectapi.presentation`. Baris 3-26 mengimpor berbagai kelas dan fungsi penting, termasuk komponen Android UI (`Bundle`, `Log`, `LayoutInflater`, `View`, `ViewGroup`, `Toast`), komponen `Fragment` (`Fragment`, `viewModels`), manajemen siklus hidup (`Lifecycle`, `lifecycleScope`, `repeatOnLifecycle`), navigasi (`findNavController`), `RecyclerView` (`LinearLayoutManager`), konfigurasi aplikasi (`BuildConfig`), sumber daya (`R`), komponen database (`DramaDatabase`), klien API (`ApiClient`), implementasi repositori (`DramaRepositoryImpl`), *use case* (`GetDramaUseCase`), kelas utilitas (`Resource`, `AppPreferences`), dan tema (`AppCompatDelegate`), serta `launch` untuk *coroutines*. Baris 28 mendefinisikan kelas `DramaListFragment` sebagai turunan dari `Fragment`, yang bertanggung jawab untuk menampilkan daftar drama. Baris 29-30 mendeklarasikan variabel `_binding` untuk View Binding dan `binding` sebagai *getter* yang aman. Baris 32-43 menginisialisasi `dramaViewModel` menggunakan *delegate* `viewModels` dan `DramaViewModelFactory` khusus, di mana dependensi seperti `DramaDao`, `DramaApiService`, `apiKey`, `DramaRepositoryImpl`, dan `GetDramaUseCase` disediakan. Baris 38-39 mendeklarasikan `dramaAdapter` untuk `RecyclerView` dan `appPreferences` untuk pengelolaan preferensi aplikasi. Baris 45-51

mengimplementasikan `onCreateView`, di mana `View Binding` (`FragmentDramaListBinding.inflate`) digunakan untuk menginflasi tata letak fragment dan mengembalikan tampilan utamanya. Baris 52-75 mengimplementasikan `onViewCreated`, yang dipanggil setelah tampilan fragment dibuat. Di sini, `appPreferences` diinisialisasi, `setupRecyclerView()` dan `observeViewModel()` dipanggil untuk menyiapkan daftar dan mengamati data. Selain itu, status *dark mode* dimuat dari preferensi dan diterapkan ke `darkModeSwitch`, dengan *listener* yang menyimpan status *dark mode* yang diubah dan menerapkan tema. Terdapat juga *listener* untuk tombol *refresh* dan *clear local data*. Baris 76-91 mendefinisikan `setupRecyclerView()` yang menginisialisasi `dramaAdapter`, mengatur `LinearLayoutManager` untuk `rvDrama`, dan menetapkan adapter tersebut. Fungsi ini juga mengatur `onDetailClickListener` untuk menavigasi ke `DetailFragment` dengan data drama yang dipilih, serta `onNetflixClickListener` yang menampilkan `Toast`. Baris 92-136 mendefinisikan `observeViewModel()`, yang menggunakan `viewLifecycleOwner.lifecycleScope.launch` dan `repeatOnLifecycle(Lifecycle.State.STARTED)` untuk mengumpulkan data dari `dramaViewModel.popularDramasState`. Blok `when` kemudian menangani tiga status `Resource`: `Loading` (menampilkan `progressBar` atau data `cache`), `Success` (menyembunyikan *loading* dan menampilkan data), dan `Error` (menyembunyikan *loading*, menampilkan pesan error, `Toast`, dan jika ada, menampilkan data `cache`).

Baris 137-147 mendefinisikan `clearLocalData()`, sebuah fungsi `suspend` yang dipanggil ketika tombol `clear data lokal` ditekan. Fungsi ini mencoba memanggil `dramaViewModel.clearLocalData()` untuk menghapus data, menampilkan `Toast` keberhasilan atau kegagalan, dan mencatat *error*. Baris 148-155 mendefinisikan `applyTheme()`, yang mengubah tema aplikasi menjadi *dark mode*.

(AppCompatDelegate.MODE_NIGHT_YES) atau *light mode* (AppCompatDelegate.MODE_NIGHT_NO) berdasarkan nilai *isDarkMode*, dan kemudian memanggil `requireActivity().recreate()` untuk menerapkan perubahan tema secara instan. Terakhir, baris 156-160 mengimplementasikan `onDestroyView()` untuk membersihkan referensi `_binding` menjadi `null` guna menghindari *memory leak* ketika tampilan fragment dihancurkan.

16. DramaViewModel.kt

Pada baris 1 `package com.example.connectapi.presentation.ui` menunjukkan bahwa file Kotlin ini berada di dalam paket `ui` di bawah `com.example.connectapi.presentation`. Baris 3-12 mengimpor kelas-kelas penting seperti `ViewModel` dan `viewModelScope` dari `AndroidX Lifecycle`, `Drama` dari model domain, `DramaRepository` dari repositori domain, `GetDramaUseCase` dari *use case* domain, `Resource` dari utilitas, serta `MutableStateFlow`, `StateFlow`, `asStateFlow`, dan `launch` dari `Kotlin Coroutines`. Baris 14 mendefinisikan kelas `DramaViewModel` yang mewarisi `ViewModel`, bertugas mengelola dan menyediakan data drama ke UI dengan siklus hidup yang sadar akan Android. Konstruktornya menerima `getDramaUseCase` dan `dramaRepository` sebagai dependensi. Baris 19-20 mendeklarasikan `_popularDramasState` sebagai `MutableStateFlow` yang menyimpan status data drama populer, diinisialisasi dengan `Resource.Loading()`. Kemudian, `popularDramasState` dideklarasikan sebagai `StateFlow` publik yang hanya-baca, diekspos sebagai `_popularDramasState.asStateFlow()`. Baris 22-24 adalah blok `init` yang akan dijalankan saat `DramaViewModel` pertama kali dibuat, memanggil `fetchPopularDramas()` untuk segera memuat data drama.

Fungsi `fetchPopularDramas()` menggunakan `viewModelScope.launch` untuk meluncurkan *coroutine* dan mengumpulkan data dari `getDramaUseCase()`, yang kemudian membarui nilai `_popularDramasState` setiap kali ada perubahan. Baris 32-34 mendefinisikan fungsi `refreshDramasFromLocal()`, yang cukup memanggil kembali `fetchPopularDramas()` untuk memicu pengambilan ulang data (termasuk dari lokal) dan memperbarui UI. Baris 36-42 mendefinisikan fungsi `clearLocalData()`. Fungsi ini juga meluncurkan *coroutine* menggunakan `viewModelScope.launch`. Di dalamnya, ia mencoba meng-*casting* `dramaRepository` ke `DramaRepositoryImpl` dan jika berhasil, memanggil `clearAllDramas()` untuk menghapus data lokal. Setelah itu, `fetchPopularDramas()` dipanggil kembali untuk memperbarui tampilan dengan data yang mungkin kosong atau data baru dari API jika ada.

17. DramaViewModelFactory.kt

Pada baris 1 `package com.example.connectapi.presentation.ui` menunjukkan bahwa file Kotlin ini berada di dalam paket `ui` di bawah `com.example.connectapi.presentation`. Baris 3-10 mengimpor kelas-kelas seperti `Application` dari `Android`, `ViewModel` dan `ViewModelProvider` dari `AndroidX Lifecycle`, `DramaDatabase` dari lokal database, `ApiClient` dari remote API, `DramaRepositoryImpl` dari implementasi repositori, `GetDramaUseCase` dari *use case* domain, dan `BuildConfig` dari konfigurasi proyek. Baris 12 mendefinisikan kelas `DramaViewModelFactory` yang mengimplementasikan `ViewModelProvider.Factory`. Kelas ini berfungsi sebagai *pabrik* untuk membuat instance `DramaViewModel` dengan dependensi yang diperlukan. Konstruktornya menerima `application` dan

`getPopularDramaUseCase` sebagai dependensi awal. Baris 15 adalah implementasi fungsi `create()`, yang wajib di-*override* dari `ViewModelProvider.Factory`. Fungsi ini bertanggung jawab untuk membuat dan mengembalikan instance `ViewModel` yang diminta. Pada baris 17, ia memeriksa apakah `modelClass` yang diminta adalah `DramaViewModel`. Jika sesuai, pada baris 19-23, secara manual menginisialisasi semua dependensi yang dibutuhkan oleh `DramaViewModel`: `dramaDao` dari `DramaDatabase`, `dramaApiService` dari `ApiClient`, `apiKey` dari `BuildConfig`, `dramaRepository` dengan dependensi yang telah dibuat, dan `getDramaUseCase` dengan `dramaRepository`. Setelah semua dependensi siap, pada baris 25, membuat dan mengembalikan instance `DramaViewModel` dengan `getDramaUseCase` dan `dramaRepository` yang telah disiapkan, dengan *annotation* `@SuppressWarnings("UNCHECKED_CAST")` untuk menghindari peringatan *casting*. Jika `modelClass` yang diminta bukan `DramaViewModel`, pada baris 28, ia akan melempar `IllegalArgumentException`.

18. MainActivity.kt

Pada baris 1 `package com.example.connectapi.presentation.ui` menunjukkan bahwa file Kotlin ini berada di dalam paket `ui` di bawah `com.example.connectapi.presentation`. Baris 3-9 mengimpor kelas-kelas penting seperti `Bundle` dari Android OS, `AppCompatActivity` dan `AppCompatActivity` dari `AndroidX AppCompat`, `NavHostFragment` dan `setupActionBarWithNavController` dari `AndroidX Navigation`, `ActivityMainBinding` yang dihasilkan dari `View Binding`, dan `AppPreferences` dari utilitas. Baris 11 mendefinisikan kelas `MainActivity` yang merupakan *activity* utama aplikasi ini dan mewarisi `AppCompatActivity`. Baris 12-13 mendeklarasikan `binding` untuk

View Binding dan `appPreferences` untuk mengelola preferensi pengguna. Baris 15-29 mengimplementasikan fungsi `onCreate()`, yang dijalankan saat *activity* pertama kali dibuat. Pada baris 17-18, `ActivityMainBinding.inflate()` digunakan untuk meng-inflasi *layout* `activity_main` dan mengaturnya sebagai tampilan konten menggunakan `setContentView()`. Baris 20 menginisialisasi `appPreferences`. Baris 22 memanggil `applySavedTheme()` untuk menerapkan tema yang terakhir disimpan. Baris 24 menyiapkan bilah aksi (toolbar) dengan mengaturnya sebagai *support action bar*, menemukan `NavHostFragment`, dan mengikat `navController` ke bilah aksi untuk integrasi navigasi. Baris 31-36 mengimplementasikan `onSupportNavigateUp()`, sebuah fungsi yang dipanggil ketika tombol 'Up' di bilah aksi ditekan. Fungsi ini menangani navigasi kembali dalam tumpukan navigasi menggunakan `navController.navigateUp()`. Baris 37-44 mendefinisikan fungsi `applySavedTheme()`. Fungsi ini mengambil status *dark mode* yang disimpan dari `appPreferences` dan kemudian menggunakan `AppCompatActivity.setDefaultNightMode()` untuk mengatur tema aplikasi ke mode terang atau gelap sesuai dengan status yang disimpan.

19. AppPreferences.kt

Pada baris 1 `package com.example.connectapi.utils` menunjukkan bahwa file Kotlin ini berada di dalam paket `utils` di bawah `com.example.connectapi`. Baris 3-4 mengimpor kelas-kelas penting seperti `Context` dan `SharedPreferences` dari `Android`.

Baris 6 mendefinisikan kelas `AppPreferences`, yang dirancang untuk mengelola preferensi aplikasi, khususnya untuk menyimpan status *dark mode*. Konstruktor kelas ini menerima objek `Context` yang diperlukan untuk mengakses `SharedPreferences`. Baris 7-9 mendeklarasikan konstanta `PREFS_NAME` sebagai nama file preferensi ("`app_prefs`"), `KEY_DARK_MODE` sebagai kunci untuk menyimpan status *dark mode*

("dark_mode"), serta properti `sharedPrefs` untuk menyimpan instance dari `SharedPreferences`. Baris 11-13 adalah blok `init`, yang dijalankan saat objek `AppPreferences` dibuat. Di sini, `sharedPrefs` diinisialisasi dengan memanggil `context.getSharedPreferences()`, menggunakan `PREFS_NAME` dan `Context.MODE_PRIVATE` untuk memastikan preferensi ini hanya dapat diakses oleh aplikasi ini. Baris 14-16 mendefinisikan fungsi `saveDarkModeState()`. Fungsi ini menerima nilai *boolean* `isDarkMode` dan menggunakannya untuk menyimpan status *dark mode* ke `SharedPreferences` di bawah `KEY_DARK_MODE`. Metode `.apply()` digunakan untuk menyimpan perubahan secara asinkron. Terakhir, baris 17-19 mendefinisikan fungsi `getDarkModeState()`. Fungsi ini mengambil nilai *boolean* yang terkait dengan `KEY_DARK_MODE` dari `SharedPreferences`. Jika kunci tersebut belum ada, ia akan mengembalikan nilai *default* `false`.

20. Resource.kt

Pada baris 1 `package com.example.connectapi.utils` menunjukkan bahwa file Kotlin ini berada di dalam paket `utils` di bawah `com.example.connectapi`. Baris 3 mendefinisikan kelas `Resource` sebagai *sealed class*. *Sealed class* adalah jenis kelas abstrak yang memungkinkan mendefinisikan himpunan subclass terbatas dalam file yang sama, memastikan semua kemungkinan kasus ditangani. Kelas ini memiliki dua properti opsional: `data` (generik `T`, yang bisa berupa tipe data apa pun) dan `message` (`String`), yang akan digunakan oleh subclass-nya.

Baris 4 mendefinisikan `Loading` sebagai subclass dari `Resource`. Kelas ini merepresentasikan status di mana data sedang dimuat. Ia memiliki konstruktor yang menerima data opsional, yang bisa digunakan untuk menampilkan data lama saat data baru sedang dimuat. Baris 5 mendefinisikan `Success` sebagai subclass dari `Resource`. Kelas ini merepresentasikan status di mana data telah berhasil diambil. Ia memiliki

konstruktor yang wajib menerima data (T) yang berhasil dimuat. Baris 6 mendefinisikan `Error` sebagai subclass dari `Resource`. Kelas ini merepresentasikan status di mana terjadi kesalahan saat mengambil data. Ia memiliki konstruktor yang wajib menerima `message` (String) yang menjelaskan kesalahan, dan juga bisa menerima data opsional untuk menampilkan data lama jika ada kesalahan.

21. Activity_main.xml

Pada baris 1 `<?xml version="1.0" encoding="utf-8"?>` adalah deklarasi XML standar. Baris 2 `ConstraintLayout` adalah elemen tata letak akar, yang memungkinkan menempatkan dan mengukur tampilan berdasarkan hubungan relatif antar tampilan. Baris 2-5 mendefinisikan *namespace* XML (`xmlns:android`, `xmlns:app`, `xmlns:tools`) yang digunakan untuk atribut-atribut Android, atribut kustom dari pustaka (`app`), dan atribut khusus *tools* yang hanya untuk pratinjau desain. Atribut `android:layout_width` dan `android:layout_height` disetel ke `match_parent` agar tata letak memenuhi lebar dan tinggi layar penuh, sementara `tools:context` pada baris 6 menunjukkan kelas `MainActivity` sebagai konteks desain untuk pratinjau. Pada baris 9, `<com.google.android.material.appbar.AppBarLayout>` adalah kontainer yang dirancang untuk bilah aplikasi (toolbar) Material Design, memungkinkan efek gulir seperti menyembunyikan atau menampilkan toolbar saat pengguna menggulir. Atribut `android:layout_width` diatur ke `match_parent` agar memenuhi lebar, `android:layout_height` ke `wrap_content` agar tingginya menyesuaikan kontennya, dan `app:layout_constraintTop_toTopOf="parent"` pada baris 12 menempatkannya di bagian atas layar. Di dalam `AppBarLayout`, pada baris 15, `<androidx.appcompat.widget.Toolbar>` adalah bilah alat aplikasi yang berfungsi sebagai *action bar*. Atribut `android:id`

memberinya pengenalan toolbar, `android:layout_width` diatur ke `match_parent` dan `android:layout_height` ke `53dp` untuk dimensinya. `android:background="?attr/colorPrimary"` mengaturnya agar menggunakan warna primer dari tema aplikasi, dan `app:titleTextColor="@android:color/white"` mengatur warna teks judul menjadi putih.

Selanjutnya, pada baris 22, `<androidx.fragment.app.FragmentContainerView>` adalah kontainer yang digunakan untuk menghosting `Fragment` navigasi. Atribut `android:id` memberinya pengenalan `nav_host_fragment`, dan `android:name="androidx.navigation.fragment.NavHostFragment"` menentukan bahwa ini adalah *host* untuk grafik navigasi. Atribut `android:layout_width` dan `android:layout_height` disetel ke `0dp` dengan batasan `(app:layout_constraintBottom_toBottomOf,`
`app:layout_constraintEnd_toEndOf,`
`app:layout_constraintStart_toStartOf)` untuk mengisi sisa ruang setelah `app_bar_layout`. Atribut `app:defaultNavHost="true"` pada baris 24 menandai ini sebagai `NavHost` default yang akan mencegat tombol kembali sistem, dan `app:navGraph="@navigation/nav_graph"` pada baris 27 menghubungkannya ke grafik navigasi yang didefinisikan dalam `res/navigation/nav_graph.xml`.

22. fragment_detail.xml

Pada baris 1 `<?xml version="1.0" encoding="utf-8"?>` merupakan deklarasi XML standar. Baris 2 `<androidx.constraintlayout.widget.ConstraintLayout>` adalah elemen tata letak utama yang berfungsi sebagai wadah untuk tampilan-tampilan lain dalam fragmen, memungkinkannya mengatur posisi elemen berdasarkan hubungan antar elemen. Baris 2-5 mendefinisikan *namespace* XML yang diperlukan untuk atribut-atribut Android, atribut

kustom (app), dan atribut khusus alat (tools) untuk pratinjau desain. Atribut `android:layout_width` dan `android:layout_height` pada baris 6-7 disetel ke `match_parent` agar tata letak memenuhi seluruh lebar dan tinggi yang tersedia, sedangkan `tools:context` pada baris 8 menunjukkan bahwa *layout* ini terkait dengan kelas `DetailFragment` untuk keperluan *preview* di *editor*.

Pada baris 10 `<ImageView>` dengan `android:id="@+id/iv_detail_poster"` didefinisikan untuk menampilkan gambar poster drama. Gambar ini akan memenuhi lebar (`match_parent`) dan memiliki tinggi tetap `250dp`, dengan `scaleType="centerCrop"` untuk memastikan gambar mengisi area tampilan sambil mempertahankan rasio aspeknya. `android:src` pada baris 15 menyediakan gambar *placeholder* awal, dan batasan `app:layout_constraintTop_toTopOf="parent"`, `app:layout_constraintStart_toStartOf="parent"`, serta `app:layout_constraintEnd_toEndOf="parent"` pada baris 16-18 menempatkan gambar ini di bagian paling atas tata letak dan membentang dari kiri ke kanan. Selanjutnya, pada baris 20, `<TextView>` dengan `android:id="@+id/tv_detail_title"` digunakan untuk menampilkan judul drama. Lebar nya diatur ke `0dp` dengan batasan horizontal untuk mengisi ruang yang tersedia, tingginya `wrap_content`, dan memiliki margin `16dp` di bagian atas, kiri, dan kanan. Teks *placeholder* "Judul Drama" disetel dengan ukuran `24sp` dan gaya **bold**. Batasan pada baris 30-32 menempatkan `TextView` ini tepat di bawah `iv_detail_poster` dan membentang nya secara horizontal.

Pada baris 35 `<TextView>` berikutnya dengan `android:id="@+id/tv_detail_release_date"` berfungsi untuk menampilkan tanggal rilis drama. Sama seperti judul, lebar nya `0dp`, tingginya `wrap_content`, dengan margin `8dp` di bagian atas dan `16dp` di kiri/kanan. Teks *placeholder* "Tanggal Rilis:YYYY-MM-DD" disetel dengan ukuran `16sp` dan `alpha="0.7"` untuk memberikan efek sedikit

transparan. Batasan pada baris 44-46 menempatkannya di bawah `tv_detail_title` dan membentangnya secara horizontal.

Terakhir, dari baris 48-62 `<TextView>` dengan `android:id="@+id/tv_detail_overview"` dirancang untuk menampilkan sinopsis atau deskripsi drama. Lebar dan tingginya disetel ke `0dp` dengan batasan, yang berarti ia akan mengisi sisa ruang yang tersedia di tata letak setelah elemen-elemen di atasnya. Memiliki margin `16dp` di semua sisi, teks *placeholder* "Deskripsi panjang drama...", ukuran `14sp`, dan `android:scrollbars="vertical"` untuk memungkinkan teks digulir jika terlalu panjang. Batasan pada baris 49-52 menempatkannya di bawah `tv_detail_release_date` dan membentangnya hingga ke bagian bawah tata letak serta dari kiri ke kanan.

23. `fragment_drama_list.xml`

Pada baris 1 `<androidx.constraintlayout.widget.ConstraintLayout>` adalah elemen tata letak utama yang membungkus seluruh tampilan fragment, memungkinkan penempatan elemen berdasarkan hubungan relatif. Atribut `android:layout_width` dan `android:layout_height` pada baris 4-5 diatur ke `match_parent` agar tata letak memenuhi seluruh ruang yang tersedia, sedangkan `tools:context` pada baris 6 menunjukkan `DramaListFragment` sebagai konteks untuk pratinjau desain.

Pada baris 9, sebuah `Button` dengan `android:id="@+id/btn_refresh_api"` didefinisikan untuk memicu *refresh* data dari API. Lebarnya disetel ke `0dp` untuk mengisi ruang yang dibatasi oleh `guideline_center`, tingginya `wrap_content`, dan memiliki teks "Refresh dari API". Batasan pada baris 16-19 menempatkannya di bagian atas layar dan di sisi kiri `guideline_center` dengan margin tertentu. Di sampingnya, pada baris 21 `Button` kedua dengan `android:id="@+id/btn_clear_local_data"` berfungsi untuk

menghapus data lokal. Lebar nya juga 0dp, tingginya wrap_content, dan teksnya "Hapus Data Lokal". Batasan pada baris 27 menempatkannya di samping kanan guideline_center sejajar dengan tombol *refresh*.

Pada baris 32

<androidx.constraintlayout.widget.Guideline> dengan android:id="@+id/guideline_center" adalah elemen pembantu tak terlihat yang membagi tata letak secara vertikal. Atribut android:orientation="vertical" dan

app:layout_constraintGuide_percent="0.5" pada baris 37 menempatkannya di tengah lebar layar, membagi ruang untuk kedua tombol secara merata. Selanjutnya, pada baris 39

<com.google.android.material.switchmaterial.SwitchMaterial> dengan

android:id="@+id/darkModeSwitch" adalah sakelar untuk mengaktifkan atau menonaktifkan mode gelap. Lebar dan tingginya wrap_content, memiliki margin atas 8dp, dan padding 8dp. Teks "Aktifkan Dark Mode" ditampilkan dengan warna yang sesuai tema (?attr/colorOnBackground) dan ukuran 16sp. Batasan pada baris 51 menempatkannya di bawah btn_clear_local_data dan sedikit ke kanan dari tengah layar.

Pada baris 53, <ProgressBar> dengan

android:id="@+id/progressBar" adalah indikator *loading*. Lebar dan tingginya wrap_content, dan awalnya visibility="gone" (tersembunyi). indeterminateTint="?attr/colorPrimary" pada baris 58 mengatur warna indikator sesuai tema. Batasan pada baris 62 menempatkannya di tengah layar, di bawah darkModeSwitch.

Pada baris 65

<androidx.recyclerview.widget.RecyclerView> dengan android:id="@+id/rvDrama" berfungsi untuk menampilkan daftar drama. Lebar dan tingginya disetel ke 0dp dengan batasan, memungkinkannya mengisi seluruh sisa ruang di bawah

darkModeSwitch hingga ke bagian bawah layar. `paddingTop="8dp"` memberikan sedikit ruang di bagian atas daftar, dan `tools:listitem` pada baris 74 menunjukkan *layout* item (`@layout/item_drama`) yang akan digunakan untuk pratinjau.

Terakhir, pada baris 76 `<TextView>` dengan `android:id="@+id/tvError"` digunakan untuk menampilkan pesan kesalahan. Lebar dan tingginya `wrap_content`, teks *placeholder* "Terjadi kesalahan!", warna teks `textColor="@color/red_error"`, dan ukuran 16sp. Awalnya juga `visibility="gone"` (tersembunyi). Batasan pada baris 87 menempatkannya di tengah layar, di bawah darkModeSwitch, di posisi yang sama dengan progressBar dan rvDrama, dan akan ditampilkan ketika ada *error*.

24. drama_item.xml

Pada baris 1 `<androidx.cardview.widget.CardView>` adalah elemen utama yang berfungsi sebagai wadah untuk setiap item drama dalam daftar, memberikan tampilan kartu dengan sudut membulat dan elevasi bayangan. Atribut `android:layout_width` disetel ke `match_parent` agar kartu memenuhi lebar yang tersedia, `android:layout_height` ke `wrap_content` agar tingginya menyesuaikan konten, dan `android:layout_margin="8dp"` pada baris 9 memberikan jarak antar kartu. `app:cardCornerRadius="8dp"` dan `app:cardElevation="4dp"` pada baris 10 mengatur radius sudut dan elevasi bayangan kartu. Di dalam `CardView`, pada baris 12, `<androidx.constraintlayout.widget.ConstraintLayout>` berfungsi sebagai tata letak internal kartu, memungkinkan penempatan elemen secara fleksibel. mengisi lebar dan tinggi `CardView` dan memiliki *padding* 16dp di semua sisi. Pada baris 17, sebuah `ImageView` dengan `android:id="@+id/iv_poster"` didefinisikan untuk menampilkan poster drama. Lebar dan tingginya disetel ke 100dp dan 150dp secara

spesifik, dengan `scaleType="centerCrop"` untuk mengisi area tanpa distorsi. `android:src` pada baris 19 menyediakan gambar *placeholder*, dan batasan

```
app:layout_constraintStart_toStartOf="parent",
app:layout_constraintTop_toTopOf="parent",      serta
app:layout_constraintBottom_toBottomOf="parent"
```

pada baris 24-25 menempatkan gambar di sisi kiri kartu dan memusatkannya secara vertikal. Selanjutnya, pada baris 27, `<TextView>` dengan `android:id="@+id/tv_title"` digunakan untuk menampilkan judul drama. Lebarinya disetel ke 0dp dengan batasan horizontal, tingginya `wrap_content`, dan memiliki margin 16dp dari poster serta 8dp dari akhir. Teks *placeholder* "Judul Drama" disetel dengan ukuran 18sp dan gaya bold. Batasan pada baris 37-38 menempatkan `TextView` ini di samping kanan `iv_poster` dan di bagian atas `ConstraintLayout`. Pada baris 40 `<TextView>` berikutnya dengan `android:id="@+id/tv_release_date"` menampilkan tanggal rilis. Lebarinya 0dp, tingginya `wrap_content`, dan memiliki margin yang spesifik. Teks *placeholder* "Tanggal Rilis:YYYY-MM-DD" disetel dengan ukuran 14sp dan `alpha="0.7"` untuk efek sedikit transparan. Batasan pada baris 51-52 menempatkannya di bawah `tv_title` dan di samping kanan `iv_poster`. Dari baris 54, `<TextView>` dengan `android:id="@+id/tv_overview"` digunakan untuk sinopsis drama. Lebarinya 0dp, tingginya `wrap_content`, dan memiliki margin. Atribut `maxLines="3"` dan `ellipsize="end"` membatasi teks hingga tiga baris, menambahkan elipsis jika teks terlalu panjang. Teks *placeholder* "Ini adalah ringkasan singkat dari drama..." disetel dengan ukuran 14sp. Batasan pada baris 65 menempatkannya di bawah `tv_release_date` dan di samping kanan `iv_poster`. Terakhir, pada baris 69, `<LinearLayout>` berfungsi sebagai wadah horizontal untuk tombol-tombol. Lebarinya 0dp dengan batasan, tingginya `wrap_content`, dan memiliki margin atas 8dp serta margin kiri 16dp.

Batasan pada baris 74-75 menempatkannya di bawah `tv_overview` dan di samping `iv_poster`, membentang hingga akhir kartu. Di dalamnya, pada baris 80 sebuah `Button` dengan `android:id="@+id/btn_detail"` didefinisikan dengan lebar 0dp dan `layout_weight="1"` agar mengisi ruang yang tersedia, serta teks "Detail" dan margin akhir 4dp.

25. Main_menu.xml

Pada baris 1 `<?xml version="1.0" encoding="utf-8"?>` adalah deklarasi XML standar. Baris 2 `<menu>` adalah elemen akar yang mendefinisikan struktur menu dalam aplikasi Android. Atribut `xmlns:android` dan `xmlns:app` pada baris 2-3 mendefinisikan *namespace* XML yang diperlukan untuk atribut-atribut Android standar dan atribut khusus dari pustaka dukungan Android (seperti `AppCompat`). Pada baris 4 sebuah `<item>` didefinisikan sebagai entri tunggal dalam menu. Atribut `android:id="@+id/action_dark_mode_switch"` pada baris 5 memberikan identifikasi unik untuk item menu ini. Atribut `android:title="Dark Mode"` pada baris 6 mengatur teks yang akan ditampilkan untuk item menu. `app:actionViewClass="com.google.android.material.switchmaterial.SwitchMaterial"` pada baris 7 menentukan bahwa item menu ini tidak hanya teks, tetapi juga akan menampilkan widget `SwitchMaterial` sebagai tampilan aksinya. Terakhir, `app:showAsAction="always"` pada baris 8 menyarankan agar item menu ini selalu ditampilkan di bilah aplikasi (toolbar) jika ada ruang yang cukup, bukan disembunyikan di menu *overflow*.

26. nav_graph.xml

Pada baris 1 `<?xml version="1.0" encoding="utf-8"?>` adalah deklarasi XML standar. Baris 2 `<navigation>` merupakan elemen akar yang mendefinisikan grafik navigasi aplikasi, yang bertindak sebagai peta untuk menunjukkan bagaimana pengguna dapat bergerak antar

layar (fragment). Atribut `android:id="@+id/nav_graph"` pada baris 5 memberikan pengenalan unik untuk grafik ini, sementara `app:startDestination="@id/dramaListFragment"` pada baris 6 menetapkan `dramaListFragment` sebagai tujuan awal saat aplikasi diluncurkan. Pada baris 8, `<fragment>` pertama dengan `android:id="@+id/dramaListFragment"` mewakili layar daftar drama.

Atribut `android:name="com.example.connectapi.presentation.ui.DramaListFragment"` pada baris 10 menghubungkan tujuan navigasi ini dengan kelas Kotlin `DramaListFragment` yang sebenarnya. `android:label="Series Lawas Populer"` pada baris 11 mengatur judul yang mungkin ditampilkan di bilah aksi aplikasi, dan `tools:layout="@layout/fragment_drama_list"` pada baris 12 adalah atribut yang berguna untuk pratinjau desain di Android Studio. Di dalam fragmen ini, `<action>` pada baris 13 mendefinisikan jalur navigasi yang

mungkin. `android:id="@+id/action_dramaListFragment_to_detailFragment"` pada baris 14 memberi pengenalan unik pada aksi ini, dan `app:destination="@id/detailFragment"` pada baris 15 menunjukkan bahwa aksi ini mengarahkan ke `detailFragment`. Selanjutnya, pada baris 17, `<fragment>` kedua dengan `android:id="@+id/detailFragment"` mewakili layar detail drama. Mirip dengan fragmen daftar,

`android:name="com.example.connectapi.presentation.ui.DetailFragment"` pada baris 19 menautkannya ke kelas Kotlin `DetailFragment`, `android:label="Drama Detail"` pada baris 20 mengatur judulnya, dan `tools:layout="@layout/fragment_detail"` pada baris 21 membantu dengan pratinjau tata letak. Bagian penting dari fragmen detail ini adalah `<argument>` pada baris 22, di mana `android:name="drama"` pada baris 23 menentukan nama data yang

akan diteruskan ke fragmen ini, dan `app:argType="com.example.connectapi.domain.model.Drama"` pada baris 24 mendeklarasikan bahwa argumen ini akan menjadi objek dari tipe `Drama`, memungkinkan penerusan detail drama yang dipilih secara aman dari layar daftar ke layar detail.

27. colors.xml

Pada baris 1 `<?xml version="1.0" encoding="utf-8"?>` adalah deklarasi XML standar. Baris 2 `<resources>` merupakan tag pembuka untuk mendefinisikan berbagai sumber daya, termasuk warna, untuk aplikasi Android. Pada baris 4-6, definisi warna dasar seperti `black` (`#FF000000`), `white` (`#FFFFFFF`), dan `red_error` (`#FFDA4433`), yang sering digunakan sebagai warna fundamental dalam antarmuka pengguna. Dimulai dari baris 8 hingga 35, terdapat serangkaian definisi warna untuk tema terang (`md_theme_light_`) yang mengikuti pedoman Material Design 3 (MD3). Warna-warna ini dikelompokkan berdasarkan peran dalam UI, seperti `primary` dan `onPrimary` untuk warna merek utama dan teks/ikon di atasnya, `secondary` dan `onSecondary` untuk warna aksen sekunder, `tertiary` dan `onTertiary` untuk warna aksen ketiga, serta warna untuk status seperti `error`, `background`, dan `surface`. Ini memastikan konsistensi visual di seluruh aplikasi pada mode terang. Selanjutnya, dari baris 37 hingga 64, definisi warna untuk tema gelap (`md_theme_dark_`) disediakan. Warna-warna ini adalah padanan yang dioptimalkan untuk lingkungan gelap, menawarkan kontras yang sesuai dan mengurangi ketegangan mata. Setiap definisi warna dalam tema gelap memiliki peran yang sama dengan pasangannya di tema terang (misalnya, `md_theme_dark_primary` adalah warna merek utama untuk tema gelap). Penggunaan dua set warna ini memungkinkan aplikasi untuk beralih antara mode terang dan gelap secara dinamis, menyesuaikan tampilan berdasarkan preferensi pengguna atau pengaturan sistem.

28. Themes.xml (light)

Pada baris 1 `<resources>` adalah elemen akar yang menampung berbagai sumber daya dalam aplikasi Android, dan `xmlns:tools` adalah *namespace* untuk atribut *tools* yang membantu pratinjau desain. Baris 3 mendefinisikan sebuah *style* bernama `Base.Theme.ConnectAPI` yang merupakan tema dasar aplikasi. Tema ini mewarisi dari `Theme.Material3.DayNight.NoActionBar`, yang berarti ia mendukung mode terang dan gelap Material Design 3 secara otomatis dan tidak memiliki bilah aksi bawaan. Dari baris 5 hingga 31, terdapat serangkaian `<item>` yang menyesuaikan tema terang aplikasi. Setiap *item* ini memetakan atribut tema generik (seperti `colorPrimary`, `colorOnPrimary`, `colorSecondary`, `colorError`, dan lainnya) ke definisi warna spesifik yang telah ditentukan sebelumnya dalam `colors.xml`, seperti `@color/md_theme_light_primary`, `@color/md_theme_light_onPrimary`, dan seterusnya. Ini memastikan bahwa komponen UI Material Design secara otomatis mengadopsi skema warna yang telah ditentukan untuk tema terang. Pada baris 29, `android:statusBarColor` diatur untuk menggunakan warna `colorPrimary` tema, sementara `android:navigationBarColor` pada baris 31 diatur untuk menggunakan `colorSurface` tema, memastikan konsistensi warna di seluruh bilah sistem (status bar dan *navigation bar*). Atribut `tools:targetApi="1"` pada baris 29 dan 31 menunjukkan bahwa pengaturan ini hanya berlaku untuk API level 21 (Lollipop) ke atas. Terakhir, pada baris 34, `style name="Theme.ConnectAPI"` didefinisikan sebagai tema utama aplikasi, yang cukup mewarisi semua properti dari `Base.Theme.ConnectAPI` yang telah ditentukan sebelumnya. Ini adalah pendekatan umum untuk memisahkan tema dasar yang dapat dimodifikasi dari tema utama yang digunakan oleh aplikasi, memberikan fleksibilitas dalam pengelolaan tema.

29. Themes.xml (night)

Pada baris 1 `<resources>` adalah elemen akar yang menampung berbagai sumber daya dalam aplikasi Android, dan `xmlns:tools` adalah *namespace* untuk atribut *tools* yang membantu pratinjau desain. Baris 3 mendefinisikan sebuah *style* bernama `Base.Theme.ConnectAPI` yang merupakan tema dasar aplikasi. Tema ini mewarisi dari `Theme.Material3.DayNight.NoActionBar`, yang berarti ia mendukung mode terang dan gelap Material Design 3 secara otomatis dan tidak memiliki bilah aksi bawaan. Dari baris 5 hingga 30, terdapat serangkaian `<item>` yang menyesuaikan tema gelap aplikasi. Setiap *item* ini memetakan atribut tema generik (seperti `colorPrimary`, `colorOnPrimary`, `colorSecondary`, `colorError`, dan lainnya) ke definisi warna spesifik yang telah ditentukan sebelumnya dalam `colors.xml`, seperti `@color/md_theme_dark_primary`, `@color/md_theme_dark_onPrimary`, dan seterusnya. Ini memastikan bahwa komponen UI Material Design secara otomatis mengadopsi skema warna yang telah ditentukan untuk tema gelap. Pada baris 31, `android:statusBarColor` diatur untuk menggunakan warna `colorPrimary` tema, sementara `android:navigationBarColor` pada baris 33 diatur untuk menggunakan `colorSurface` tema, memastikan konsistensi warna di seluruh bilah sistem (*status bar* dan *navigation bar*). Atribut `tools:targetApi="1"` pada baris 31 dan 33 menunjukkan bahwa pengaturan ini hanya berlaku untuk API level 21 (Lollipop) ke atas.

30. Build.gradle.kts

Pada baris 1-6, blok `plugins` mendeklarasikan *plugin* Gradle yang diterapkan pada modul ini. Ini termasuk `alias(libs.plugins.android.application)` untuk fungsionalitas aplikasi Android dasar, `alias(libs.plugins.kotlin.android)` untuk dukungan Kotlin, `id("kotlin-parcelize")` untuk otomatisasi implementasi `Parcelable` di Kotlin, dan `id("org.jetbrains.kotlin.plugin.serialization")`.

versi 2.0.0 untuk dukungan serialisasi Kotlin, dan `id("kotlin-kapt")` untuk pemrosesan anotasi Kotlin. Pada baris 8-39, blok `android` berisi konfigurasi spesifik untuk proyek Android. Baris 9 `namespace = "com.example.connectapi"` mengatur *namespace* Gradle, sedangkan `compileSdk = 35` pada baris 10 menentukan versi SDK yang digunakan untuk kompilasi. Dalam blok `defaultConfig` pada baris 12-20, `applicationId = "com.example.connectapi"` adalah ID unik aplikasi, `minSdk = 30` adalah SDK Android minimum yang didukung, `targetSdk = 35` adalah SDK Android target, `versionCode = 1` dan `versionName = "1.0"` adalah informasi versi aplikasi. `testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"` mengatur *runner* untuk pengujian. Pentingnya, `buildConfigField("String", "TMDB_API_KEY", "\"c85206126e56c036e856104caa98a28a\"")` pada baris 19-20 menyuntikkan kunci API TMDB sebagai variabel *build config* sehingga dapat diakses dengan aman dalam kode. Blok `buildTypes` pada baris 22-28 mendefinisikan konfigurasi untuk *build type* yang berbeda. Untuk *release build type*, `isMinifyEnabled = false` menonaktifkan *code minification*, dan `proguardFiles` menentukan aturan ProGuard untuk mengoptimalkan dan mengaburkan kode. Blok `compileOptions` pada baris 30-33 mengatur kompatibilitas *source* dan *target* Java ke `VERSION_11`. Blok `kotlinOptions` pada baris 35-37 mengatur `jvmTarget` untuk Kotlin ke 11. Terakhir, blok `buildFeatures` pada baris 39-42 mengaktifkan `viewBinding` dan `buildConfig`, yang mempermudah interaksi dengan tampilan dan akses ke variabel *build config* yang dibuat otomatis. Pada baris 44-67, blok `dependencies` mendefinisikan semua *library* eksternal yang digunakan proyek. Ini mencakup *library* inti Android seperti `androidx.core.ktx`, `androidx.appcompat`, `material`, `androidx.activity`, dan `androidx.constraintlayout`. Untuk navigasi, ada `androidx.navigation:navigation-fragment-ktx` dan `androidx.navigation:navigation-ui-ktx`. Untuk pemrosesan gambar, `io.coil-kt:coil` digunakan. Untuk pengujian, `junit`, `androidx.junit`, dan `androidx.espresso.core` disertakan. Bagian penting lainnya adalah dependensi terkait basis data dan jaringan: `androidx.room:room-runtime`, `kapt("androidx.room:room-compiler")` untuk pemrosesan anotasi Room, dan `androidx.room:room-ktx` untuk dukungan Kotlin Coroutines dengan Room. Untuk jaringan, ada

```

com.squareup.retrofit2:retrofit,
com.squareup.okhttp3:okhttp,                                dan
com.squareup.okhttp3:logging-interceptor (untuk logging
permintaan/respons HTTP). Untuk serialisasi JSON dengan Kotlin,
org.jetbrains.kotlinx:kotlinx-serialization-json
dan com.jakewharton.retrofit:retrofit2-kotlinx-
serialization-converter digunakan. Akhirnya, untuk dukungan
coroutine dan komponen arsitektur Android, ada
org.jetbrains.kotlinx:kotlinx-coroutines-android,
androidx.lifecycle:lifecycle-viewmodel-ktx,
androidx.lifecycle:lifecycle-runtime-ktx,                    dan
androidx.lifecycle:lifecycle-livedata-ktx.

```

31. AndroidManifest.xml

Pada file ini, menambahkan baris `<uses-permission android:name="android.permission.INTERNET"/>` yang merupakan deklarasi izin (*permission*) untuk memberi tahu sistem Android bahwa aplikasi membutuhkan akses ke jaringan internet. Izin ini sangat penting terutama jika aplikasi memuat konten dari sumber eksternal seperti gambar dari URL menggunakan *library* Glide. Selain itu, juga menyertakan deklarasi `<activity android:name="com.example.viewmodel.MainActivity">` yang dilengkapi dengan `<intent-filter>` untuk menetapkan MainActivity sebagai titik masuk utama aplikasi. Baris `<action android:name="android.intent.action.MAIN" />` menandakan bahwa aktivitas ini akan dijalankan pertama kali saat aplikasi dibuka, sedangkan `<category android:name="android.intent.category.LAUNCHER" />` memastikan bahwa aktivitas ini akan muncul di daftar aplikasi (*launcher*) pada perangkat Android, sehingga pengguna dapat meluncurkan aplikasi melalui ikon di layar utama. Atribut tambahan seperti `android:theme`, `android:icon`, dan `android:label` digunakan untuk menentukan tampilan dan nama aplikasi di perangkat. Sementara itu, `android:allowBackup`, `android:fullBackupContent`, dan

`android:dataExtractionRules` mengatur perilaku pencadangan data aplikasi, dan `android:supportsRtl="true"` memungkinkan aplikasi mendukung tata letak kanan ke kiri (RTL) untuk bahasa tertentu.

TAUTAN GIT

https://github.com/dinzzti/Pemrograman_Mobile.git